



Design and Development of a Small-Scale All-Terrain Ground Vehicle for Autonomous Operation

Miguel Pereira Francisco Pinho

Thesis to obtain the Master of Science Degree in

Mechanical Engineering

Supervisors: Prof. Alexandra Bento Moutinho

Lt. Engr. Artur Jorge Silva Coutinho Machado

Examination Committee

Chairperson: Prof. Duarte Pedro Mata de Oliveira Valério

Supervisor: Prof. Alexandra Bento Moutinho

Member of the Committee: Prof. Nuno Alexandre Antunes Martins Pessanha Santos

July 2025

Dedicated to my special people...

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

I express my sincere gratitude to all involved in the development of this thesis, either directly or indirectly. Firstly Prof. Alexandra Moutinho and Lt. Engr. Artur Machado for the encouragement and crucial guidance throughout difficult challenges.

I extend my gratitude to Bruno Cândido, for his company in the Laboratory and his selfless willingness to help, delivering key insights, specially during robot assembly. I thank Mr. Luís Raposeiro and Eng. Camilo Christo for their endless patience and important practical advice.

I direct my profound appreciation to my dearest friends Diogo Oliveira, João Heleno, Pedro Veiga and Raquel Rodrigues. Sharing this long, difficult journey with them made everything easier.

Finally, I am eternally grateful to my mother Graça, my father Mário and my sister Beatriz. Their unconditional support, patience, and belief in me have been the foundation of my academic journey.

Resumo

Os robôs móveis representam uma tecnologia emergente, com aplicações em diversas áreas, incluindo o setor da defesa. Em resposta a esta tendência, o Exército Português tem investido em iniciativas como o projeto EXE03. Esta tese contribui para esse esforço, apoiando o desenvolvimento de um Veículo Terrestre Não Tripulado (VTNT). Este veículo, projetado à escala, constitui uma plataforma experimental para o desenvolvimento e validação de soluções destinadas a veículos autónomos em ambientes reais.

Os objetivos do projeto foram definidos com base nos requisitos funcionais estabelecidos pelo Fundo Europeu de Defesa para UGVs de uso militar, com foco na teleoperação, navegação por pontos de passagem e prevenção de colisões. Para tal, foi construído de raiz um robô com tração diferencial, baseado na plataforma Devastator e equipado com sensores adequados às funcionalidades pretendidas.

A arquitetura de software desenvolvida é baseada em ROS2 e foi containerizada com recurso ao Docker, assegurando a sua reprodução em diferentes plataformas. Um algoritmo de prevenção de colisões foi desenvolvido, que auxilia a teleoperação através da aplicação de comandos de correção de velocidade, com base na proximidade de obstáculos.

O robô possui capacidade de localização e mapeamento simultâneos (SLAM). Esta funcionalidade foi validada em cenários interiores e exteriores, demonstrando um desempenho preciso de localização e mapeamento.

Foi implementada uma solução robusta de navegação por pontos de passagem, com recurso ao framework Navigation 2. Num cenário exigente, foram avaliados três planeadores locais distintos, com pontos de passagem definidos aleatoriamente. Foram alcançados 60 pontos com sucesso, evidenciando uma navegação autónoma fiável.

Palavras-chave: Veículo Terrestre Não Tripulado, Navegação Autónoma, Evasão de Colisões, Localização e Mapeamento Simultâneos, Robótica Móvel, Todo-terreno

Abstract

Mobile robots are an emerging technology with expanding applications across various fields, including defence. In response to this trend, the Portuguese Army has invested in initiatives like the EXE03 project. This thesis contributes to that effort by supporting the development of an Uncrewed Ground Vehicle (UGV). This vehicle, designed at scale, constitutes an experimental platform for the development and validation of solutions aimed at autonomous vehicles in real environments.

The objectives of the project were defined based on functional requirements outlined by the European Defence Fund for military UGVs, focusing on teleoperation, waypoint navigation, and collision detection and avoidance. A functional differential drive robot was designed and built from the ground up, based on the Devastator platform and equipped with a sensor suite aligned with the defined objectives.

A ROS2 based software framework was implemented and containerized with Docker to ensure reproducibility and scalability across different platforms. A collision avoidance algorithm was developed to assist teleoperation by applying corrective velocity commands, based on obstacle proximity.

The UGV is SLAM capable through the integration of the SLAM Toolbox package. This functionality was validated in both indoor and outdoor environments, exhibiting accurate localization and mapping performance.

A functional waypoint navigation solution was implemented, utilizing the Navigation 2 framework. Three different local planners were tested, in a challenging scenario, where waypoints were assigned randomly. A total of 60 waypoints were reached, demonstrating reliable autonomous navigation.

Keywords: Uncrewed Ground Vehicle, Autonomous Navigation, Collision Avoidance, Simultaneous Localisation and Mapping, Mobile Robotics, All-terrain

Contents

Acknowledgments	vii
Resumo	ix
Abstract	xi
List of Tables	xvii
List of Figures	xix
Nomenclature	xxi
Acronyms	xxiii
1 Introduction	1
1.1 Level of Autonomy of UGVs	1
1.2 Objectives	4
1.2.1 General Objectives	5
1.2.2 Functional Requirements	5
1.2.3 Project Objectives	6
1.2.4 Collision Detection and Avoidance	7
1.2.5 Teleoperation	7
1.2.6 Waypoint Navigation	8
1.3 Thesis Outline	8
2 The Devastator Platform	11
2.1 Robot Platform	11
2.2 DC Motors and Motor Driver	12
2.3 Nvidia Jetson TX2	14
2.4 Active Ranging Sensors	15
2.4.1 LiDAR	16
2.5 Additional Components	18
2.6 Robot Assembly	20
2.7 Robot Development Software - ROS2	23
2.8 Ubuntu Linux and Docker	25
2.8.1 Docker Containerization	25
2.9 Data Visualisation and Simulation Overview	27

2.9.1	Digital Robot Modelling	28
2.9.2	Simulation with Gazebo	29
3	Control, Mapping and Navigation	31
3.1	Low Level Control	31
3.1.1	Arduino Motor Controller	31
3.1.2	Differential Drive Robot Kinematics	33
3.1.3	ROS2 Control	35
3.2	Teleoperation Command	37
3.3	Collision Detection and Avoidance	39
3.3.1	Collision Detection and Avoidance Overview	39
3.3.2	Collision Detection and Avoidance Algorithm Implemented	40
3.3.3	Twist Multiplexer	42
3.4	SLAM	43
3.4.1	SLAM Packages Overview	45
3.4.2	SLAM Implementation	46
3.5	Waypoint Navigation	47
3.5.1	Environment Modelling	47
3.5.2	Path Planners Overview	48
3.5.3	Trajectory Controllers Overview	49
3.5.4	Navigation 2 Framework	50
3.5.5	Plugin Selection and Parameter Tuning	50
4	Experimental Results	57
4.1	ROS2 Control Validation	57
4.2	Obstacle Avoidance Algorithm Testing	59
4.3	SLAM Validation	62
4.3.1	SLAM Map Evaluation	62
4.3.2	SLAM Localization Evaluation	65
4.3.3	SLAM Computational Cost	69
4.4	Waypoint Navigation Validation	70
4.4.1	Path Planner Evaluation	72
4.4.2	Local Planners Evaluation	74
4.4.3	Navigation Computational Cost	75
5	Conclusions	77
5.1	Achievements	77
5.2	Future Work	78
	Bibliography	81

List of Tables

1.1	Levels of Autonomy and Associated Roles [5]	4
1.2	Autonomy level standards for the proposed UGV.	7
2.1	FIT0493 DC Motor Specifications	13
2.2	Motor Driver Schematic Label [11]	14
2.3	Hokuyo URG-04LX-UG01 main specifications.	18
2.4	Main components of the UGV's hardware system.	23
3.1	Supported Robot Types for Navigation Plugins [78].	53
3.2	Supported Robot Types and Tasks for Controller Plugins [78].	55
4.1	Mean ($\bar{x}$), and standard deviation (σ) of CPU and Memory Usage across trial groups.	69
4.2	Mean ($\bar{x}$), standard deviation (σ), median of lateral error, and total path length for each trial.	75
4.3	Mean ($\bar{x}$), Standard Deviation (σ) and Maximum CPU and memory usage across all trials.	76
4.4	Mean ($\bar{x}$) and standard deviation (σ) of CPU and memory usage by the Nav2 controllers.	76

List of Figures

1.1	SAE J3016 Levels of driving automation. From SAE International, 2021 [3].	3
1.2	Contextual Categorisation of RAS, with examples [4].	5
2.1	Devastator Tank Mobile Platform [9]	12
2.2	FIT0493 DC Motor.	13
2.3	Used Motor Driver	13
2.4	Different carrier boards for the Nvidia Jetson TX2.	14
2.5	Schematic drawing of a 2D LiDAR [15].	16
2.6	Range estimation through phase-shift measurement [15].	17
2.7	Hokuyo URG-04LX-UG01 2D laser rangefinder [18].	18
2.8	Robot's components electrical circuit.	21
2.9	Robot 3D CAD.	22
2.10	Custom made auxiliary parts.	22
2.11	The Devastator robot (without radar and camera). Components labelled in table 2.4. . . .	23
2.12	ROS2 node interfaces: topics, services, and actions [26].	25
2.13	Realistic robot representation in both data visualisation tools.	28
2.14	Example of Gazebo worlds.	30
3.1	Control system architecture.	32
3.2	Kinematics of a tracked vehicle [36].	34
3.3	ROS2 Control Architecture [37].	36
3.4	Dualshock 4 gamepad.	38
3.5	Developed Collision Avoidance Algorithm.	41
3.6	Illustration of the sequence of events in SLAM [15].	44
3.7	Different types of robot footprint.	51
3.8	Different approaches to the inflation layer.	52
4.1	Velocity commands vs measured velocity on SLAM trial 18.	58
4.2	Commanded vs measured wheel angular velocities on SLAM trial 18.	58
4.3	Collision avoidance algorithm test scenarios.	59
4.4	Teleoperation and corrective velocity commands on a straight line wall test trial (Fig. 4.3a).	60
4.5	UGV's trajectory during a trial of the straight line wall test (Fig. 4.3a).	61

4.6 Teleoperation and corrective velocity commands on a trial of the narrow corridor test, second variation (Fig. 4.3b).	61
4.7 Outdoor scenario used to test SLAM mapping performance.	63
4.8 Different satellite imagery of the outdoor scenario.	63
4.9 Map obtained with SLAM and alignment with the ground truth.	64
4.10 Obstacle course assembled in the Robotics Arena.	65
4.11 Trajectory comparison: Qualisys vs SLAM Toolbox.	66
4.12 Position and orientation RMSE for the three sets of trials.	67
4.13 Position and orientation throughout trial 26.	68
4.14 Position and orientation error throughout trial 18.	69
4.15 CPU usage throughout trial 34.	70
4.16 Obstacle course assembled to test waypoint navigation.	71
4.17 Global costmap of the obstacle course assembled in the Robotics Arena with waypoints and paths generated in the first trial.	72
4.18 Planned path length across different waypoints and trials.	73
4.19 Average and median planning time in each trial.	73
4.20 Planned path length across different waypoints and trials.	74
4.21 Breakdown of planning, execution, and idle time for each navigation trial.	76

Nomenclature

Greek symbols

ψ	LiDAR phase difference
α	Scaling constant for angular velocity correction in collision avoidance
ω_{cost}	Weight penalizing high costs in Smac Planner 2D
ω	UGV angular velocity around z axis
ω_L, ω_R	Left and right wheel angular velocity
ω_{scale}	Cost scaling factor in the costmap inflation layer
ϕ	Angle of the closest obstacle
ϕ_{max}	Maximum angular bound of collision avoidance
θ	Yaw

Roman symbols

b	Distance between wheels
c	Speed of wave propagation, emitted from an active ranging sensor
C_{travel}	Travel cost between cells in Smac Planner 2D
D	Distance from the LiDAR to an obstacle
d	Distance from an active ranging sensor to the obstacle
d_c	Distance between cells
d_o	Distance to an obstacle in the costmap inflation layer
e	Lateral error
e_{pos}, e_{θ}	Position and orientation errors between SLAM-estimated and ground-truth poses.
f	LiDAR modulation frequency
k	Scaling constant for linear velocity correction in collision avoidance

N	Total number of time instances in a SLAM trial
r	Wheel radius
r_f	Inscribed footprint radius in costmap inflation
t	Time of flight of the wave emitted from an active ranging sensor
f_t, f_r	Frequency of the transmitted/reflected Radar wave
v	Object relative velocity via radar Doppler (m/s).
x, y	2D Pose Cartesian components

Subscripts

gt	Relating to the ground truth
p	Pertaining to the interpolated path in waypoint navigation
$slam$	Relating to SLAM
i	Time instance
j	Zone in the collision avoidance algorithm

Superscripts

$\cdot$	Time derivative
$\wedge$	Corrective velocity command

Acronyms

2D Two-Dimensional

3D Three-Dimensional

ADF Australian Defence Force

ADNN Average Distance to the Nearest Neighbour

CAD Computer-Aided Design

CD Cell Decomposition

DA Dijkstra's Algorithm

DC Direct Current

DDS Data Distribution Service

EDF European Defence Fund

EKF Extended Kalman Filter

FMCW Frequency-Modulated Continuous Wave

GA Genetic Algorithm

GPIO General Purpose Input/Output

GPS Global Positioning System

LiDAR Light Detection and Ranging

LiPo Lithium Polymer

MPC Model Predictive Control

MPPI Model Predictive Path Integral

NATO North Atlantic Treaty Organization

NavFn Navigation Function

PF Potential Field

PID Proportional-Integral-Derivative

PSO Particle Swarm Optimization

PWM Pulse Width Modulation

RA Roadmap Approach

RAM Random-Access Memory

RGB Red Green Blue

RMSE Root Mean Square Error

ROS Robot Operating System

RPP Regulated Pure Pursuit

RRT Rapidly-exploring Random Tree

SLAM Simultaneous Localization and Mapping

SMC Sliding Mode Control

UGV Uncrewed Ground Vehicle

URDF Unified Robotics Description Format

VM Virtual Machine

VP Vector Pursuit

Chapter 1

Introduction

In an era of rapid technological advancements, autonomous mobile robots are becoming increasingly integrated into various aspects of our day to day lives. Their ability to navigate dynamic environments, make real-time decisions, and operate with minimal human intervention has made them indispensable tools in modern applications. With continued developments in artificial intelligence, sensor technology, computer power and communications, mobile robots are poised to play a central role in shaping the future of automation and intelligent systems, including in defence, where Uncrewed Ground Vehicles (UGVs) are increasingly used in a variety of combat and support missions.

A UGV can be described as a comprehensive intelligent system that integrates environmental perception, localization, navigation, path planning, decision-making and motion control [1]. UGVs' capabilities, such as the ability to support military tactics in combat, Intelligence, Surveillance and Reconnaissance, Logistics, and Explosives and Mine Disposal situations, have motivated organizations such as the European Union and the North Atlantic Treaty Organization (NATO) to invest large amounts of resources to develop UGVs, while referencing UGVs as one of the key technologies by 2035 [2].

Portugal, through the Portuguese Army in partnership with Instituto Superior Técnico, launched the EXE03 - Uncrewed Ground Systems Project. One of the work packages within this project is *UGVSecurity*. This master's thesis aligns with this work package, which aims to develop capabilities for a transportable UGV for active security of critical infrastructures and/or detection of objects of interest. The UGV to be developed will serve as a scaled platform for application development and testing, ultimately to be implemented on larger, full-scale vehicles for real-world deployment.

1.1 Level of Autonomy of UGVs

As the field of mobile robotics matures, efforts are being made to standardize definitions to establish clear development benchmarks. When it comes to the level of autonomy of a UGV, the use of concrete and concise definitions is specially important for a variety of reasons. These can be as simple as increasing consistency and clarity to facilitate the comparison between different UGVs and discussions about their capabilities and limitations. One more serious concern that is always present at every step

of UGV development is safety and security. With standardized levels of autonomy, it becomes easier for regulatory bodies to develop and enforce safety standards. This ensures that vehicles meet the necessary safety requirements for each level of autonomy, helping prevent accidents and ensuring that UGVs operate safely in their intended environments.

Recent classifications that contribute to this standardization include the "SAE J3016 - Levels of Driving Automation" by the Society of Automotive Engineers and the "Concept for Robotic and Autonomous Systems" by the Australian Defence Force.

SAE J3016

Introduced in 2014, the SAEJ3016 [3], commonly known as the SAE Levels of Driving Automation has since been the industry's most-cited source for driving automation. The SAE J3016 is used to describe the full range of driving automation features in the context of motor vehicles and their operation on roadways. Even though this is not the prevalent scenario for the vehicle developed in this project, this taxonomy can still be referenced as a good guideline. SAE J3016 defines the SAE Levels from no driving automation (Level 0) to full driving automation (Level 5) as follows:

- Level 0 - No Driving Automation;
- Level 1 - Driver Assistance;
- Level 2 - Partial Driving Automation;
- Level 3 - Conditional Driving Automation;
- Level 4 - High Driving Automation;
- Level 5 - Full driving Automation.

Figure 1.1 presents the visual graph that further details the SAE J3016 Levels of Driving Automation, containing all the latest updates to the recommended practice.

The definition for each level, combined with additional supporting terms and example features, help identify to which level a new feature might belong to.

Each feature present on a UGV can be categorised in one and only one level of autonomy. The levels apply to the driving automation features engaged at any instance of operation. Therefore, while a vehicle may have a driving automation system capable of various features across different levels, the level of automation exhibited at any given moment is determined by the specific features in use.

Australian Defence Force's categorisation

In 2020 the Australian Defence Force (ADF) published its concept for Robotic and Autonomous Systems (RAS) [4], which articulates how the ADF aspires to leverage emerging technology such as Artificial Intelligence, autonomy, and robotics to gain operational advantage. In it, the ADF classifies robotic systems based on four stages of human-system interaction:



SAE J3016™ LEVELS OF DRIVING AUTOMATION™

Learn more here: [sae.org/standards/content/j3016_202104](https://www.sae.org/standards/content/j3016_202104)

Copyright © 2021 SAE International. The summary table may be freely copied and distributed AS-IS provided that SAE International is acknowledged as the source of the content.

		SAE LEVEL 0™	SAE LEVEL 1™	SAE LEVEL 2™	SAE LEVEL 3™	SAE LEVEL 4™	SAE LEVEL 5™
What does the human in the driver's seat have to do?		You are driving whenever these driver support features are engaged – even if your feet are off the pedals and you are not steering			You are not driving when these automated driving features are engaged – even if you are seated in “the driver's seat”		
		You must constantly supervise these support features; you must steer, brake or accelerate as needed to maintain safety			When the feature requests, you must drive	These automated driving features will not require you to take over driving	
Copyright © 2021 SAE International.							
What do these features do?		These are driver support features			These are automated driving features		
		These features are limited to providing warnings and momentary assistance	These features provide steering OR brake/acceleration support to the driver	These features provide steering AND brake/acceleration support to the driver	These features can drive the vehicle under limited conditions and will not operate unless all required conditions are met	This feature can drive the vehicle under all conditions	
Example Features		• automatic emergency braking • blind spot warning • lane departure warning	• lane centering OR • adaptive cruise control	• lane centering AND • adaptive cruise control at the same time	• traffic jam chauffeur	• local driverless taxi • pedals/steering wheel may or may not be installed	• same as level 4, but feature can drive everywhere in all conditions

Figure 1.1: SAE J3016 Levels of driving automation. From SAE International, 2021 [3].

- **Full human control:** A human is responsible for overseeing every aspect of the system's function, either physically or remotely;
- **Human In The Loop:** The system can independently execute certain functions, but depends on a human to perform tasks that complete the system's operational cycle;
- **Human On The Loop:** The system performs all the functions autonomously, however a human operator has the power to halt or reset the process at any time before it reaches completion;
- **Human Starts The Loop:** A human sets the operational parameters and commences the system's operation; from thereafter the process is fully autonomous, not requiring any intervention until its fulfilment.

The ADF also defines the RAS Autonomy Spectrum, where each system is categorized into one of four autonomy levels, depending on its necessity for human intervention. The spectrum levels range from Remotely Operated to Autonomous, as follows:

- **Remote Systems:** Systems which are controlled remotely by a human operator, and whose operation is not feasible when devoid of the remote control element;
- **Automatic Systems:** Systems which are pre-programmed to respond to certain stimuli in an automatic, deterministic fashion. This way they can complete the required tasks without further

human input;

- **Autonomic Systems:** Such systems execute human defined tasks by adhering to previously established guidelines and responding to stimuli in a probabilistic manner;
- **Autonomous Systems:** These systems autonomously determine the methods necessary to complete a certain task. An autonomous system responds to stimuli in a probabilistic fashion and can modify its behaviour accordingly to better complete the objective.

Table 1.1 summarizes the main characteristics of the four levels of autonomy in the RAS Autonomy Spectrum, specifying the human and the machine roles, and the system's autonomous capability.

Table 1.1: Levels of Autonomy and Associated Roles [5]

Level of Autonomy	Human Role	Agent Role	Autonomous Capability
Remotely Operated	Full human control	Human dependent	Actuate platform subsystems: Move steering wheel, push brake
Automatic	Monitor-intervene (human in the loop)	Self-operating	Complete actions: Pre-programmed movements in subsystem
Autonomic	Supervise-task (human on the loop)	Self-managing	Achieve tasks: Interpret information to dynamically achieve individual tasks (autonomous subsystems)
Autonomous	Collaborate-rely (human starts the loop)	Self-governing	Achieve goals: Plan and achieve multiple tasks to reach final objective

Figure 1.2 helps provide context on how these categories are attributed by showcasing some examples. It also frames the four categories through a Technical (capability) context and a Control (use) context.

1.2 Objectives

When designing a UGV, there are a set of standards that must be respected. This project was a request from the Portuguese Army, so the robot is expected to be used for military applications. Portugal is a member of the European Union and NATO, both of which contain guidelines for UGV development.

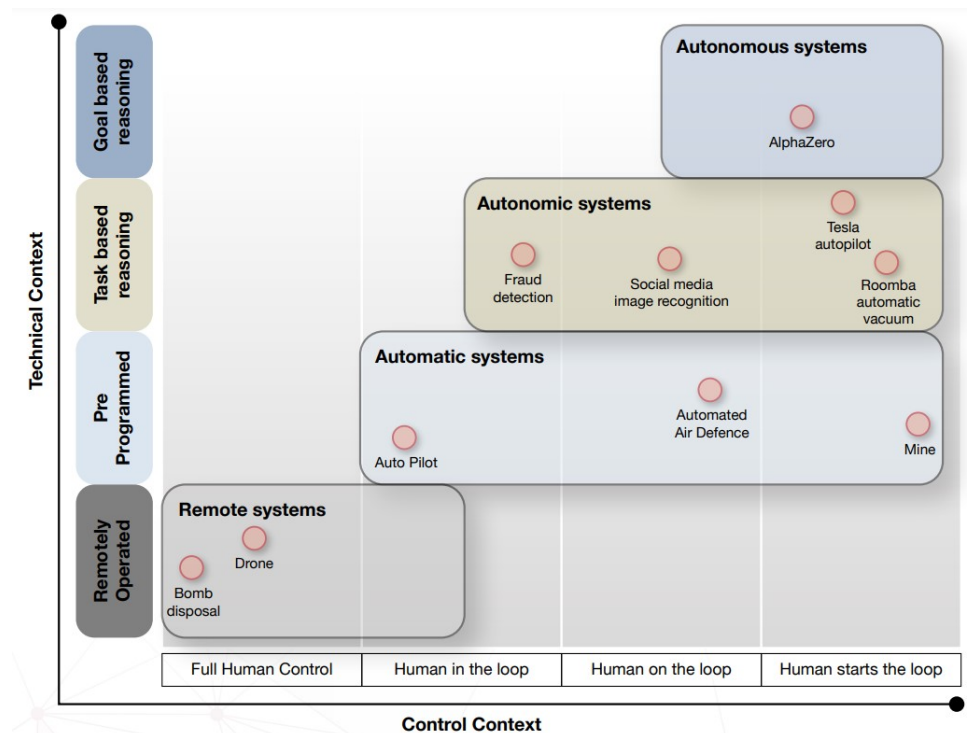


Figure 1.2: Contextual Categorisation of RAS, with examples [4].

In March 2024, the European Parliament and the Council established the Commission Implementing Decision on the financing of the European Defence Fund (EDF). In Annex 3 of this document [6], "Multipurpose Unmanned Ground Systems" are addressed. In this section, it is specified what are the objectives for these systems, and the requisites they must possess to be eligible for funding from this EDF call.

1.2.1 General Objectives

UGVs come in many shapes and sizes, and can perform a myriad of functions. However, despite their varying designs and functionalities, all UGVs are unified by a set of common objectives.

In a military context, the main purpose of a UGV is to enhance overall battlefield effectiveness through intelligent and effective cooperation with other military assets, such as other vehicles, their operators, and dismounted soldiers. When smartly integrated in a framework with other crewed and uncrewed systems, UGVs perform high-risk tasks, which helps minimize loss of life, the risk of collateral damage and the cognitive load placed upon operators. UGVs also support improved Intelligence, Surveillance, Target Acquisition, and Reconnaissance (ISTAR), boost survivability, enhance situational awareness, and increase mobility.

1.2.2 Functional Requirements

The Commission Implementing Decision on the financing of the EDF defends the development of modular components, enabling them to incorporate advanced sensor and effector networks within a stan-

standardized crewed-uncrewed teaming framework. These components will equip UGVs with the capability to operate in connected, hybrid battlefield teams, ensuring a robust and flexible architecture that supports crewed-uncrewed collaboration. Annex 3 of the document in question [6] highlights the following functional requirements:

- The UGV must have the ability to be driven remotely from any position;
- The operator must be assisted by autonomous functionalities with the intent of preventing collisions and reducing collateral damage;
- The UGV should have multiple alternative driving modes, switchable between each other at will. A human operator must also have the ability to suspend the autonomous function and take control of the vehicle at any moment;
- The UGV must possess the capability to navigate effectively in environments where GPS is unavailable;
- The UGV needs to autonomously calculate the most efficient route and plan the path between two locations, actively adapting the path in order to avoid imminent obstacles;
- The UGV should be constantly scanning its environment in order to detect and identify important elements, such as possible threats, allies or civilians;
- The operator must be provided with a decision-making support tool, which grants the user real-time trusted situational awareness;
- A range of sensors should be utilized to ensure access to information from multiple data sources.

1.2.3 Project Objectives

Upon reviewing the mentioned functional requirements, a set capabilities were defined as the objectives of this project, and consequently programmed to be tackled progressively. Those capabilities are:

- Teleoperation
- Waypoint Navigation

There is also a feature to be implemented, whose application is general, being utilized simultaneously with the capabilities above, which is:

- Collision Detection and Avoidance

All these capabilities and feature can be further detailed below and ranked according to their level of autonomy, based on the two classifications described earlier (SAE J3016 and Australian Defence Force's categorisation), summarized in table 1.2.

In addition to these defined capabilities, a key objective of the project was to ensure that the resulting robotic platform would remain adaptable and extensible. With a modular hardware and software

Table 1.2: Autonomy level standards for the proposed UGV.

UGV Capability	SAE J3016	ADF Autonomy Spectrum
Teleoperation	Level 0	Remote System
Collision Detection and Avoidance	Level 2	Automatic System
Waypoint Navigation	Level 3	Autonomic System

architecture, the platform should be designed not only to meet the immediate functional goals but also to support future developments. This flexibility allows for the integration of new sensors, control strategies, or autonomy features with minimal reconfiguration, thereby laying the groundwork for continued research.

1.2.4 Collision Detection and Avoidance

Most real life application scenarios will be known to contain obstacles, or at least have the possibility for them to appear. In such scenarios, it is imperative to have a method for dealing with obstacles so that the robot can proceed with its motion to the desired destination. This is important not only so that the robot can complete its task, but also as a mean of loss prevention. The hardware used may be highly expensive and for such the preservation of its integrity is of the greatest priority. For this reason, collision detection and avoidance will be incorporated into every scenario of control for the robot.

When classifying this feature based on the SAE J3016 one important caveat stands out. What separates a collision avoidance system from being of level 1 or level 2 is whether it provides braking or steering assistance independently, or if it serves both functions. Many collision avoidance algorithms purely decelerate the vehicle in proportion to the proximity to an obstacle, categorising them as level 1. However, the algorithm developed for this project functions by both decelerating and steering the vehicle away from an obstacle, making it of level 2. On the ADF standard, this functionality can be defined as "Automatic", as it reacts to a certain stimulus (proximity to an obstacle), and responds to it in a automatic, deterministic fashion, by altering the vehicle's linear and angular velocities according to that proximity.

1.2.5 Teleoperation

The simplest capability any UGV ought to possess is the ability to be teleoperated, meaning controlled from a distance. Although one would like robots to be as autonomous as possible, in many instances human control is still required. This method of control requires no autonomy at all, since all the velocity commands are given by the operator. These velocity commands are usually provided by a dedicated gamepad, with analogue sticks, although a joystick can sometimes be used, for specific applications.

Teleoperation relies entirely on the human operator and does not possess any autonomous com-

ponent. This means classifying the level of autonomy of this functionality is quite straightforward as it corresponds to the lowest level on both standards. On the SAEJ3016 scale this equates to level 0, and on the ADF categorisation teleoperation can be defined as a "Remote System", with "Full human control".

While traditional teleoperation implies full manual control, it is important to acknowledge that teleoperation can also include varying levels of operator involvement. In more advanced implementations, the operator may provide only high-level commands or reference inputs, while low-level execution and safety mechanisms are handled autonomously by the robot.

1.2.6 Waypoint Navigation

Waypoint Navigation can be described as the robot's locomotion between pre-established geographic points. In this scenario the user is responsible for choosing a sequence of points in which the robot must navigate by. The UGV will then utilize its own position, combined with the surrounding environment, to design a trajectory to the desired waypoint position.

On the SAE J3016 standard, this capability marks the transition from driver support to automated driving. Optimally, such function would correspond to the maximum level of autonomy (level 5). Ultimately, the level of autonomy for waypoint navigation will depend on the robustness of the implemented solution. While a flawless system would qualify as level 5, more realistic solutions could occasionally require the operator to assist the robot, making it of level 3.

On the ADF autonomy spectrum, such system can be classified as "Autonomic" and with "Human On The Loop", since it responds to stimuli in a probabilistic manner, while a human operator has the power to halt or reset the process. In order to avoid imminent obstacles, the robot must actively define its path, making constant adjustments. This requires the system to self-manage and make decisions dynamically rather than following a strictly pre-programmed path, as an "Automatic System" would.

1.3 Thesis Outline

After this introductory chapter, Chapter 2 details the hardware architecture of the robotic platform, describing the Devastator chassis, the motors, sensors, and processing unit, as well as how the components are integrated and physically arranged within the system. In this chapter, ROS2 is introduced as the middleware, while simulation and visualization tools such as Gazebo and RViz, used during development, are also discussed.

Chapter 3 focuses on the software framework, covering the low level control implementation, teleoperation, collision avoidance, simultaneous localization and mapping (SLAM), and autonomous waypoint navigation, all integrated within the ROS2 environment.

Chapter 4 presents the experimental methodology and results, evaluating the robot's performance across several tasks including collision avoidance effectiveness, mapping and localization ability, and autonomous navigation capabilities.

Finally, Chapter 5 summarizes the work carried out, reflects on the results obtained, and outlines possible directions for future development.

Chapter 2

The Devastator Platform

This chapter presents the Devastator robot platform and all the hardware components that comprise it. Section 2.1 introduces the Devastator platform, highlighting its aluminium alloy frame, high-performance suspension, and suitability for housing components. The direct current (DC) motors and motor driver used are presented in section 2.2, followed by the Nvidia Jetson TX2, as the UGV's processing unit, presented in section 2.3. A brief overview on active ranging sensors is provided in section 2.4, where the characteristics of the LiDAR sensor used are presented. Section 2.5 enumerates additional components essential for the UGV's operation. Section 2.6 demonstrates the interaction between the robot's components and explains how various constraints influenced their physical placement, assisted by a 3D CAD model and custom 3D printed mounts. Section 2.7 introduces ROS2 as the middleware, while section 2.8 describes the use of Ubuntu with Docker containerization to resolve ROS2 Humble compatibility issues. Finally, 2.9 covers RViz and Foxglove Studio for data visualization, URDF-based digital robot modelling, and Gazebo for simulation. Access to all the developed software is made available in the robot's Github repository [7].

2.1 Robot Platform

When choosing a robot platform, emphasis was given to its robustness, while ensuring it had a dimension large enough to accommodate all the desired components. Its price was also an important factor, with larger platforms constituting a financial investment that was higher than desired.

With that in mind, the Devastator Tank Mobile Platform [8] was selected (figure 2.1). Although compact, with dimensions of 225*220*108mm, it can house all the necessary components if stacked as layers. It includes a high-strength aluminium alloy frame which contains multiple mounting holes, allowing for the addition of various peripherals, while being lightweight at 780g. It also includes tracks and a high-performance suspension, granting the robot outstanding mobility across even the toughest terrains. The main technical specifications of this robot platform can be seen in [9].



Figure 2.1: Devastator Tank Mobile Platform [9]

2.2 DC Motors and Motor Driver

A DC motor converts electrical energy into mechanical energy. It operates based on the principle of electromagnetism, utilizing the interaction between magnetic fields and electric current to generate motion.

The DC motor used (FIT0493 [10]) contains a 34:1 gearbox and an integrated quadrature encoder. A quadrature encoder is an incremental encoder with two out-of-phase output channels (A and B), where each channel provides a specific number of equally spaced pulses per revolution (PPR), and the direction of motion is detected by the phase relationship of one channel leading or trailing the other channel. The total number of pulses counted corresponds to the angular displacement of the motor, while the pulse frequency provides information on rotational speed. By knowing the wheel radius and gearbox ratio, the encoder data can be used to compute the linear displacement of the robot. This information, when integrated over time, enables odometry calculations, allowing the estimation of the robot's position and trajectory.

Figure 2.2 displays the referenced DC motor and its interface, containing the cable connections necessary for the incorporation of the motors into the electronic system. Table 2.1 enumerates the main technical specifications of the FIT0493 DC motor.

In order to control the speed, direction, and torque of the electric motors, it is necessary to incorporate a motor driver into the system. The motor driver acts as an intermediary between the motor and the control system, translating control signals into the appropriate power outputs to drive the motor, meaning the motor driver and the motor must be compatible.

The motor driver communicates with the motor via Pulse Width Modulation (PWM). PWM generates variable-width pulses, whose on-off pattern can simulate voltages by changing the portion of the time the signal spends on versus the time that the signal spends off. The ratio of on time to off time is called the duty cycle, and determines the percentage of the total voltage that the motor effectively receives.

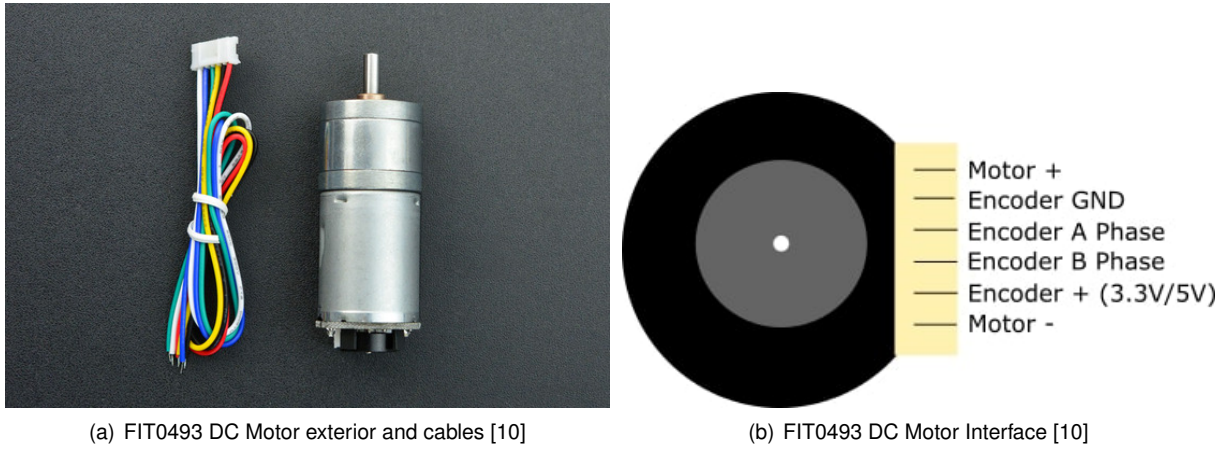


Figure 2.2: FIT0493 DC Motor.

Table 2.1: FIT0493 DC Motor Specifications

Rated Voltage	12V
Gear Reduction Ratio	34:1
No-Load Speed	350 rpm @ 12V
No-Load Current	0.23A
Stall Current	5.5A
Stall Torque	12 kg.cm
Hall Feedback Resolution	374
Weight	98g

The motor driver in [11] was utilized. Figure 2.3 displays said motor driver and its schematic. The overview of its possible connections can be seen in table 2.2.

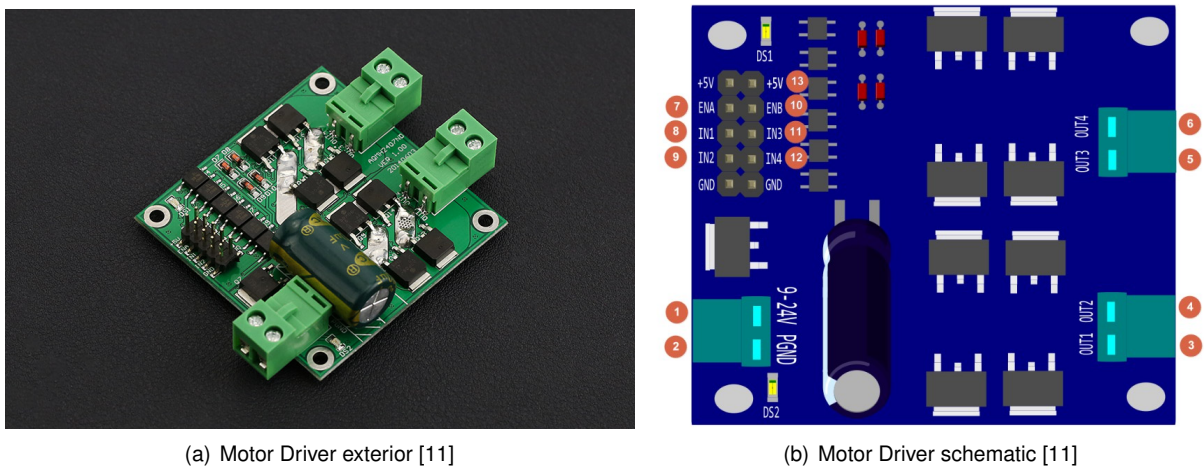


Figure 2.3: Used Motor Driver

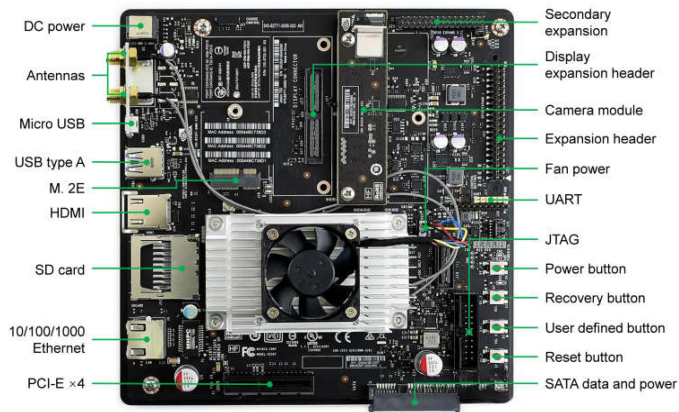
This layout allows for the control of the motors by manipulating the signal in the pins IN1 to IN4 and ENA/ENB using the logic in [11]. Pin number 13 and its twin correspond to a built in +5V regulator, which are used to provide a 5V connection to the Arduino and to power one motor encoder.

Table 2.2: Motor Driver Schematic Label [11]

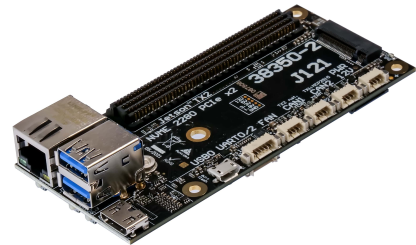
Num	Label	Description
1	9 - 24V	Power Supply, +
2	PGND	Power Supply, Ground
3	OUT1	Motor1 +
4	OUT2	Motor1 -
5	OUT3	Motor2 +
6	OUT4	Motor2 -
7	ENA	Motor1 PWM
8	IN1	Motor1 Control Signal
9	IN2	Motor1 Control Signal
10	ENB	Motor2 PWM
11	IN3	Motor2 Control Signal
12	IN4	Motor2 Control Signal
13	+5V	Voltage Reference Input, +5V or 3.3V

2.3 Nvidia Jetson TX2

The UGV requires an onboard computer to perform the system-wide applications. These can be strenuous computationally, making it indispensable for the computer in question to contain a high degree of computational power. While capable of high performance, such computer must also be small enough to be mounted on and carried by the robot platform. These conditions narrow down the possible candidates to either a Raspberry Pi or an Nvidia Jetson variant. The Nvidia Jetson TX2 module computing platform was selected, given its availability in the faculty laboratory.



(a) Jetson TX2 Developer Kit [12].



(b) Auvideo J121 carrier board [13].

Figure 2.4: Different carrier boards for the Nvidia Jetson TX2.

The NVIDIA Jetson TX2 in question was available as a Developer Kit [14] (figure 2.4a), which includes both the Jetson TX2 module and a carrier board. The Jetson TX2 module is a small, high-performance computing unit integrating an NVIDIA Pascal GPU with 256 CUDA cores, a 6-core ARM CPU, 8GB of LPDDR4 memory, and 32GB of eMMC storage. However, since the module alone lacks the necessary interfaces for direct use, the carrier board provides essential connectivity options, including HDMI, USB, Ethernet, and General Purpose Input/Output pins (GPIOs). This board was used to flash the Jetson TX2 and for early development.

The portability of the Jetson TX2 is granted by the J121 carrier board produced by Auvideo [13]. This board provides many of the same capabilities as the Nvidia's developer kit, only more compact and lighter, turning the TX2 into a mini-computer, ideal for integration onto mobile robots such as UGVs. It supports various peripheral connectivity options, including USB, Ethernet and HDMI, facilitating integration with sensors and peripherals.

However, the use of this carrier board comes with some downsides. The major one is that this board does not contain the GPIO (general-purpose input/output) pins, present in the developer kit, which are essential for providing the PWM signals to the motor driver and for receiving the encoder data. This creates the necessity for an intermediate board, like an Arduino, that can perform those duties and communicate the required information via USB.

2.4 Active Ranging Sensors

Active ranging sensors remain the preferred choice for mobile robotics due to their widespread availability, affordability, and straightforward outputs, providing direct distance measurements from the robot to nearby objects. As visual interpretation capabilities improve, active ranging sensors may gradually become less dominant among mobile robotics, however, in the meantime, they remain a crucial tool.

This type of sensor resorts to time-of-flight ranging to calculate the distance to the obstacles in the vicinity. These sensors emit a wave (radar, sound, electromagnetic) and measure the time it took for it to return. Knowing the propagation speed of the emitted wave, one can calculate the distance it travelled, and therefore the distance to the detected obstacle:

$$d = \frac{c \cdot t}{2}, \quad (2.1)$$

where: d = distance to the obstacle; c = speed of wave propagation; and t = time of flight. The division by 2 accounts for the round-trip of the wave's propagation, as the measured time of flight corresponds to the wave travelling to the obstacle and back.

Active Ranging Sensors are not infallible, and can be associated with several problems, of which the following are highlighted ([15],[16]):

- External noise sources, electromagnetic interference, or signal distortion can corrupt the reflected signal, making it challenging to accurately determine the time of arrival;

- The high speed of electromagnetic signals (0.3 m/ns) create a minuscule time-of-flight which can be technologically challenging to accurately measure;
- The physical characteristics of the target vary the way the wave interacts with it. Some materials absorb or attenuate the emitted signal. Smooth surfaces such as mirrors, polished metals, or glossy objects can induce the emitted signal to reflect off the surface. In environments with multiple reflective surfaces or complex geometries, the emitted signal may undergo multiple reflections before returning to the sensor, causing signal distortion, interference and delays;
- The speed of signal propagation can vary depending on the properties of the medium through which the signal travels. For example, in air, the propagation speed of sound waves may be affected by factors such as temperature, humidity, and air density. This variation in the signal propagation speed may lead to an inaccurate calculation of the real distance to the target;
- The speed of the mobile robot and the target (in the case of a dynamic target) can also influence the measurements.

2.4.1 LiDAR

LiDAR, an acronym of "light detection and ranging", is the most powerful active ranging sensor, as it emits a collimated laser beam, the type of wave with the highest speed. LiDARs may operate in a fixed direction or in multiple directions, in the case of 3D LiDARs. As figure 2.5 shows, the mechanism contains a rotating mirror, sweeping the laser beam to cover the surrounding environment.

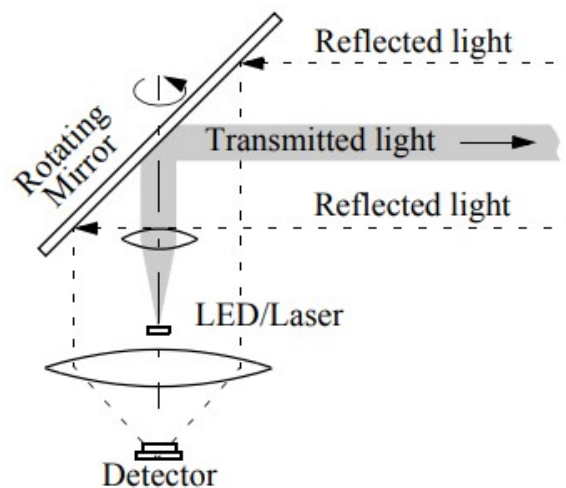


Figure 2.5: Schematic drawing of a 2D LiDAR [15].

There are three main methods for measuring the time of flight in LiDARs [15]:

- Using a pulsating beam, and measuring the elapsed time. These values are in the order of picoseconds, and the equipment able to measure these minuscule time differences is very expensive;

- In Frequency-Modulated Continuous Wave (FMCW) LiDARs, a laser emits a continuous wave that is frequency-modulated over time. Due to the time delay caused by the distance, the light travelled to the object and back, there is a frequency difference between the transmitted and reflected signals, known as the beat frequency, which is directly proportional to the elapsed time;
- Measuring the phase shift of the reflected light. This method is further explained in detail, since it is the approach the LiDAR employed in this project employs.

Consider a LiDAR transmitter emitting a collimated beam that hits point P of an object at a distance D , as exemplified by figure 2.6a, and the phase-shift it induces between the transmitted and reflected light beams (figure 2.6b).

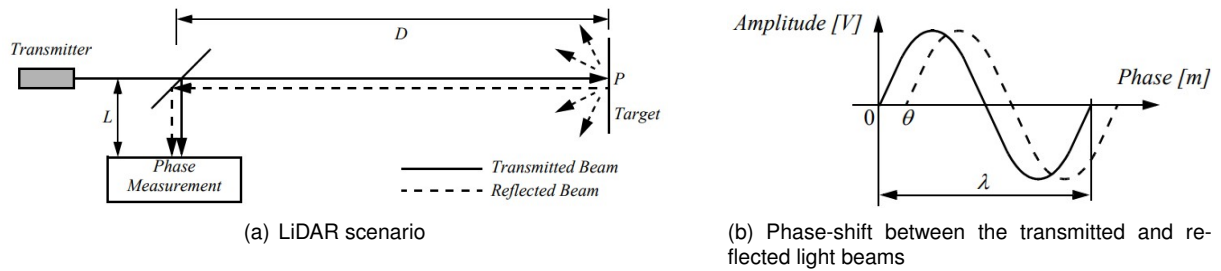


Figure 2.6: Range estimation through phase-shift measurement [15].

While highly polished reflecting objects, may induce specular reflections, forcing the signal away from the detector, most surfaces will induce diffuse reflection as the wavelength of the incident light is smaller than the roughness of the surface. The sensor emits amplitude-modulated light at a specific frequency and then measures the phase shift between the emitted and the reflected signals to determine distance D , resorting to:

$$D = \frac{\psi c}{4\pi f}, \quad (2.2)$$

where ψ is the electronically measured phase difference between the transmitted and reflected light beams in radians, as seen in figure 2.6b, and f is the modulation frequency in Hz [17]. One detail to note is that in theory this method can result in ambiguous range estimates, since phase angles 360° apart induce the same distance calculation. However, in practice, this is almost never an issue since a signal with more than 360° phase-shift can only be detected at a range that is unattainable for most sensors.

The 2D LiDAR utilized in this project was the Hokuyo URG-04LX-UG01 [18], shown in figure 2.7. Of its many specifications, the following are of note: The light source of the sensor is infrared laser of wavelength 785 nm with laser class 1 safety. Scan area is 240° semicircle with a maximum radius 4000 mm. Pitch angle is approximately 0.352° ($360^\circ/1024$) and sensor outputs the distance measured at every point (683 steps). Laser beam diameter is less than 20 mm at 2000 mm with maximum divergence 40 mm at 4000 mm. Its principle of distance measurement is based on calculation of the phase difference. Table 2.3 summarizes its main properties. The full specification list can be found at [19].

With dimensions of $50 \times 50 \times 70$ mm and a weight of approximately 160 g it can be classified as compact and lightweight, ideal for a mobile robot, specially of small proportions. Ultimately its selection for this project, out of all the possible alternatives, was due to availability in the laboratory.



Figure 2.7: Hokuyo URG-04LX-UG01 2D laser rangefinder [18].

Table 2.3: Hokuyo URG-04LX-UG01 main specifications.

Power source	5V DC $\pm 5\%$ (Supplied by USB bus power)
Current consumption	500 mA or less (Rush current 800 mA)
Wavelength	785 nm
Detectable range	20 - 5600 mm
Accuracy	0.06-1 m: ± 30 mm*, 1-4 m: 3% of the detected distance
Scan Angle	240°
Angular Resolution	Approx. 0.352° (360°/1024)

2.5 Additional Components

In addition to the main components, a set of auxiliary components had to be utilized in order for the system to function correctly. These include:

LiPo Batteries

As a mobile platform, the robot requires a portable source of power, i.e., batteries. The most suitable and therefore most commonly used batteries in robotics are: Li-Ion (Lithium-ion), LiPo (Lithium polymer) and NiMH (Nickel-metal hydride). LiPo batteries are a subset of Li-Ion batteries, being that the main difference relies on their casing, as their chemical properties are similar. While Li-Ion batteries are typically encased in a cylindrical metal can, LiPo batteries can be produced in pouch cells. This makes them lighter, therefore increasing the power-to-weight ratio. However, the absence of a metal external casing makes LiPo batteries more prone to mechanical damage and consequently safety incidents. LiPo batteries also generally offer higher discharge rates, making them slightly better suited for robotics. NiMH batteries are safer compared to lithium-based cells, however, their energy-to-weight ratio is notably inferior. Considering these factors, the decision was made to use LiPo batteries in this robot platform.

The next step was selecting how many batteries to utilize in the vehicle. Using just one battery

would be preferable for simplicity, and as to minimize the weight and volume used. This, however, is not recommended, as the Jetson TX2 and the motor driver should be powered by different power sources. Motor drivers and motors draw high currents which can lead to voltage drops and electrical noise in the shared power supply line, especially when motors start, stop, or change direction. This can influence the power being provided to the Jetson TX2, causing brownouts. For this reason, two batteries were utilized to power this robot system: one for the motor driver and motors; other for the Jetson TX2 and the peripherals. In this setup, one might question if the peripherals could also influence the Jetson TX2 behaviour. In practice, this interference is negligible, as the power spikes caused by the Camera, the Radar and the LiDAR that occur during high data transmissions are predictable and minor.

Arduino Duemilanove

As mentioned earlier, the absence of GPIO pins in the Nvidia Jetson induces the need for an intermediate board to provide PWM signals to the motor driver and to receive encoder data. For this effect, an Arduino was used, more specifically the Arduino Duemilanove [20], as it was available in the laboratory. This model contains all the necessary pins and does not require soldering to a board, like some smaller models.

Powered USB Hub

The Auvideo carrier board possesses two USB ports through which peripherals can be powered. At least five components need to be attached to the Auvideo Board (Arduino, LiDAR, Radar, Camera, Wi-Fi dongle), which creates the need for a USB Hub to accommodate all of these.

USB hubs are available in both bus-powered and externally powered configurations. To select the appropriate type, it is crucial to assess the total current requirements of the connected peripherals. In this system, the combined current draw of the LiDAR and the camera alone reaches approximately 1A, potentially exceeding this value due to inrush currents during startup. Additionally, the radar module contributes significantly to the overall power demand. The Auvideo board supplies power to peripherals via a USB 3.0 interface, which is limited to a maximum current of 900 mA. Given that the cumulative load exceeds this limit, the use of an externally powered USB hub becomes necessary. The hub connects to the Auvideo board via a standard USB interface for data transmission, while its power is independently supplied from a dedicated battery source, thereby ensuring stable and sufficient current delivery to all connected devices.

Step Down Converter

While the LiPo batteries supply power at around 12V, the robot system operates at two different voltages:

- **12V:** Motor Driver; Motors; Nvidia Jetson.
- **5V:** USB Hub; LiDAR; Radar; Camera; Arduino.

To accommodate components operating at 5V, the voltage supplied by the main power source must be stepped down from 12V. This is accomplished using a DC-DC step-down (buck) converter, which accepts a higher input voltage and provides a regulated 5V output. When selecting an appropriate converter, it was critical to ensure that its current handling capacity exceeded the total current requirements of all connected 5V components, thereby guaranteeing reliable and stable power delivery under all operating conditions.

Radar

The mapping and navigation solutions implemented in this project did not require the use of a radar sensor. One was assembled nonetheless, given the incorporation of the Devastator robot platform in other projects, most notably in a project by Tiago Silva, developed simultaneously, which utilized a radar sensor and a RGB camera for people and vehicle detection. The model used assembled in the robot platform was the AWR1642 [21], integrated in a sensor evaluation board AWR1642BOOST [22], both manufactured by Texas Instruments.

RGB Camera

As with the radar sensor, the solutions utilized in this project did not require a RGB camera, but one was incorporated due to the use of the robot platform in other projects. The RGB camera used was the Logitech Brio 4K Pro Webcam [23]. The Brio 4K Pro is powered by a USB Type-C cable and allows for multiple resolutions: 4K/30fps, 1080p/60fps and 720p/90fps. It also possesses a 65°, 78°, or 90° diagonal adjustable field of view and a 5x digital zoom. The full specification list can be found at [23].

2.6 Robot Assembly

The diagram in figure 2.8 illustrates the circuit that connects the robot's components.

Beyond respecting the connections depicted in figure 2.8, many constraints had to be followed when designing a robot architecture that fully accommodates all its components:

- The motor driver and the motors must be adjacent given the high number of cables connecting both;
- The batteries must be easily accessible in order to be readily swapped;
- The LiDAR must be placed centrally and at the front of the vehicle and not blocked in its 240° scan area by any other robot part;
- The Radar must be placed centrally and not obstructed;
- The Camera must be placed above and in line with the Radar and not obstructed;
- The robot's mass must be evenly distributed across its body, at least roughly, as to not provoke tilting in any direction.

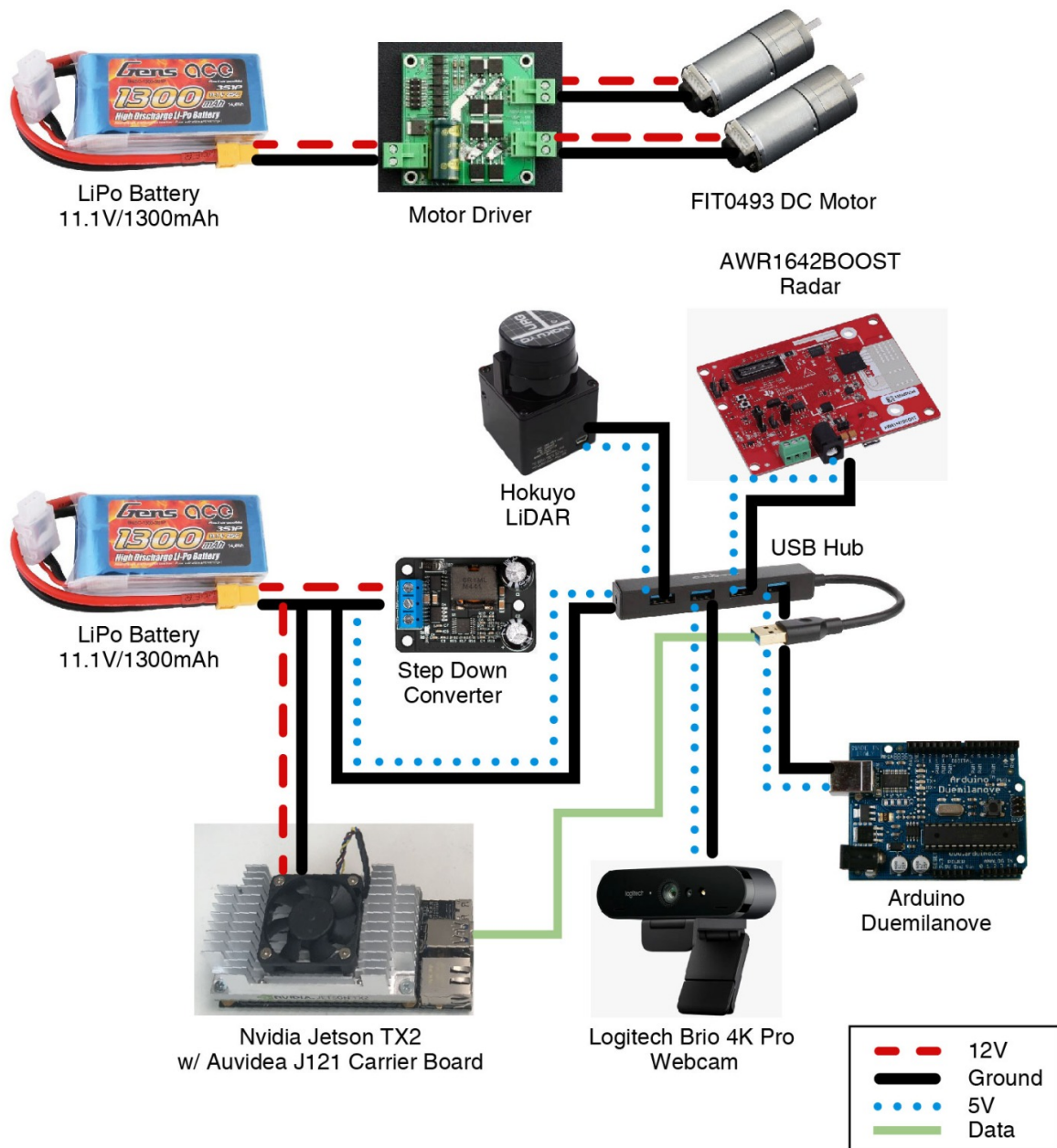


Figure 2.8: Robot's components electrical circuit.

Given the high number of components and physical constraints, a 3D CAD model of the robot was constructed in Solidworks [24] to aid in the architecture design (figure 2.9).

The Devastator platform contains an array of mounting holes. These are restricting if used directly to connect the external components. The better option is to construct custom parts that use these perforations to attach to the platform. As they are custom made, these parts can then contain the holes and pins necessary to perfectly house the external components. Three parts were designed and 3D printed with this exact intent (figure 2.10): motor driver mount; upper level mount; Camera and Radar mount.

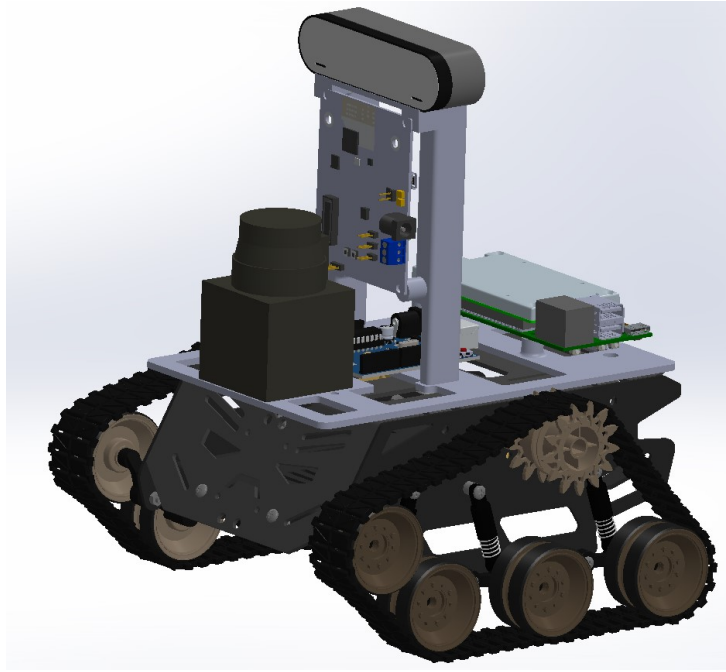
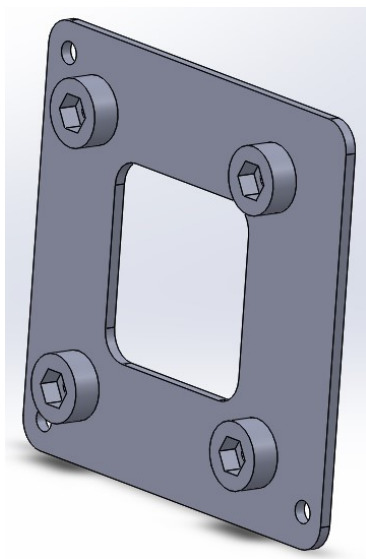
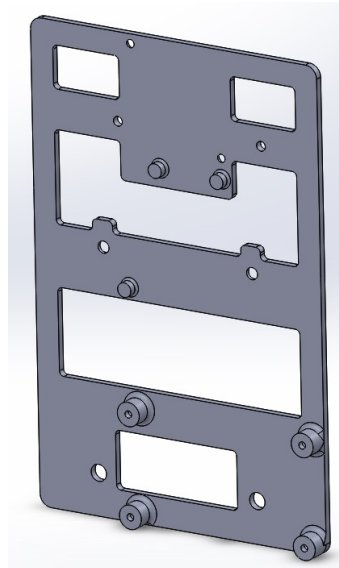


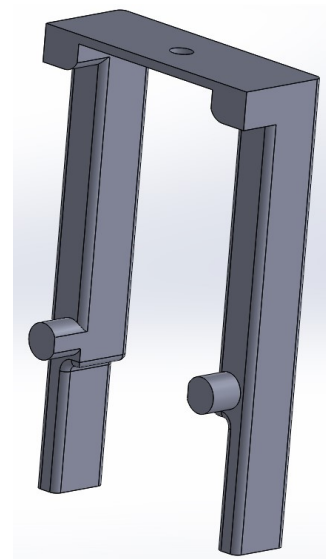
Figure 2.9: Robot 3D CAD.



(a) Motor driver mount.



(b) Upper level mount.



(c) Camera and Radar mount.

Figure 2.10: Custom made auxiliary parts.

Figure 2.11 showcases the final assembly of the UGV, from multiple views. The camera and radar are not shown as they were being utilized in a concurrent project at the time. The motors and motor driver are placed inside the Devastator platform, and therefore are not visible in the pictures. Table 2.4 accompanies figure 2.11 by labelling the UGV components.

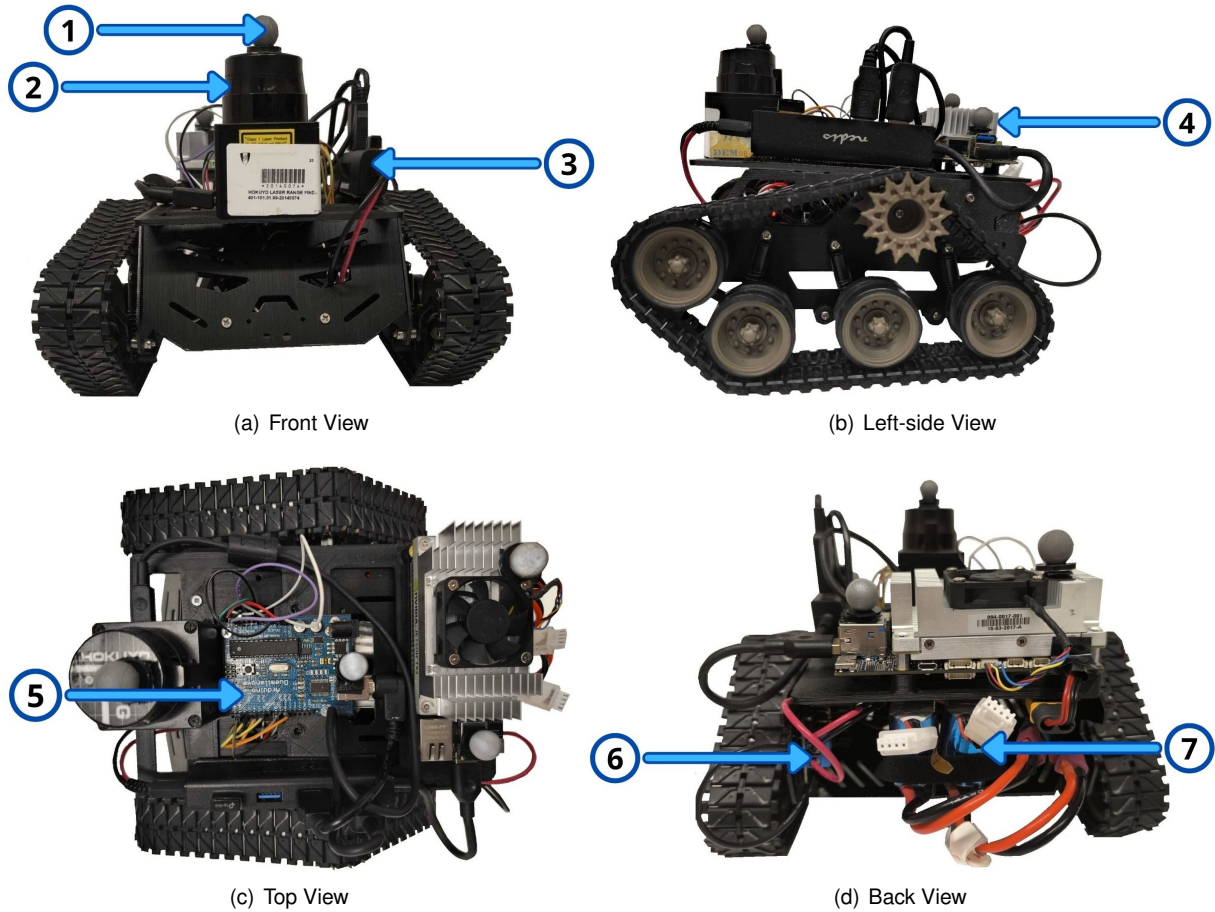


Figure 2.11: The Devastator robot (without radar and camera). Components labelled in table 2.4.

Table 2.4: Main components of the UGV's hardware system.

Identifier	Component Name
1	Reflective Marker
2	Hokuyo URG-04LX-UG01 LiDAR
3	USB Hub
4	Nvidia Jetson TX2 w/ Auvideo J121 Carrier Board
5	Arduino Duemilanove
6	Step Down Converter
7	LiPo Batteries

2.7 Robot Development Software - ROS2

Throughout the development of robotics, many software platforms, or middlewares, have been proposed to help build robot systems. No middleware had as much impact on the field as the Robot Operating System (ROS1) [25]. ROS1 was originally developed as a research platform, which caused limitations as companies began to use it on commercial products. Security and reliability problems, combined with a lack of support for large scale embedded systems, caused many companies to modify ROS1 or create

alternative applications altogether. In order to address these concerns, the next generation of the Robot Operating System, ROS2 [26], was designed. ROS2 is open source and grants users rights to modify, apply, and redistribute the software, with no obligation to contribute back. The design of ROS2 is guided by several fundamental principles [26]:

- **Distribution:** ROS2 allows robotic applications to be built using a distributed system approach. By separating components such as device drivers, control systems, and perception modules, each part can operate independently, promoting decentralization;
- **Abstraction:** Is achieved by defining interfaces that govern the communication between components. These abstract interfaces allow for easier substitution of components without affecting the overall system, fostering an ecosystem of interoperable modules;
- **Asynchrony:** ROS2 enables components to communicate asynchronously through an event-based system. This approach is particularly beneficial in robotics, where components such as sensors and actuators operate at different frequencies, requiring flexible communication patterns;
- **Modularity:** Each component, whether a library, API, or tool, is designed to perform a specific function well, allowing for extensive reusability and scalability.

The core communication patterns in ROS2 are organized around the concept of a node, which serves as a fundamental building block. Nodes communicate with each other through three primary mechanisms:

Topics enable asynchronous message-passing between nodes using a publish-subscribe model. This allows for many-to-many communication, where any number of publishers can send messages to multiple subscribers without direct interaction. This pattern is particularly useful for real-time data streams such as sensor data.

Services follow a synchronous request-response communication pattern. This is useful when a direct interaction between nodes is required, such as when a task needs to be confirmed as completed.

Actions are used for long-running tasks that require periodic feedback or the ability to be cancelled, such as robot navigation or object manipulation. The action pattern extends the service model by adding feedback and result communication, making it ideal for tasks that involve complex interactions.

Figure 2.12 illustrates how nodes interact using these mechanisms.

ROS2 architecture is built upon a series of abstraction layers. At the core is the rmw (ROS MiddleWare) interface, which serves as an abstraction layer between the client libraries and the underlying middleware, typically Data Distribution Service (DDS). Above the rmw layer lies the rcl (ROS Client Library), a C-based library that provides essential communication interfaces, including topics, services, and actions. ROS2 integrates security features directly into its core, utilizing the DDS-Security standard, whose three main components include authentication, access control and encryption.

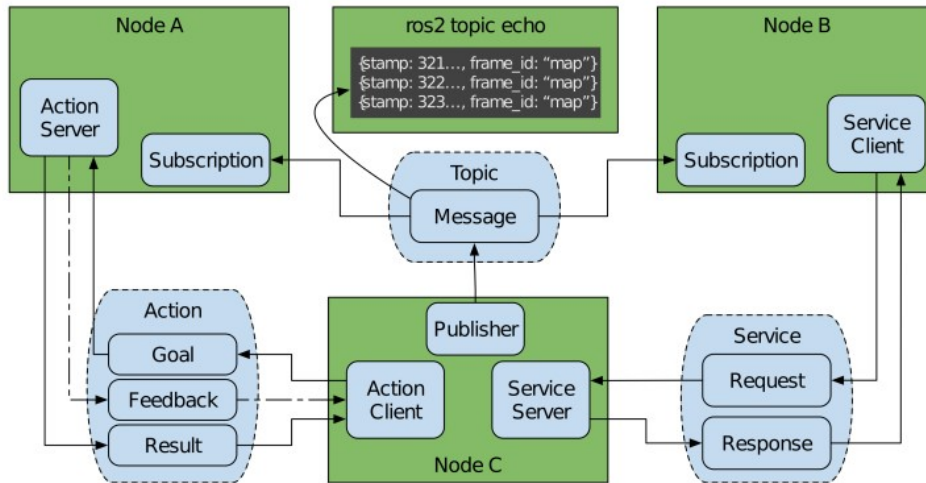


Figure 2.12: ROS2 node interfaces: topics, services, and actions [26].

2.8 Ubuntu Linux and Docker

The computing hardware of the robot's setup includes a host computer, in addition to the Jetson TX2. The host computer is a necessity, as it provides the interface through which the operator commands and visualises the robot's tasks. This computer is also used as the development machine, to create the programs and run simulations. ROS2 was primarily developed for Linux-based systems, with Ubuntu being the most popular and widely used distribution. This popularity grants Ubuntu with a large and active community of users and developers, making it easy to find help and support for ROS development. Therefore, the Linux distribution utilized in the host computer was Ubuntu 22.04.5 as it was the newest version at the time the development commenced. Since the host machine is a Windows laptop, Ubuntu had to be installed and used through a virtual machine (VM). Partitioning the disk would also be an option, but was deemed impractical when compared to simply using a VM. On the Jetson TX2, Ubuntu 20.04.6 is the most recent distribution compatible. This creates a compatibility issue, as the newest ROS2 version at the time of development (Humble Hawksbill or simply Humble) is only supported by 22.04 or newer versions. While an older version of ROS2 could technically be used, this is not an optimal alternative as any developed applications would not perform to the limit of their potential. Additionally, many of the external packages utilized are also updated to be used with ROS2 Humble only.

2.8.1 Docker Containerization

The solution to this problem was found in containerization using Docker [27], which combines the application's code, related dependencies, and system libraries in the form of standardized units called containers. Additionally, Docker helps mitigate the technical challenges to reproducibility of the code behind the applications, that is, the ability to easily employ the developed features in different robot platforms and computing hardware. Docker containers are created using Docker images. These serve as the blueprint for creating containers, containing all the necessary files, libraries, dependencies, and

instructions to run a specific application. One way to access useful images is through Docker Hub, a registry built for developers and open source contributors to find, use, and share their container images. However, custom made applications require specially made Docker images. This is accomplished by writing a Dockerfile, a simple script that defines exactly how to build up the image. Contrary to images, which can be quite large (gigabytes), a Dockerfile is just a small plain text file, which means it can easily be stored and shared. A Dockerfile was created in this project, as to create a Docker image and consequently a container consisting of the entirety of the ROS2 workspace. It follows standard procedure, which includes the following steps:

1. **Specify the base image:** The build extends the official `arm64v8/ros:humble-ros-core` image, which provides a minimal ROS2 Humble installation on an ARM64 architecture.
2. **Set up essential environment variables:** Several environment variables are defined to ensure a smooth setup:
 - `DEBIAN_FRONTEND=noninteractive` prevents interactive prompts during package installations.
 - `COLCON_WS=/proj_ws` sets the workspace directory.
3. **Install system dependencies and ROS2 packages:** The Dockerfile commands the installation of essential system tools and ROS2 packages, including:
 - Core utilities like `nano`, `git`, `cmake`, `build-essential`.
 - Python tools such as `python3-pip`, and `python3-vcstool` for workspace management.
 - ROS2 packages including `ros-humble-navigation2` and `ros-humble-slam-toolbox`.
 - Additional tools like `usbutils` and `x11-apps` for hardware and graphical support.
4. **Create and configure the ROS2 workspace:**
 - The directory `$COLCON_WS/src` is created to serve as the workspace.
 - The source files are copied from the local machine into the workspace.
5. **Create a non-root user for security:**
 - A new user named `ros` is created with a user ID of 1000.
 - Passwordless `sudo` access is granted to the newly created user
6. **Source the ROS2 Humble setup script and build the workspace:** using `colcon build`.
7. **Persist environment settings and define the default command:**
 - The workspace's `setup.bash` is added to `~/.bashrc` so that it is automatically sourced in new shell sessions.
 - The default command is set to `bash`, so that the container starts, with an interactive shell.

Once the Docker image is built, it can be used to create and start a container. This is achieved using a `run` command in which multiple important conditions are specified, such as:

1. `--network=host`: This option allows the container to use the host machine's network stack. Instead of creating a separate virtual network, the container shares the same network interfaces as the host, which facilitates ROS2 communication.
2. `-v /home/nvidia/proj_ws:/proj_ws`: This flag mounts the local directory `/home/nvidia/proj_ws` into the container at `/proj_ws`, which ensures that any modifications to the workspace persist outside the container.
3. `--device=/dev/ttyUSB0 --device=/dev/ttyACM0`: These options provide direct access to specific serial devices from the host system, such as:
 - `/dev/ttyUSB0`: Used for USB-to-serial adapters.
 - `/dev/ttyACM0`: Used for the Arduino board.

The run command effectively launches a ROS2 environment inside a Docker container while maintaining access to network interfaces, graphical displays, input devices, and serial communication ports, made possible by specifying the environment variables.

2.9 Data Visualisation and Simulation Overview

At the basis of robotics development is the ability to visualise and manage the data regarding the robot, more specifically the data from its various sensors. This data is what grants the operator the understanding of the state of the robot and its surroundings. Therefore, an effective visualisation tool is required, with the ability to receive data streams from the robot multiple sensors, preferably via ROS topics and display this data in a user friendly and interactive manner. Most importantly, such tool must provide a 3D visualisation environment, capable of reproducing the 3D scene that surrounds the robot, when given the appropriate sensor data. Such tool should also be configurable, so that the user can select exactly what data to monitor, and with what priority, by resorting to adjustable markers.

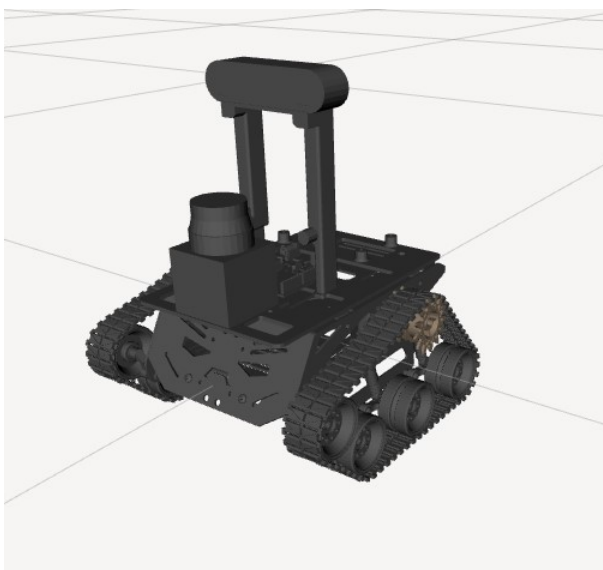
The two most popular data visualisation tools that satisfy the requirements mentioned are RViz [28] and Foxglove Studio [29]. Both tools were used in this project, depending on the specific requirements of each task, as each has distinct strengths. Foxglove is superior when it comes to the clarity of data presentation. It offers modular visualisation interfaces called panels, that the user can arrange into the layout as it pleases. These panels can include, for example: the live image feed received from the robot's camera; the 3D visualisation of the scenario; graph of a ROS topic throughout time, etc. An advantage of Foxglove over RViz is that it does not require ROS2 to be installed and functional. It comes accessible as a stand-alone web app and desktop app, available for macOS, Windows and Linux. It connects remotely to the machine running the robot and receives the ROS2 topics directly. This is extremely helpful when working on multiple platforms. RViz is more established and its popularity has incentivized the creation of useful third party extensions. The practicality of the SLAM Toolbox and Nav2 extensions alone was reason enough to use RViz in this project.

2.9.1 Digital Robot Modelling

At the centre of any 3D visualization scene is the virtual representation of the robot. To ensure accurate simulation and system integration, this virtual model should closely reflect the real robot, if not in visual detail, then at least in its functional characteristics. Some software components also require information about the physical properties of the robot to function, especially its main dimensions.

In ROS2, the information that defines the robot characteristics is called the *robot description* and is stored in a Unified Robotics Description Format (URDF) file. In this file the robot is defined as a combination of links and joints, where a link is a robot part, and the joint is the intersection between links. There are many types of joints (revolute, continuous, prismatic, fixed, floating and planar), depending on how the links that connect in them behave with each other.

Manually writing a URDF file may be feasible to define robots with simple geometries. Production level robots, however, have a high number of links and joints, as far as the URDF stipulation goes. Their parts also possess irregular shapes, which cannot be easily constructed with the limited options available in ROS2. For these cases meshes can be utilized. These can be designed in 3D CAD software, granting the user a large increase in freedom, as the possibilities for what the part can be become limited by just the CAD software restrictions. After modelling, the part can then be loaded directly into the URDF file as a link. Various libraries of meshes free to use also exist online, which can be a useful resource. Alternatively, modern tools exist that create the URDF file from scratch, from just the CAD file. This was the method used in this project to create a realistic representation of the robot in ROS2, resorting to *solidworks_urdf_exporter* [30], an extension for Solidworks. While optional, it is still helpful for the user to create auxiliary axis and points to assist the extension, most importantly the geometric centre of the largest parts and their coordinate system, and the axis of rotation of the wheels and gears. When based on a realistic CAD, this method produces great results, with a digital robot essentially identical to the real one, visible both in Rviz (figure 2.13a) and in Foxglove Studio (figure 2.13b).



(a) Simulated robot in RViz



(b) Simulated robot in Foxglove Studio

Figure 2.13: Realistic robot representation in both data visualisation tools.

While promising in theory, the realistic model proved impractical during testing. The high complexity of the mesh models significantly hindered rendering performance, resulting in latency that prevented the digital representation from accurately reflecting the real-time position and motion of the physical robot. This led to a user experience that was neither responsive nor realistic. It remains uncertain whether these performance limitations were due primarily to hardware constraints, as the simulation was executed within a Linux virtual machine. It is possible that a more powerful host system or a native installation with proper disk partitioning could mitigate these issues. Given these limitations, a simplified model of the robot was adopted for use in the simulation environment. This version utilizes basic geometric shapes and includes two actuated wheels and a passive spherical caster for balance. While visually less detailed, the simplified model preserves the robot's key physical dimensions, particularly the distance between the driven wheels and the relative placement of onboard sensors.

2.9.2 Simulation with Gazebo

RViz and Foxglove Studio are purely for data visualisation. They can take sensor data from the robot, but also from simulated sensors. However, they are not simulators. Possessing a trustworthy simulation environment is invaluable in the development of any robotic platform. Testing on the real equipment is usually unpractical and time and resource consuming. It also carries a big risk for the integrity of the hardware, as any mistake can damage the robot, forcing expensive repairs and time delays. Many 3D simulators for robotics development exist. De Melo et al. [31] presents a comprehensive analysis and comparison between the most used simulators, placing emphasis on the following metrics: ROS Integration; Physics engine quality; Ability to import and export URDF and SDF files; Languages supported and extensibility; Software license requirement. By classifying each simulator regarding these metrics, the article narrows the options down to just three simulators: V-Rep [32], Gazebo [33] and Unity [34]. Gazebo is open source, contrary to the other two, and it includes by far the best ROS2 integration. These two main advantages were enough to select Gazebo as the simulator to use in this project. More recent alternatives exist, but not notable to a point in which they can challenge Gazebo.

As previously mentioned, Gazebo is an open-source robotics simulation environment developed by Open Robotics, the same organization responsible for the development of ROS. Although Gazebo is not inherently part of the ROS framework, the two are designed to integrate seamlessly. The simulated environment inside Gazebo is designated as the *world*. Users can create custom worlds tailored to specific application scenarios, or leverage publicly available libraries containing a wide range of community developed environments. Robots are defined in simulation through a URDF file, and can then be loaded into the Gazebo world. The simulated sensors within Gazebo publish data to the same ROS topics as their physical counterparts. This facilitates the testing of algorithms and minimizes the changes required to transition from a simulated environment to the real robot implementation.

Two Gazebo worlds containing a simplified version of the robot can be seen in figure 2.14. Figure 2.14(a) consists of a simple custom world created out of cones and barrels. This was the environment used for simulating the functionalities implemented in this project. Figure 2.14(b) demonstrates how

complex some community worlds can be. This world, called cafe.world even contains simulated humans walking in it. This rise in complexity can seem optimal at first, in the sense that it may increase the similarity with the real world and constitute a more trustworthy simulation. While that is true, big, more detailed worlds are also heavier computationally. When running on a machine with limited graphical capabilities they can be slow to open, close and might lag during visualisation, specially when switching camera angles and zooming in and out. This is slow and frustrating, specially when developing new features that require the user to open and close the world repeatedly, while making code adjustments. This was the main reason why big worlds were avoided when running simulations in this project, and why a simpler world was used.

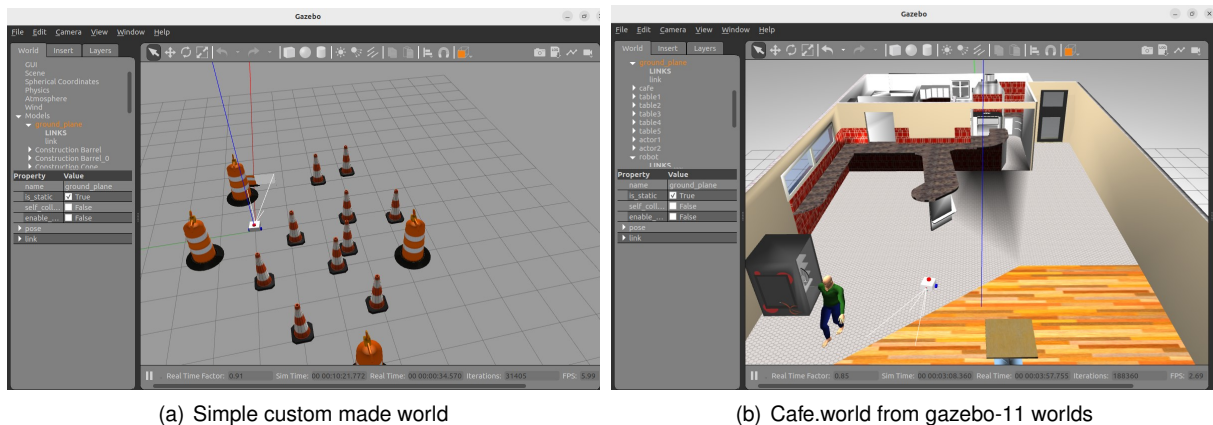


Figure 2.14: Example of Gazebo worlds.

Chapter 3

Control, Mapping and Navigation

This chapter describes the software framework implemented in the developed robot. Section 3.1 explains how velocity commands are translated into motor actuation, resorting to Arduino-based motor control, the kinematic model for differential drive robots, and integration with the ROS2 Control framework. Section 3.2 describes how teleoperation was implemented for this UGV. Section 3.3 provides an overview of common collision avoidance strategies, and presents the implemented algorithm, integrated with a twist multiplexer for command prioritization. Section 3.4 covers simultaneous localization and mapping, from the theoretical introduction to the SLAM Toolbox implementation. Finally, section 3.5 outlines autonomous navigation through environment modeling, path planning, and trajectory control using the Navigation 2 framework.

3.1 Low Level Control

In robotics, low-level control involves the direct management of the robot's hardware, more specifically, the motors, actuators, and sensors. It represents the first layer of the robot's control. In this specific case, as relating to the handling of the robot's motors, low-level control refers to the process of receiving target motor speeds from a given source, and actuating them on the real motors. Figure 3.1 illustrates the major components of the control system and how they interact. A detailed explanation of each component is provided below in this section.

3.1.1 Arduino Motor Controller

As previously mentioned, the motor driver receives velocity commands in the form of PWM signals. As the Auvideo board does not contain pins capable of delivering these signals, an Arduino was introduced as an intermediate controller. While this may present a downside in the sense that an additional component is required, it provides the opportunity to use the Arduino as the motor controller, instead of the Jetson TX2 itself, which presents numerous advantages. The Jetson is expected to run algorithms and programs which may be computationally expensive. Poor resource allocation can interfere with the motor control, causing the robot to become unpredictable and potentially crash. By keeping things separated,

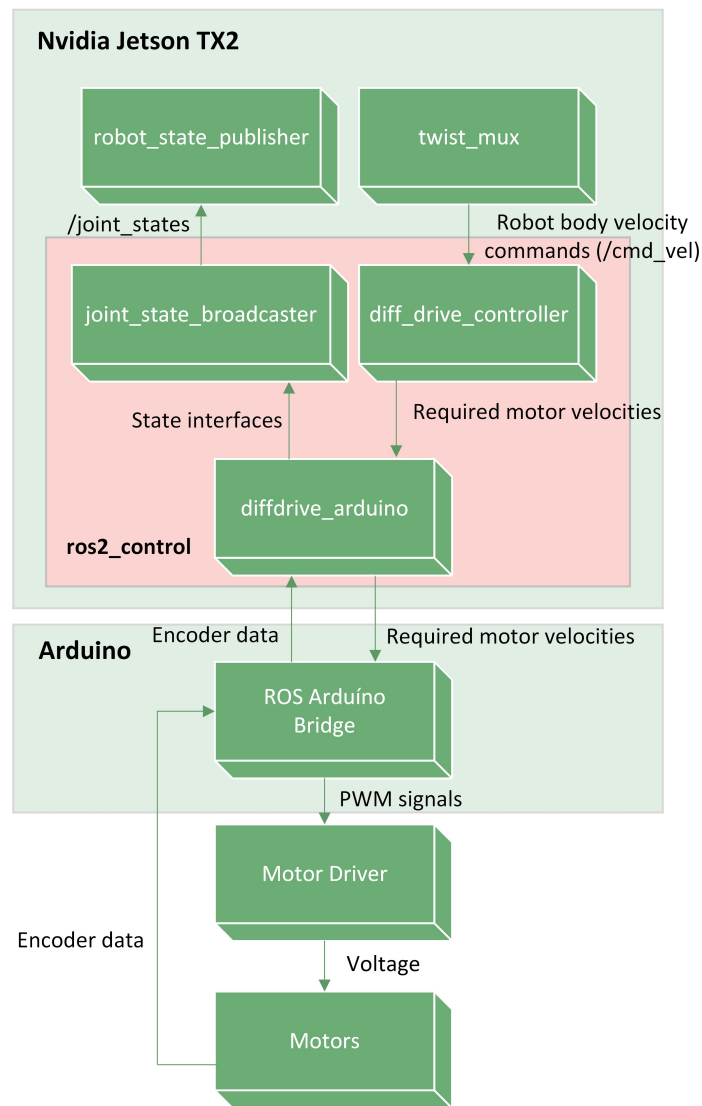


Figure 3.1: Control system architecture.

full priority is given to the motor control code, preventing this issue. This separation also increases the modularity of functions, which is almost always advantageous.

The Arduino code implemented for motor control was developed based on `ros_arduino_bridge` [35]. The code is structured to receive motor velocity commands over a USB serial connection, and output the appropriate PWM signals to the motor driver. The target speed is given in counts per loop, which represents how many encoder counts the wheel should achieve in each control cycle. It employs close-loop Proportional-Integral-Derivative (PID) control to regulate motor speeds based on the encoder data. This ensures that the wheel speeds are regulated based on feedback, rather than being set to a fixed PWM value as in open-loop control. This Arduino implementation uses the target motor velocities as input. In ROS2, however, it is standard to define velocities using the `geometry_msgs/Twist` message type. This message type defines the robot's velocity as a combination of its body's linear velocity in x, y and z of the local axes, and its angular velocity around the same axes. Therefore, an intermediate step that performs the conversion from robot to wheel velocities is required.

3.1.2 Differential Drive Robot Kinematics

The conversion from robot to wheel velocities can be obtained from the kinematic model of the robot. Kinematics can be described as the study of the motion of mechanical bodies and systems without consideration of their associated physical properties and the forces acting on them. This directly opposes the study of dynamics, which emphasises the causes of motion. Kinematics then focuses on the geometric aspects of motion, such as position, velocity, and acceleration. A mobile robot can freely move with respect to its environment, making it impossible to measure its position instantaneously, requiring integration of the robot's motion over time. This process induces various inaccuracies, mainly due to slippage, which helps explain why accurately measuring the robot's position can be considered one of the most daring challenges of mobile robotics.

First of all, it is essential to establish a clear reference frame for accurately expressing the positions and movements of the robot and its various components. In mobile robotics such representation is made by establishing two different frames: the global frame (X, Y) and the robot's local frame (x, y) , placed with its origin at the geometric centre of the vehicle. The angular difference between these reference frames is given by θ and can be described as the robot's orientation or attitude. The coordinates of the robot in the global frame (X, Y) , coupled with its orientation (θ) is called the pose of the robot and can be defined as a vector of these three elements:

$$\xi = [X \ Y \ \theta]^T$$

Land mobile robots such as rovers can only move in a two dimensional plane, which allows one to disregard the coordinate z when creating a kinematic model. This model is built on the assumption that the robot is always parallel to the ground and the distance of all the points in its chassis to the ground remains constant. These conditions can be broken, for example, if irregularities in the road were to make the robot tilt along one of its axes. The representation of a tracked differential drive robot is detailed in figure 3.2.

The relation between the local and global reference frames is given by the rotation matrix $R(\theta)$, which combines the translation vector $[x, y]^T$ with a rotation about θ :

$$R(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.1)$$

So, the velocity of the robot in the local frame (v_x, v_y) can be expressed in relation to the velocity in the global frame $(\dot{X}, \dot{Y})$:

$$\begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} \dot{X} \\ \dot{Y} \end{bmatrix} \quad (3.2)$$

A fundamental aspect of mobile robot kinematics is understanding how each wheel impacts the robot's motion. Wheels provide both freedom, by enabling the robot to move by rolling, as well as constraints, by being incapable of skidding laterally, for example. The combination of all wheel constraints,

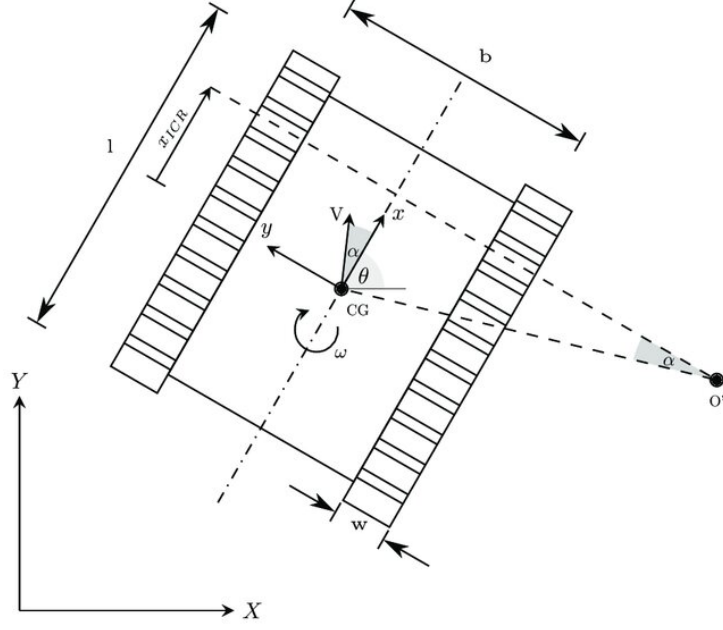


Figure 3.2: Kinematics of a tracked vehicle [36].

imposed by their interaction with the chassis, limits the robot's overall motion.

The kinematic model of a wheeled mobile robot relies on two major constraints: the rolling constraint and the sliding constraint. The rolling constraint assumes the wheel undergoes motion only under conditions of pure rolling and rotation about the vertical axis through a single contact point on the ground. The sliding constraint assumes there is no lateral slippage, meaning the wheel must not slide orthogonally to the wheel plane. Siegwart et al. [15] demonstrates how the equations obtained by these constraints allow for the development of a kinematic model. In the case of a wheeled differential drive vehicle, the model obtained defines how the robot's linear ($\dot{x}$) and angular velocities (ω) can be calculated as the result of the left (ω_L) and right wheel angular velocities (ω_R):

$$\dot{x} = \frac{r(\omega_R + \omega_L)}{2} \quad (3.3)$$

$$\omega = \frac{r(\omega_R - \omega_L)}{b} \quad (3.4)$$

where r is the wheel radius and b is the distance between the right and left wheels (as seen in figure 3.2). Equations (3.3) and (3.4) correspond to the forward kinematics. In order to obtain the wheel angular velocities necessary to produce a desired linear and angular velocity for the robot, the inverse kinematics are utilized:

$$\omega_R = \frac{\dot{x} + \frac{b}{2}\omega}{r} \quad (3.5)$$

$$\omega_L = \frac{\dot{x} - \frac{b}{2}\omega}{r} \quad (3.6)$$

The inverse kinematics, as the name suggests, are merely the inverse of the forward kinematics and can easily be deduced from (3.3) and (3.4).

For tracked differential drive vehicles the wheel constraints used to create the kinematic model are not applicable. The rolling constraint is based on the condition that there is a single point of contact with the ground, which is untrue with a track. The sliding constraint, on the other hand, assumes there is no lateral slippage, which is also untrue on a tracked vehicle. The kinematic model for wheeled vehicles could still be used, as a simplified version, but more complex models have been developed over time, which take into account this slippage. Examples of these models are presented in annex A.

3.1.3 ROS2 Control

Although custom controllers can be implemented directly using the robot's kinematic equations, standardized and modular solutions are available. For common robot geometries, this is a problem which has already been addressed by ROS2 Control [37], a framework for real-time control of all types of robots. It builds upon ROS Control, which delivered a similar service, but only for ROS1 and not ROS2. ROS2 Control works as the connection between the robot's software and hardware, sending velocity commands to the hardware and receiving joint states from it. Figure 3.3 presents the architecture of the `ros2_control` framework, whose main components are the controllers, the hardware interface, the controller manager and the resource manager, further detailed below.

Controllers

Controllers in `ros2_control` are responsible for computing the control actions that drive the hardware. While these controllers can be custom-made, the `ros2_control` framework contains an array of different controllers that facilitate the process. For differential drive vehicles it contains the `diff_drive_controller`, which takes velocity commands for the robot body as input and converts them to wheel commands for the differential drive base, resorting to the kinematic equations (3.5) and (3.6). It also utilizes the encoder feedback to compute and publish the ROS topic regarding the robot's odometry.

Additionally, the `ros2_control` framework contains broadcasters, which operates in unison with the controllers. Broadcasters are used to publish sensor data from the hardware to ROS topics. In this robot, the `joint_state_broadcaster` [38] is used to read the state interfaces and publish its information on the `/joint_states` topic. This enables visualization tools such as Rviz and Foxglove to display the state of the robot joints, via the `robot_state_publisher` [39].

Hardware Interface

In its default state, `ros2_control` cannot communicate with the hardware. Firstly, a hardware interface must be set up that exposes the hardware in the `ros2_control` standard, so that it can be detected. This hardware interface is often provided by the robot's manufacturer, or provided online in the case of common robot builds. For this specific case, it is necessary an interface between the `diff_drive_controller` and an Arduino running firmware from `ros_arduino_bridge`, which is exactly what is provided by `diffdrive_arduino` [40]. The hardware interface is configured in the robot's URDF file, using the `< ros2_control >` tag, and loaded as a plugin. Under this tag, the details of the hardware interface

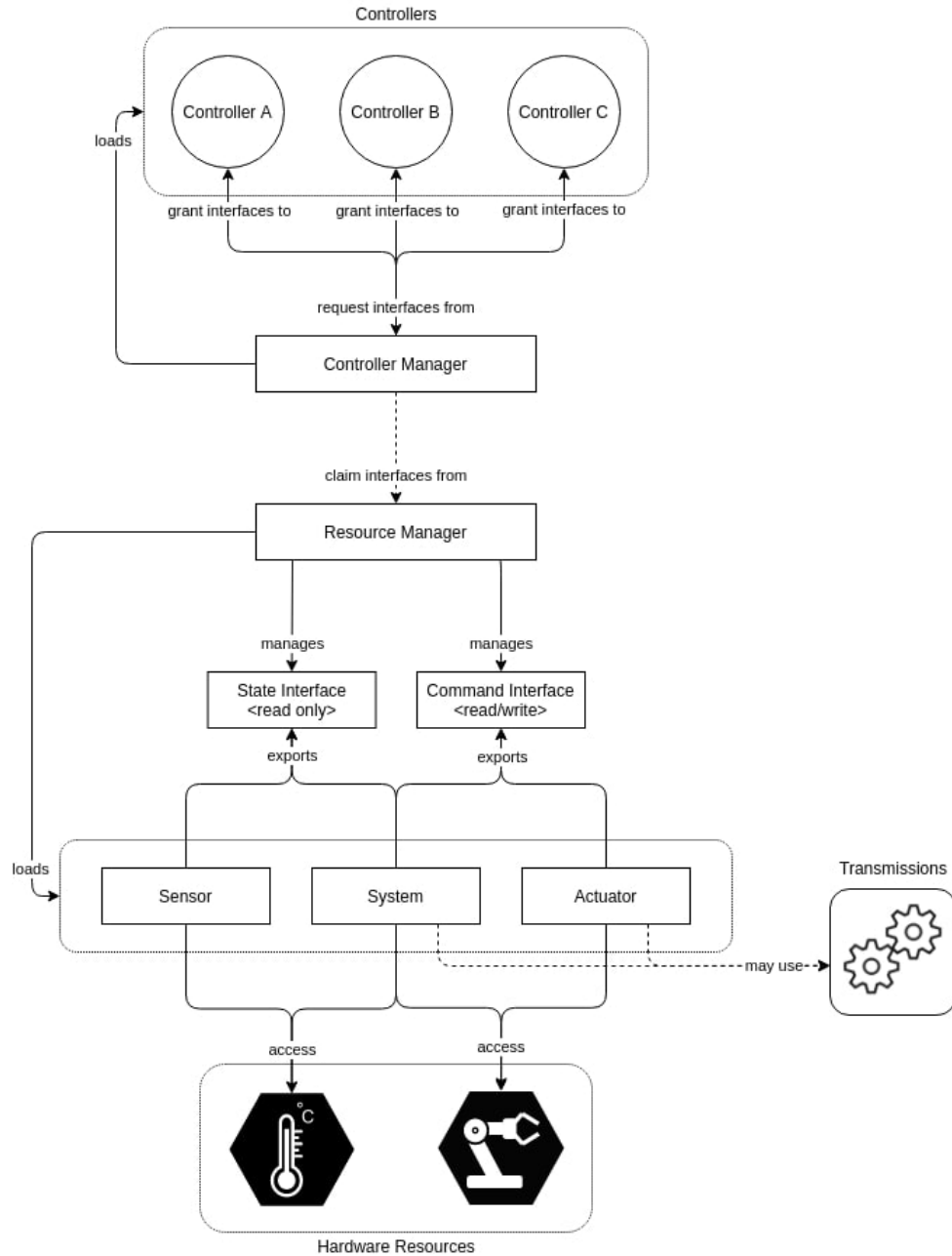


Figure 3.3: ROS2 Control Architecture [37].

used are stated. This is where the use of the `diffdrive_arduino` hardware interface is declared along with its configuration parameters, including the address of the serial device, the baud rate of the serial connection, the frequency of the processing loop on the Arduino, etc. Under the `< ros2_control >` tag is also where the joints and their command and state interfaces are specified, which correspond to the writing and reading capabilities of the hardware component. In this case each joint possesses a command interface in its velocity, as it is controllable, and two state interfaces in its velocity and position, as it is measurable data reported by the wheel encoders.

When using the Gazebo simulation environment, the hardware interface is substituted by mock components, simulations of hardware components which enable testing the `ros2_control` framework without using any real hardware. This is achieved by switching the hardware plugin to `gazebo_ros2_control`.

Controller Manager

A single robot may comprise multiple actuation systems that require more than one controller. The controller manager (CM) is responsible for loading, activating, deactivating or unloading the controllers, therefore managing their lifecycle. The CM also has the task of connecting each controller to the appropriate hardware interface, made accessible to the CM via the resource manager.

Resource Manager

In the same way that multiple controllers might exist, more than one hardware interface can be present. The Resource Manager is responsible for registering and exposing of reading and exposing all the hardware interfaces in a way that can be adequately detected by the controller manager, that is, by listing all the command and state interfaces.

Implementation

The first step in implementing `ros2_control` is to alter the robot's URDF file as described in the 3.1.3 'Hardware Interface' subsection. The next step is to create a standard ROS2 parameter file (.yaml), which specifies which controller to load. It also loads the `joint_state_broadcaster`, which reads and publishes the state interfaces. This is where the use of the `diff_drive_controller` is indicated, along with several parameters it requires: URDF name of the base frame and left and right wheel joints; wheel separation and radius; etc. Here, one can also impose limits for the velocity, acceleration and jerk of the robot frame.

Finally, a launch file is used to firstly launch the controller manager node, with the robot description and the created parameter file as conditions, and later launch the `diff_drive_controller` and the `joint_state_broadcaster`, only after the controller manager is fully loaded.

3.2 Teleoperation Command

Teleoperation in robotics predominantly relies on one of two options regarding hardware: gamepads and joysticks. Keyboards can also be used, but they are impractical as they are oversized and require a stable surface to be placed on. For teleoperation of a differential drive vehicle, it is optimal to use a system with two analogue sticks, one for the robot's linear velocity and the other for the angular velocity. Joysticks typically include one analogue stick and were therefore excluded. Gamepads, on the other hand, tend to possess two analogue sticks, which allows for a more ergonomic setup. For this project, it was stipulated that a simple gamepad would suffice to accomplish all the planned tasks. Due to availability, a Dualshock 4 gamepad was used (figure 3.4). This is the standard gamepad for the PlayStation 4 gaming console and meets all the needs, as it is ergonomic, portable, and contains two analogue sticks.

When setting up the teleoperation environment for this robot, two possible configurations were considered. These being connecting the gamepad directly to the Jetson TX2 or connecting it to the host



Figure 3.4: Dualshock 4 gamepad.

computer, which in turn communicates the velocity commands to the Jetson TX2. The latter was preferred, as it would function for both simulation and real robot teleoperation, thus facilitating seamless switching between modes. This approach can also increase the operational range of the UGV. When connecting the gamepad directly to the robot, the maximum range of teleoperation is dictated by the Bluetooth range. By connecting the gamepad to the host computer instead, the operational range is significantly extended, as long as both the host machine and the Jetson TX2 are connected to the same network.

The initial step in configuring a functional gamepad is to verify that the necessary drivers are properly installed on the Linux host system. Then, it is necessary to integrate the gamepad in ROS2, which can be accomplished through the use of two nodes: `joy` node and `teleop_twist_joy` node. The `joy` node detects the gamepad and publishes the state of its buttons and axes at a given frequency. The `teleop_twist_joy` node subscribes to the `joy` node and uses the thumbstick data to calculate the velocity commands in Twist format, which are then published as `cmd_vel_joy` messages. To correctly configure the nodes, the gamepad axes must be mapped to the linear and angular velocity commands. The maximum values for these commands are set according to the full deflection of the thumbstick. The measured velocity will then be a value from 0 to the maximum, proportionate to the thumbstick's position. As it is most common in the teleoperation of differential drive vehicles, the left stick (top to bottom motion) was assigned to the linear velocity, and the right stick (left to right motion) to the angular velocity. Additionally, the L1 button was stipulated to work as an activation button, which means that it must be pressed for velocity commands to be sent from the gamepad. This was required to prevent the gamepad from emitting 0 m/s velocity commands when inactive, which would prevent lower-priority sources from sending commands via `ros2_control`.

3.3 Collision Detection and Avoidance

The first consideration when developing a collision avoidance solution for this project was which sensors would be suitable for the detection of obstacles. A proposed solution should function throughout the entire robot's operation, serving as the fail-safe mechanism against collisions. Given the critical need to prevent collisions and avoid physical damage, it is evident that robustness is a key requirement. The system must operate reliably under all conditions, regardless of external factors such as adverse weather or low-light environments. This lack of robustness in low-light conditions is precisely the main downside of RGB cameras, making them unsuitable for this purpose. After comparing both active ranging sensors (RADAR and LiDAR), the LiDAR emerged as the better option, as the RADAR mostly excels in measuring velocities, which is not essential for this application.

3.3.1 Collision Detection and Avoidance Overview

In teleoperated mobile robots, particularly under human-in-the-loop control, ensuring real-time collision avoidance without the overhead of path planning is essential for maintaining responsiveness and user agency. Wang et al. [41] classify collision avoidance techniques broadly into path-planning and reactive (local) methods, with the latter being more appropriate for teleoperation in dynamic environments. Among local methods, Artificial Potential Fields (APF) stand out due to their simplicity and responsiveness. Although APFs are traditionally used in autonomous navigation, their real-time obstacle repulsion capabilities make them suitable for modulating user commands during teleoperation, exerting virtual repulsive forces to attenuate unsafe commands. However, limitations such as local minima can hinder performance in cluttered environments. Velocity-based approaches, such as the Dynamic Window Approach (DWA) and Velocity Obstacle (VO) method, are also relevant. While often applied in conjunction with planners, these can be adapted to filter or limit teleoperator inputs directly, using real-time LiDAR data to compute safe velocity sets. Furthermore, fuzzy logic controllers and learning-based reactive strategies (e.g., reinforcement learning or behaviour cloning) are mentioned as viable options for interpreting sensor data and adapting user inputs without path dependencies.

In terms of ROS2 based implementations, emphasis is placed on Nav2's collision monitor [42]. It allows users to define safety zones (as polygons or circles) around the robot and monitors these areas in real time using LiDAR scans. Each zone is associated with a specific safety behaviour model, and multiple zones with different models can operate simultaneously. Three behaviour models are supported. The Stop model halts the robot entirely if more than a set number of obstacle points are detected within its designated zone. The Slowdown model reduces the robot's maximum velocity by a configurable percentage when the number of points within its zone exceeds a threshold, allowing the robot to continue moving cautiously near obstacles. The Approach model dynamically estimates the time to collision based on the robot's current speed and distance to nearby obstacles. If this time is below a set threshold of N seconds (e.g., 0.5, 2, or 5), the robot's speed is reduced proportionally to maintain at least N seconds from collision.

Another example is the TurtleBot3 Obstacle Avoidance application [43]. This implementation sub-

scribes to LiDAR scan data and determines whether an obstacle exists within a defined angular field (typically 60°) in front of the robot. If an obstacle is detected closer than a threshold distance (e.g., 0.75 m), the node issues a rotational command to turn the robot away, otherwise, it proceeds forward.

While Nav2's collision monitor is comprehensive due to its support for multiple behavioural models, it lacks the rotational command component present in the TurtleBot3 Obstacle Avoidance application. This can cause the teleoperation to be tiresome on the operator, as frequent deceleration around obstacles delays the process. This is especially true when mapping large areas, where the focus is placed on covering a large amount of ground quickly. Additionally, the operator might have limited information about the environment, therefore increasing the number of times the robot is driven straight at an obstacle, further delaying the process. Ideally, the operator should be able to drive the robot in a general direction without needing to employ small corrections to circumvent obstacles. This way, not only is operator fatigue diminished, but less skill and learning is required to competently drive the robot. While TurtleBot3's Obstacle Avoidance application performs rotation commands, they can be drastic as the algorithm only stipulates one zone and one fixed correctional velocity command for all cases.

3.3.2 Collision Detection and Avoidance Algorithm Implemented

Drawing upon the strengths of both the TurtleBot3 Obstacle Avoidance and Nav2 Collision Monitor approaches, a new algorithm was developed that integrates the key elements of both systems. It defines multiple safety zones around the robot (figure 3.5b), each triggering increasingly conservative responses as obstacles are detected closer to the robot. The algorithm incorporates rotational commands, allowing the robot to reorient itself away from hazards. Figure 3.5a illustrates the sequence of actions employed by the algorithm.

Upon initialization, the node subscribes to four ROS topics: `/scan` for laser scans, `/cmd_vel_joy` for velocity commands issued via joystick, `/cmd_vel` for velocity commands of other sources, and `/joy` for raw joystick inputs. Three zones ahead of the LiDAR are defined, from Zone 1 to Zone 3, representing decreasing levels of urgency. These zones are demarcated by establishing a radius around the LiDAR and an angular interval, narrower as the level of urgency decreases. The algorithm executes upon reception of each new `/scan` message, which is then processed in order to calculate the distance and angle of each measurement. For each of the zones, the algorithm iterates through this data to find the closest reading within that zone, effectively identifying the nearest obstacle and its angle. If an obstacle is detected and a forward velocity command is given, a correction to that command is applied. This correction is applied to the linear velocity $\dot{x}$ and the angular velocity ω , resulting in a new command $(\hat{\dot{x}}, \hat{\omega})$, published on the `/cmd_vel_stop` topic. Let:

- d : the distance to the closest obstacle in a given zone,
- ϕ : the angle to the closest obstacle, relative to the forward axis,
- The angular bounds of each zone j are defined by $\phi \in (-\phi_{\max_j}, \phi_{\max_j})$,
- α_j and k_j : user defined constants.

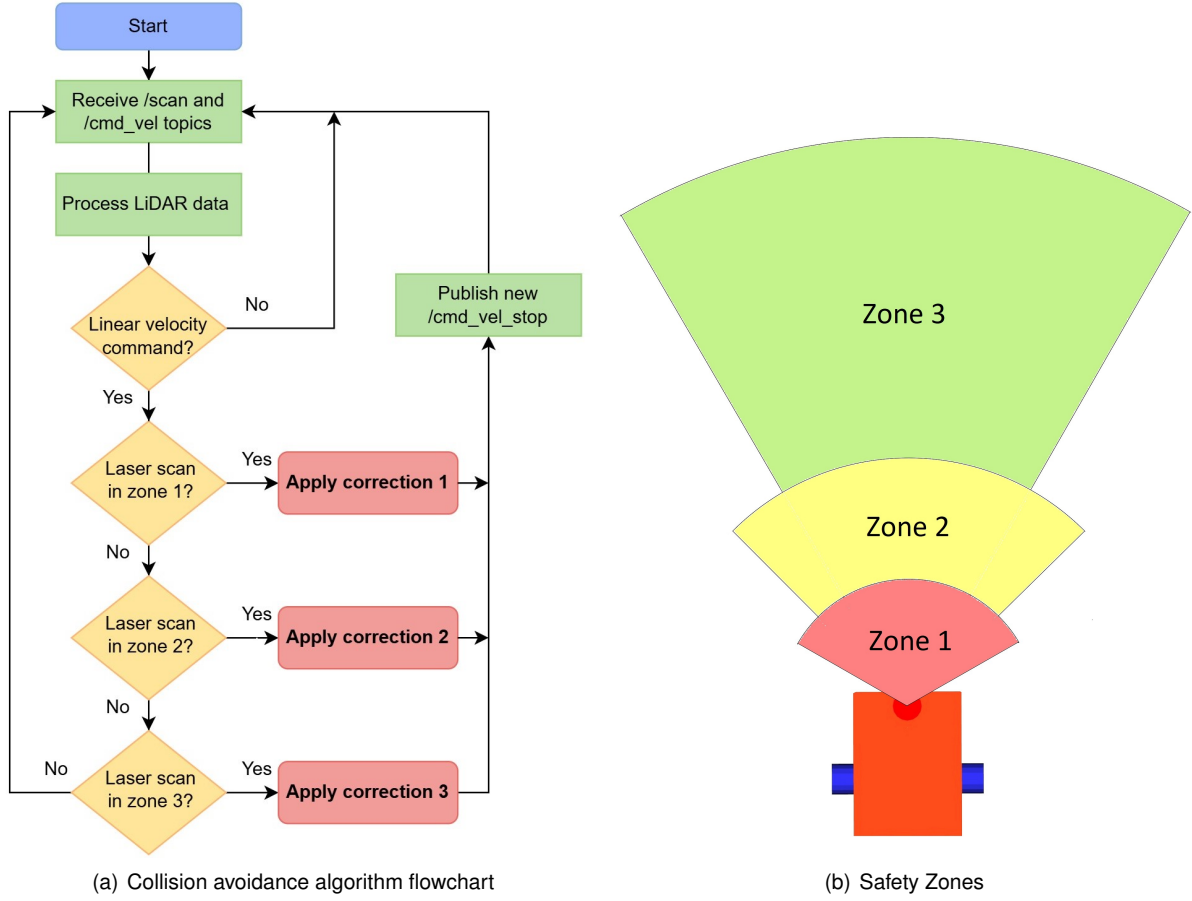


Figure 3.5: Developed Collision Avoidance Algorithm.

Equations (3.7) and (3.8) describe the correction applied to the linear and angular velocities for zone 1.

$$\hat{x} = 0 \quad (3.7)$$

$$\hat{\omega} = \alpha_1 \cdot \omega \quad (3.8)$$

Zone 1 represents the highest level of urgency. When the robot is moving forward and an obstacle is detected in this zone, the linear velocity is immediately halted to prevent a collision, with only negative linear velocity commands being permitted. The angular velocity is reduced proportionally to the user's command, resorting to a scaling constant α_1 . This allows the user to maintain control of the UGV and steer it away from danger, slowly to minimize the risk and severity of a collision.

Equations (3.9) and (3.10) describe the correction applied to the linear and angular velocities for zone 2.

$$\hat{x} = k_2 \cdot \dot{x} \cdot d \quad (3.9)$$

$$\hat{\omega} = \begin{cases} \min(\alpha_2(-\pi/2 + \phi), \omega), & \text{if } \phi \geq 0 \\ \max(\alpha_2(\pi/2 + \phi), \omega), & \text{if } \phi < 0 \end{cases} \quad (3.10)$$

When the robot is commanded forward and an obstacle is detected in zone 2, the linear velocity command is scaled down. Making the linear velocity dependent on the distance to the obstacle d creates

a gradual decrease in velocity as the robot approaches said obstacle, while smoothing the transition between zones 2 and 3. In this zone, a correction to the angular velocity is applied to steer the UGV away from the obstacle. Here, $\pi/2$ is used as a constant, ensuring the correction occurs regardless of the value of ϕ , while faster when the obstacle is head on, where $\phi = 0$. The *min* and *max* conditions serve to prioritize the user given rotation command, if larger and in the same direction.

For zone 3, the analogous corrections can be seen in (3.11) and (3.12).

$$\hat{x} = k_3 \cdot \dot{x} \cdot d \quad (3.11)$$

$$\hat{\omega} = \begin{cases} \min(\alpha_3(-\pi/4 + \phi), \omega), & \text{if } \phi \geq 0 \\ \max(\alpha_3(\pi/4 + \phi), \omega), & \text{if } \phi < 0 \end{cases} \quad (3.12)$$

The k_1 and k_2 constants can be tuned to apply a stricter linear velocity correction for zone 2. For the angular velocity correction of zone 3, a smaller constant of $\pi/4$ was used, given the reduced urgency.

3.3.3 Twist Multiplexer

As mentioned, this algorithm is meant to work as the last line of defense of the robot against collisions. For that to happen, however, a framework must be put in place, whose task is to prioritize the velocity commands provided by this algorithm above all else, ensuring it overrides any other commands. Throughout the robot's operation, different sources of velocity commands coexist, namely, the gamepad, the collision avoidance algorithm, and Nav2. As multiple sources can emit commands simultaneously, there is a need to establish and enforce a hierarchy between them, so as to allow the robot low-level control to appropriately select and execute one only. This is precisely what `twist_mux` [44] accomplishes. Firstly, it is good practice to attribute a different name to the topic of each command. Then `twist_mux` simply requires the topic name, a priority number from 0 to 255 (the higher the number, the higher the priority), and a timeout, which specifies how long (in seconds) a topic can remain without messages until other topics are selected instead. A single topic is selected every time a new message arrives through the input twist topics, and its messages are published in the `/cmd_vel` topic, recognizable by `ros2_control`. Among the velocity command sources, the highest priority was assigned to the obstacle avoidance algorithm, due to its role as a safety mechanism. The gamepad commands were given intermediate priority, followed by those from the Nav2 stack. Placing the gamepad above the commands provided by the navigation stack increases the overall robustness of the robot's operation. Autonomous tasks, such as waypoint navigation, are not infallible and can induce situations where human intervention is required. Using the described setup, the operator can, at all times, take control of the robot, by using the gamepad, overriding any commands from Nav2.

3.4 SLAM

As a functional requirement, it was stipulated that this UGV must possess the capability to navigate effectively in environments where GPS is unavailable. To align with this principle, the waypoint navigation platform utilized must rely on waypoints in the context of the surrounding environment map. This approach differs from defining waypoints in the form of GPS coordinates, which is common practice in mobile robotics. To make this possible, the robot must be capable of mapping its surroundings, while also accurately localising itself in them. This is what defines the SLAM problem, described in section 3.4.

While localisation attempts to estimate the robot position given a known map, mapping constructs a map of the environment knowing the true path of the robot. Resorting to data of the environment acquired from the sensors, SLAM aims to perform both tasks simultaneously, as the name suggests (Simultaneous Localisation and Mapping). The essence of the SLAM problem is that for localisation, the robot needs to know the location of the features, whereas for map building, the robot needs to know where it is on the map. This problem is exacerbated by the existence of noise in both the path and the extracted features location estimations. The SLAM procedure typically follows the given sequence of events, shown in Figure 3.6.

- a) The robot starts at a known initial location and detects a given feature, with an uncertainty associated with the exteroceptive sensor error model;
- b) The robot moves, creating uncertainty about its position due to errors introduced by the odometry;
- c) As new features are observed, the robot maps them, combining the uncertainty from the robot's position and the measurement errors;
- d) The robot continues its motion, increasing the position uncertainty;
- e) As the robot detects a feature that had been previously observed, loop closure occurs. This causes the uncertainty of the robot pose to shrink, which in turn updates the map, reducing the uncertainty of the observed features and the robot's previous poses.

Filter-Based SLAM is the classical approach to solving the SLAM problem, by firstly predicting the vehicle and map states with processing models and control inputs, and then applying a correction to the predicted state using the current sensor observations. Earlier Filter-Based SLAM implementations include methods based on Extended Kalman filters (EKF) [45] and Particle Filter (e.g. FastSLAM [46]). Optimization-Based SLAM methods, such as Graph SLAM algorithms [47] construct the map using a graph structure, consisting of nodes and arcs. Nodes represent robot poses throughout its trajectory and the associated sensor measurements, while arcs encode constraints between these poses, which can be a motion event or a measurement event. The map is generated by determining the spatial arrangement of nodes that satisfies the constraints imposed by the arcs.

The SLAM problem can be addressed using a variety of sensor modalities, with the main sensor for SLAM being LiDAR, its simplicity and accuracy compared to other sensors. LiDAR-based SLAM func-

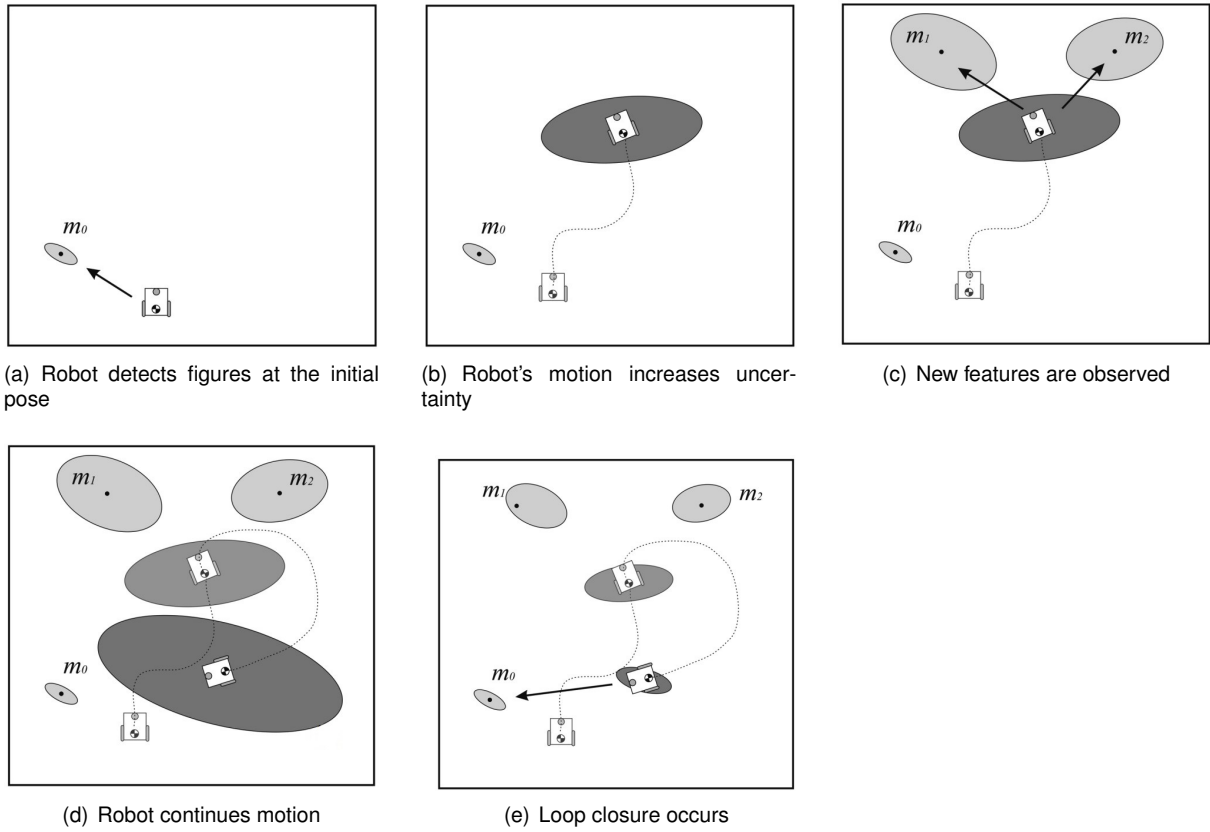


Figure 3.6: Illustration of the sequence of events in SLAM [15].

tions on the basis of scan-matching, which recovers the relative position and orientation of two scans or point clouds, followed by graph optimization. Radar and Sonar/ultrasonic sensors can also be used for SLAM estimation. Compared to LiDAR, Radar offers advantages such as lower cost, reduced power consumption, and greater resilience to atmospheric conditions, making it particularly suitable for outdoor environments. However, Radar's lower resolution and sparser detection data make it challenging to handle data association and match measurements effectively. As a result, Radar-based mapping is generally less precise than LiDAR-based systems [48]. Vision sensors, such as monocular and stereo cameras are another group of popular sensors for SLAM, in what is defined as Visual SLAM, or V-SLAM. V-SLAM methods can be either: feature-based, which detects features in images and utilizes descriptive features for map building (e.g. Orb-SLAM [49]); Direct SLAM, which uses the entire images as a whole (e.g. LSD-SLAM [50]). Compared to LiDAR and Radar, cameras are cheap and can capture detailed visual information, providing additional data about the environment. However, their performance is heavily influenced by environmental factors like texture and lighting conditions, and can struggle in environments with a weak geometric diversity (e.g. long hallways). Additionally, camera-based systems often have high computational requirements, which can limit their real-time processing capabilities. Multiple sensor fusion methods exist, which combine data from a variety of sensors, making the SLAM system more robust.

3.4.1 SLAM Packages Overview

Due to its complexity, the SLAM problem is extremely difficult to implement from a computational perspective. The creation of an in-house solution from scratch is unfeasible, as it would lack the robustness necessary for a successful implementation. Thankfully, in recent years, development has been made to create SLAM software packages. Presented in 2007, GMapping [51] represented a breakthrough in this area. It uses a particle filter approach to SLAM for the purpose of building grid maps from 2D LiDAR data. However, GMapping underperforms in large spaces and is not suitable for an industrial scale.

HectorSLAM [52], presented in 2011, relies on LiDAR scan matching and a 3D navigation filter based on EKF state estimation. HectorSLAM is unique in the sense that it does not use odometry data, which can cause inaccurate pose and map updates when LiDAR scans arrive at a lower rate, or when mapping large or featureless spaces. KartoSLAM (2010) [53] and Cartographer (2016) [54] are graph-based algorithms, which only need to maintain the pose-graph, making them efficient in resource management, especially when building large-scale maps. RTAB-Map (2018) [55] is another graph-based SLAM algorithm, composed by a C++ library and a ROS package. RTAB-Map focuses on optimizing long-term localization and mapping by resorting to distributed storage systems (short-term, working, and long-term memory), limiting the processed space for efficient loop closure.

Filipenko and Afanasyev [56] performed a qualitative comparison of Gmapping, HectorSLAM, Cartographer, and 7 visual based SLAMs using a mobile platform in an indoor environment. The experimental results present Cartographer as the most accurate 2D laser-based method, while showing encouraging results for the Monocular ORB SLAM and Stereo RTAB Map methods. Yagfarov et al. [57] extended the comparative analysis of Gmapping, HectorSLAM and Cartographer, using a FARO laser tracker as the ground truth. This research compared the three SLAM packages according to the ADNN-based errors (Average Distance to the Nearest Neighbour), varying the speed of the ride, the sharpness of rotations and the existence of loop closure. Once again Cartographer obtained the best results of all three packages, trailing only on the fast ride with sharp rotations, loop closure scenario, to Gmapping. Fan et al. [58] expanded this comparison to scenarios where the robot performed autonomous exploration, obtaining again best results for Cartographer. Trejos et al. [59] included RTAB-Map and KartoSLAM in the comparison, evaluating the SLAM packages on Gazebo on the basis of four metrics: pose error, map accuracy, CPU usage, and memory usage. In this research, KARTO-SLAM outperformed all the other algorithms as it balanced the use of resources, while holding a good SLAM performance. Without considering resources usage, the best algorithm was considered to be RTAB-Map.

Slam Toolbox (2021) [60] is described in its release paper as the state of the art in SLAM packages for ROS2. SLAM Toolbox builds upon the Karto-SLAM package, as both are graph-SLAM based algorithms. SLAM Toolbox focuses on overcoming Karto's limitations with larger maps and offers a more optimized integration with ROS2, while also introducing improvements, such as enhanced graph manipulation, visualization, and data storage. SLAM Toolbox has been used in numerous robotics applications, including large-scale mapping, with scenes up to 24,000 m^2 successfully mapped in real-time by non-expert technicians. It achieves this through a concept known as lifelong mapping, where a space can be mapped either partially or fully and continuously refined and updated over time. This is accomplished by

serializing and deserialising map data, enabling users to update existing maps, serialize them for use in future mapping sessions, or reload distant sections within the same session.

Piazza et al. [61] compared GMapping, HectorSLAM, and SLAM Toolbox on the metrics of: normalized relative translation error, success rate, absolute trajectory error, CPU utilization, and max memory. The experimentation was conducted on Gazebo, where the dependency of the performance metrics from a set of features (LiDAR maximum range, LiDAR field of view angle, and odometry noise beta) was analyzed via the creation of generalized linear models. It demonstrated a clear all-around superiority for SLAM Toolbox, followed by Gmapping and lastly by HectorSLAM, which only outperformed the other methods on CPU usage. Torres et al. [62] evaluated SLAM Toolbox, Cartographer, and two particle filter algorithms (GMCL and AMCL) on localization performance. These were assessed in Gazebo in 3 different environments, progressing in complexity, with and without dynamic agents in each. The pose error estimates in translation were obtained for each method on the six evaluation scenarios, and demonstrated that SLAM Toolbox achieved the best performance in the simplest and in the most complex scenarios, trailing Cartographer in the intermediate scenario. Cartographer, however, could not be quantitatively evaluated in the most challenging scenario, as it found wrong data associations, resulting in wrong relative constraints that caused localization failure.

Upon reviewing the existing literature on the state of the art in SLAM implementations, the decision was made to use SLAM Toolbox to perform mapping and localization for this project's mobile platform. Firstly, it was given priority to the use of 2D LiDAR-based SLAM solutions over Visual SLAM. The main factor influencing this decision was the higher map accuracy and overall robustness, especially in environments with low feature variety. When selecting SLAM Toolbox in favour of the remaining SLAM packages, emphasis was given on the superior performance registered in the mentioned literature, coupled with its widespread use in production grade robotics projects and active support.

3.4.2 SLAM Implementation

SLAM Toolbox offers four mapping modes, combining online/offline operation with synchronous/asynchronous processing:

- **Offline Synchronous:** This mode is for situations where the data is pre-recorded. This allows the mapping can occur without time constraints, ensuring every measurement is processed. It provides the highest quality maps of all modes.
- **Online Synchronous:** This mode is used in real-time applications, where the map is built and updated as the robot navigates through the environment. Since data is processed in strict chronological order, the system requires steady and predictable sensor input to maintain real-time performance. Any delay in the LiDAR scans can introduce latency and compromise mapping accuracy.
- **Online Asynchronous:** This mode processes the most recent LiDAR scan always, which can result in scans being skipped if the processing time exceeds the sensor update rate. While this may result in data loss, consequently reducing mapping accuracy, it minimises delays, creating a more robust alternative to the synchronous mode, more suited for dynamically changing environments.

The online asynchronous mode was selected for this project's scenario, as real-time mapping was a requirement, and the LiDAR data could not be received and processed consistently enough for synchronous use. LiDAR scan messages would often queue up, forcing SLAM Toolbox to process outdated messages. In asynchronous mode, these outdated messages would eventually get dropped, prioritizing newer messages. This ensures that pose estimates remain more temporally aligned with the robot's actual position. SLAM activation in this mode is achieved using the `online_async_launch.py` launch file, with parameters tuned in the `mapper_params_online_async.yaml` file to ensure proper implementation for this specific robot model. To facilitate the process, this SLAM file was incorporated into the general launch file, while referencing the corresponding parameter file.

When performing SLAM in a new scenario for the first time, the user is greeted with a nearly empty map, that only showcases obstacles detected from the robot's initial position. The user must then teleoperate the robot to explore the unmapped areas, ideally passing through each area more than once, as to allow SLAM to successfully perform all the necessary loop closures. After sufficient exploration, the user can obtain a satisfactory map according to their standards. This map can then be saved and serialized, resorting to the SLAM Toolbox Rviz extension. When operating in repeated scenarios, the user can deserialise previously built maps, removing the need to redo the entire mapping process. By specifying the path to the saved map in the SLAM parameters file, the robot can be spawned at launch within the previously built map. One important remark is that the initial pose in which the robot begins mapping will be considered from there on as the robot's dock, and if the map is reopened, SLAM Toolbox will assume the dock as the robot's initial pose. Therefore the user must ensure the robot is placed in the presumed dock's position before launching the map, at least approximately, otherwise SLAM Toolbox may struggle to localize the robot within the map.

3.5 Waypoint Navigation

The Waypoint navigation problem, and autonomous navigation in mobile robotics as a whole, relies on two fundamental principles. The first is path planning, which involves identifying the optimal trajectory that the robot must follow to reach the desired goal location. The second is path execution, which ensures the robot is adequately guided by the planned path, through the use of feedback control, where a controller uses tracking errors to compute the robot's velocity required for the robot to align with the desired path. Many solutions have been developed in recent years to satisfy both conditions, each solution unique as to function on different types of robots, fulfilling a variety of needs.

3.5.1 Environment Modelling

A path planner requires information about the environment, most importantly the position of obstacles, so that it can recognize what areas can be traversed or not. The question that arises is how should the surface that composes the environment be organized and utilized, which is the essence of environment modelling. Liu et al. [63] states that among 105 papers regarding environment modelling, 85.71% of

models use the grid method, demonstrating it represents the mainstream modelling method in path planning of mobile robots. The grid method partitions the mobile robot's workspace into a series of network cells, most frequently in a square shape. Each cell is then attributed a cost, as the metric that the robot accumulates by moving, with the objective of the planner being to minimize the total cost. Simpler models have the cost as binary, with a free cell usually marked as 0 and an obstacle cell as 1. Alternatively, the cost map can assign different values of cost to different accessible areas, for example, increasing the cost proportionately to the proximity to an obstacle. Many other methods of cost attribution exist, dependent on factors such as the robot's heading, the terrain characteristics, the expected energy consumed by traversing the cell, etc. Alternatives to the grid method include the topological method, which represents the environment as a roadmap: a graph built upon nodes connected by edges. Examples of roadmaps include Voronoi graphs, Visibility graphs, and State-Lattice graphs [64]. Ultimately, the environment modelling method will depend on the path planning algorithm, as different planners will require different models.

3.5.2 Path Planners Overview

The majority of review papers (e.g. [65],[66]) categorize path planning techniques for UGVs' navigation into classical and heuristic (or meta-heuristic) approaches. Classical path planning algorithms, such as Cell Decomposition (CD), Potential Field (PF), Dijkstra's Algorithm (DA), Rapidly-exploring Random Tree (RRT), Probabilistic Road Map (PRM) and Roadmap Approach (RA), are renowned for their simplicity and consistent results, due to their deterministic nature, at the cost of being computationally intensive. These algorithms are more suited for global path planning applications in static environments, although recent optimization methods have been developed to expand their robustness in dynamic environments.

Heuristic approaches offer an alternative to classical methods, classified as more robust against uncertainties in the search space [65]. These algorithms use intelligent search mechanisms to iteratively find the best path toward the goal. Unlike classical approaches, which aim to generate a single global path upfront, heuristic methods create temporary sub-populations of possible paths during each iteration. The most promising path is then selected based on a fitness evaluation. Examples include the Genetic Algorithm (GA), Neural Networks (NN), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO). Loganathan and Ahmad [65]'s review of over 200 articles on AMR navigation identified a dramatic increase in the employment of heuristic methods in the past decade, with GA and PSO found to be the most popular techniques. Within the classical methods, the PF, RRT, and RA appear to be competitive in terms of popularity due to their suitability for reactive navigation and ease of implementation on hardware compared to CD.

Wahab et al. [66] compared the performance of multiple classical and meta-heuristic approaches on two different environments, on a variety of metrics (Execution Time, Battery Consumption, Travelled Distance, etc), using Dijkstra's Algorithm (DA) as the benchmark. In this study DA outperformed other methods in all relevant categories, however, it was the only method to be granted access to the complete environmental layout as to constantly perform global path planning. Therefore, the results achieved

by DA are considered as the absolute optimal performance. Not considering DA, the study classified Chaotic Particle Swarm Optimization (CPSO) as the best performing approach, followed by Potential Field (PF) in second, and Differential Evolution (DE) in third. In general, heuristic-based motion planning methods outperformed the classical counterparts, with the exception of PF. This is to be expected as these methods perform better if they are implemented under a global path planning condition, where prior environment information is given. Nonetheless, they were proven as reliable, especially DA and PF.

3.5.3 Trajectory Controllers Overview

The motion control task of a UGV can be divided into point-to-point motion, path following, and trajectory tracking. While point-to-point motion aims to reach a destination point from the initial starting position, disregarding course and trajectory, path following seeks to approach and follow a predefined path. Trajectory tracking adds a time constraint to this problem, ensuring the path is followed while adhering to a predefined timeline. Ruslan et al. [67] divided control strategies into five main categories, namely geometric and kinematic, classical, adaptive, intelligent, and model-based controllers.

The most popular controllers of the geometric and kinematic type are the pure pursuit controller, with its many variations, and the Stanley controller.

In the classical methods, emphasis is given to PID and Sliding Mode Control (SMC) controllers. Al-Mayyahi et al. [68] presented a fractional-order PID path tracking control strategy to compute the UGV's heading angle and speed control laws, optimizing the parameters via Particle Swarm Optimization (PSO). SMC utilizes the UGV's dynamics and slipping characteristics to obtain the torques required to drive the vehicle and converge its trajectory to the desired one. Sabiha et al. [69] demonstrated that SMC causes the vehicle to converge faster to the reference trajectory than PID.

Adaptive controllers enhance robustness to disturbances and environmental changes, which is essential in dynamic environments. Hiramatsu et al. [70] developed an adaptive path tracking controller for tracked robots on rough terrains, addressing slippage using a disturbance observer. Neural networks and Fuzzy logic are commonly used in adaptive control, with implementations either as supervisory tuning methods or primary controllers. Li et al. [71] demonstrated that Fuzzy Logic Controllers (FLC) outperform PID controllers for trajectory tracking.

Model-based controllers, or Model Predictive Control (MPC), improve path tracking by predicting vehicle responses using dynamic models. Tao et al. [72] combined MPC with an EKF for tracked vehicle trajectory tracking, enhancing lateral accuracy by 24%. Li et al. [73] developed a hierarchical MPC system for dual-track speed control, showing superior performance with higher-order Runge-Kutta methods.

Various other control methods are widely used, such as the Linear Quadratic Regulator (LQR), Lyapunov's direct method, the feedforward and feedback control methods, and the H_∞ controller. These are discussed in detail in literature reviews [67], [74] and [75].

3.5.4 Navigation 2 Framework

Navigation 2 (Nav2) [76] provides a platform for seamless integration of planners and controllers, with a plethora of additional useful functionalities. The Navigation 2 stack is a framework designed to provide autonomous navigation capabilities to mobile robots. It builds upon the principles and lessons learned from the original ROS1 Navigation stack, incorporating improvements and new features that take advantage of ROS2's architecture. Nav2 uses behaviour trees to manage navigation tasks, orchestrating multiple modular servers that handle specific functions like path planning, control, and recovery. These servers communicate via ROS interfaces, such as action servers or services, enabling customized and intelligent navigation behaviours. The main servers used in NAV2 are:

- **Planner Server:** The Planner Server is responsible for computing paths for the robot to follow in order to reach a specified goal.
- **Controller Server:** The Controller Server is responsible for the real-time execution of the path generated by the Planner Server. It ensures that the robot follows the planned path accurately while avoiding obstacles and adapting to changes in the environment.
- **Smoother Server:** The Smoother Server is responsible for enhancing the quality of the paths generated by the Planner Server. Its primary role is to take a path, which may be suboptimal or jagged due to the discrete nature of planning algorithms, and produce a smoother, more efficient path for the robot to follow. This process can improve the robot's navigation performance, making movements more natural and reducing wear on mechanical components.
- **Recovery Server:** The Recovery Server is responsible for executing recovery behaviors when the robot encounters situations that prevent it from following its planned path. These situations could include getting stuck, encountering an unplanned obstacle, or deviating significantly from the path. The Recovery Server helps ensure the robot can autonomously recover from such situations and continue its mission.

3.5.5 Plugin Selection and Parameter Tuning

The Navigation 2 stack is a highly flexible tool due to its extensive collection of out-of-the-box plugins and customizable parameters. The most optimal combination of parameters will be different, depending on factors such as robot shape and application. Therefore, it is essential to select appropriate planners and controllers, while performing parameter tuning, as to maximize the robot's performance. This parameter tuning was conducted through the use of tuning guides such as [77], [78] and [79], coupled with personal experience and trial and error.

Robot footprint

In Nav2, the robot's shape can be defined as a geometric footprint (figure 3.7b) or as a circle encompassing the robot (figure 3.7a). This shape is important as it equates to the robot dimensions considered

by Nav2's path planners and obstacle avoidance algorithms. An accurate depiction of the robot's true dimensions ensures smooth navigation in tight spaces like narrow hallways, for example. The holonomic planners are not kinematically feasible with a non-circular footprint and will always consider the robot as circular. On the other hand, all path controllers support rectangular footprints. The main reason to use a circular footprint for a non-circular robot is that custom non-circular footprints can increase the computational burden. Both types of footprints were tested and since no major difference in processing speed was detected, the more accurate rectangular footprint was utilized.

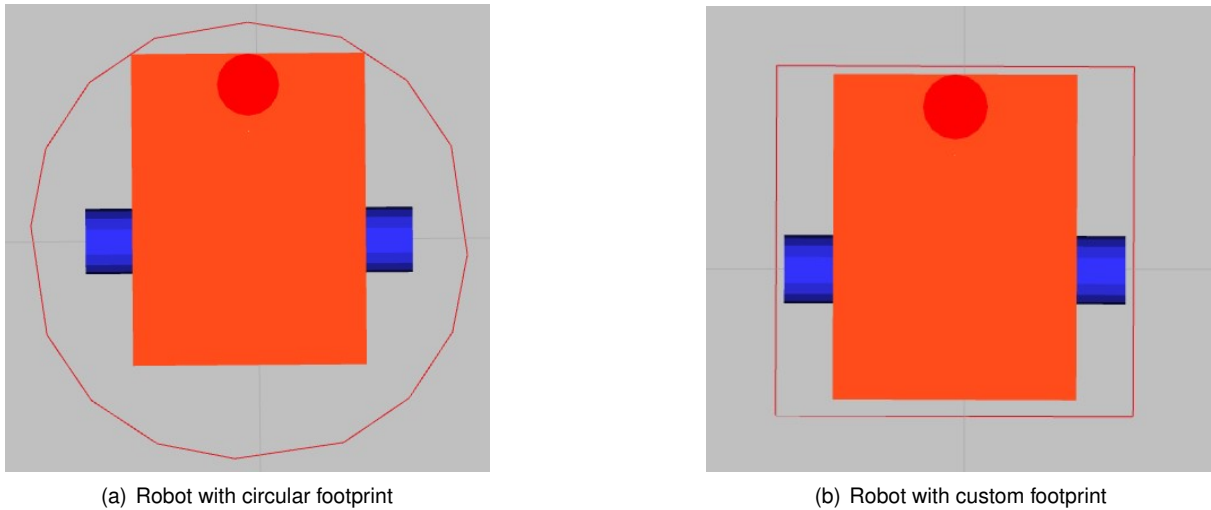


Figure 3.7: Different types of robot footprint.

Costmap Creation

The adequate configuration of the environment's costmap is essential for the planner's success. In Nav2, costmaps are built based on a combination of layers, which can be modularly added or removed depending on the user's requirements. Additionally, navigation in Nav2 is coordinated using two different costmaps: the local costmap and the global costmap. While the global costmap is built using the SLAM map, sharing its size, the local costmap only records the area immediately surrounding the robot, and is built based on the LiDAR data. The local and global costmaps can and should be composed of different layers, as they serve different functions. The following layers were used to construct the costmaps in this implementation:

- **StaticLayer:** This layer functions as the global costmap's base layer, by using the SLAM map to set the environment and its key features.
- **ObstacleLayer:** This layer sets the local occupancy grid, based on the live LiDAR scans, which can then be converted to the local costmap.
- **InflationLayer:** This layer exponentially inflates the lethal costs of the occupied space. While in the local costmap, this inflation is applied to the real-time LiDAR scans, in the global costmap, it is applied to the SLAM map. This is what creates the costmap per se, as it turns a trinary occupation

grid, with cells that are either occupied, unoccupied, or unknown, into a grid consisting of cells with cost ranging from 0 to 255. The layer accomplishes this by expanding outwards from obstacles to a given distance, applying (3.13).

$$cost(x, y) = (cost_{lethal} - 1)e^{-\omega_{scale}(d_o - r_f)} \quad (3.13)$$

where ω_{scale} is the cost scaling factor, d_o is the distance to the obstacle, $cost_{lethal}$ is the lethal obstacle cost (usually 255), and r_f is the inscribed radius of the footprint. This is the layer more deserving of attention, as the tuning of its parameter influences the planners behavior the most, through the use of the `inflation_radius` and `cost_scaling_factor` parameters. The `inflation_radius` parameter defines how far away the zero cost point is from an obstacle, and `cost_scaling_factor` defines the exponential decay factor across the inflation radius, as in, a higher value will cause the inflation decay curve to be steeper. An approach when configuring the inflation layer is to set a small inflation radius to weigh against collisions, while attributing a 0 cost to the majority of the empty space, as seen in figure 3.8a. This approach is not optimal as it induces the planner to create paths dangerously tangent to obstacles. The best procedure is to increase the `inflation_radius`, and reduce the `cost_scaling_factor` as to create a consistent potential field across the entire map, as in figure 3.8b. This way, the 0 cost areas are minimised and the planners produce smoother paths, mostly equidistant from the surrounding obstacles. This is visible in the path produced in figure 3.8b, which contrasts to the path created in 3.8a, which is simply straight, since the area it crosses is considered as having 0 cost.

- **DenoiseLayer:** The denoise layer attempts to remove noise that may exist in a costmap's layers due to sensor noise.



(a) Low radius, high decay approach



(b) High radius, low decay approach

Figure 3.8: Different approaches to the inflation layer.

The costmap's resolution is another important factor to take into account. In Nav2, resolution is

defined in the parameters as the length of the side of the square pertaining to the minimal subdivision. Smaller values increase the resolution costmap by increasing the number of cells it contains. all at the expense of increasing the computational cost. For the global cost map a resolution of 0.05 m was defined, as this is the resolution of the maps produced by SLAM, and therefore a smaller resolution would be inconsequential. For the local costmap a smaller resolution of 0.02 m was selected, while dimensioning the local costmap to a 1x1 m square. These values were the result of constant adjustment to find the trade-off between computational performance and retaining as much information as possible about the environment.

Planner Selection

Nav2 provides users with a selection of global planner plugins, all being an extension of Dijkstra's algorithm at their core, in trying to find the path with minimal cost. In practice, the outcome is typically the shortest path that maintains a safe distance from any obstacles, when the costmap is properly set up as described earlier. The provided Nav2 planner plugins, coupled with the supported robot types can be seen in table 3.1.

Table 3.1: Supported Robot Types for Navigation Plugins [78].

Plugin Name	Supported Robot Types
NavFn Planner	Circular Differential, Circular Omnidirectional
Smac Planner 2D	Circular Differential, Circular Omnidirectional
Theta* Planner	Circular Differential, Circular Omnidirectional
Smac Hybrid-A* Planner	Non-circular or Circular Ackermann, Non-circular or Circular Legged
Smac Lattice Planner	Non-circular Differential, Non-circular Omnidirectional, Arbitrary

The robot type is the pivotal factor in the path planning problem as it imposes the kinematic and dynamic constraints of the robot, dictating its possible manoeuvres. Table 3.1 contains a caveat regarding holonomic planners (NavFn, Smac 2D, Theta Star), in the sense that they can also be applied to non-circular differential vehicles, under the condition that their shape allows for them to rotate in place anywhere in their environment without collision. A brief description of each available planner plugin is given below.

The **Navigation Function** planner (NavFn) uses Dijkstra's Algorithm to expand a potential field, calculated using a quadratic kernel function, from the goal cell to the start. The kernel at a cell is determined by its cost and the neighbouring cells' potentials. Costs are linearly adjusted from the cost grid to create a neutral-to-maximum range, generating increasing wavefront costs until the start cell is located. The path is traced back using gradient descent, halving resolution increments for a smooth and uniform path. While the oldest planner in Nav2, it is highly optimized and remains one of the most effective holonomic planners in terms of plan time. Although fast, it produces paths 5% longer by design, as the planner is made to prefer slightly longer routes to minimize cost, which provides additional margins from collision. The NavFn planner is a good general-purpose option due to its speed and long-

term stability, remaining as the default planner in Nav2.

The **Smac Planner 2D** is a cost grid-A* planner that uses Moore neighbourhoods with an L2 distance heuristic. It is implemented within the Smac Planner Framework, whose planners produce paths within 3% of each other, as they share much of the search and penalty structures. The travel cost between cells is computed as:

$$C_{travel} = d_c \left(1.0 + \omega_{cost} \frac{cost(x, y)}{cost_{max}} \right) \quad (3.14)$$

where d_c represents the distance between cells, ω_{cost} is a configurable weight penalizing high costs, $cost(x, y)$ is the cost at the cell (x, y) , and $cost_{max}$ normalizes the term relative to the penalty weight.

The **Theta* Planner** is an any-angle path planner, which modifies A* with line-of-sight (LoS) checks between the parent of a node and the neighbour being expanded from that node. If there is LoS, a shortcut is considered. LoS checks use Bresenham's ray-casting algorithm to detect collisions between nodes. The Theta* planner adds a cost-aware evaluation, rejecting paths if their integrated cost exceeds a configurable threshold. This ensures paths don't route unacceptably close to obstacles. Theta* excels in environments with long, straight corridors, such as office buildings, warehouses, and retail spaces. Its straight-line path characteristics enhance the predictability of robot movement, making it more understandable to bystanders. In the experiments conducted in [77], this algorithm achieved the shortest path lengths but incurred the longest runtime due to computationally expensive LoS checks. However, with execution times consistently under 100 ms, it remains a viable solution.

The **Smac Hybrid-A* Planner** is a kinematically feasible planner. It extends on traditional grid-A*, by incorporating motion primitives to search a grid map in a tree-like manner while using continuous coordinates linked to discrete cells (x, y, θ) . This ensures paths are both continuous and feasible. The motion primitives consist of straight segments and circular arcs that adhere to the minimal turning radius required for a car-like base. Additionally, the values of the cost map are used to steer the feasible search via an admissible cost-aware obstacle heuristic function. This leads to optimal paths into regions of lower cost, away from obstacles. This method is specifically designed for Ackermann vehicles, however, it has utility for legged, large robots and high-speed robot systems.

The **Smac Lattice Planner** is analogous to the Smac Hybrid-A* Planner with the exception that it leverages offline computed motion primitives that establish a state lattice graph instead of expanding using trees. Additionally, the motion primitives for the Smac Lattice planner can be computed to handle differential-drive in-place rotations and omnidirectional lateral motions, making it suitable for any robot type when matched with a complementary minimum control set. Its path lengths and planning times are within 0.5% and 1.6%, respectively, of the ones obtained by the Smac Hybrid-A* Planner [77]. Both these planners are faster than the holonomic alternatives.

For circular differential-drive robots, the Nav2 maintainers [77] recommend the NavFn planner, for its high performance and highly deployed implementation. Theta* is recommended for robots navigating in environments with long, straight, and potentially non-axially aligned corridors or open spaces. For non-circular differential-drive robots, the Nav2 maintainers recommend the use of a feasible planner to guarantee manoeuvrability in confined spaces where in-place rotation would induce a collision. In cases

where a non-circular robot is able to rotate in place anywhere in its environment, the NavFn planner is recommended instead. This is the case for robots with similar length and width, and with a small size compared to the environment. As this is the case for the robot in use, the NavFn planner was utilized.

Controller Selection

In a similar fashion to the global path planners, Nav2 also provides a selection of controller plugins. These plugins are sometimes denoted as local trajectory planners, as they also consider objectives such as reducing the likelihood of collision, improving smoothness, and minimizing important curvature, acceleration, jerk, etc. This represents an upgrade from simply using velocity control to follow a given path. Similarly to the global path planner selection, controller selection is highly dependent on the robot kinematics, and therefore its type. Table 3.2 showcases the controller plugins provided by Nav2, coupled with their supported robot types and a brief description of the task they execute. The explanation of each controller plugin is given below.

Table 3.2: Supported Robot Types and Tasks for Controller Plugins [78].

Plugin Name	Supported Robot Types	Task
DWB Controller	Differential, Omnidirectional	Dynamic obstacle avoidance
MPPI Controller	Differential, Omnidirectional, Ackermann, Legged	Dynamic obstacle avoidance
RPP Controller	Differential, Ackermann, Legged	Exact path following
Rotation Shim	Differential, Omnidirectional	Rotate to rough heading
VP Controller	Differential, Ackermann, Legged	High-speed path tracking

The **DWB** controller implements the Dynamic Window Approach, which samples velocity commands from the dynamic window of possible velocities, obtained through the robot's dynamic limits. These velocities generate a set of circular arcs as candidate trajectories, which are then evaluated based on critical functions (e.g., goal alignment, path distance), and the best option is selected. **DWB** is the default Nav2 controller, largely due to the historical precedence of the DWA Local Planner in ROS1. This planner excels in configurability, which, while good at first glance, results in a highly complex tuning process where poorly tuned configurations can create extremely suboptimal behaviours.

Model Predictive Path Integral (MPPI) is a predictive trajectory planner. It differentiates itself from the reactive types (e.g. DWB controller), in the sense that it considers temporal information by refining the previous trajectory at each time step to optimize future velocity commands. Unlike reactive planners, MPPI predicts sequences of velocity commands over a time horizon, enabling it to handle dynamic obstacles and avoid local minima effectively, and rarely requires active recovery behaviours. To achieve this, MPPI generates a large number of noisy samples, derived from the previous time-step's optimal trajectory, which are then evaluated, identifying the optimal trajectory. The samples are created randomly and are then added to the previous optimal trajectory's velocities, creating a set of trajectories while applying constraints relative to the velocity limits of the robot base. Each trajectory is scored by a configurable set of critics functions, based on the following criteria: goal angle, goal distance, obstacle

map, path alignment, path angle, path follow, prefer forward, anti-twirling, and dynamic constraints. After scoring, the optimal trajectory is obtained, and the first velocity commands are executed. The process is then conducted iteratively, refining the trajectory throughout the process. MPPI represents a performance upgrade over the DWB controller, and is expected to become the default local trajectory planner for Nav2 [77].

The **Regulated Pure Pursuit (RPP)** controller is relatively simple compared to reactive and predictive controllers, as its aim is to simply follow the path computed by the global planner without deviation. Pure Pursuit tracks a reference path by computing a lookahead point at a fixed distance from the robot, which determines the curvature and steering commands needed to follow the path. RPP enhances this by incorporating dynamic adjustments to the lookahead distance based on the robot's velocity, ensuring smoother tracking in varying conditions. For instance, at higher speeds, the lookahead distance increases, providing more stability, while at lower speeds, it decreases for greater precision in tight turns. Additionally, RPP dynamically adjusts speed when the robot approaches obstacles, slowing down as the distance to the nearest obstacle falls below a configurable threshold.

Rotation Shim is a reusable module designed for robots capable of in-place rotation, allowing them to align with the path's initial heading before the primary controller, such as DWB or MPPI, takes over to track the path. Without the Rotation Shim, controllers may exhibit suboptimal behaviour, such as spiralling arcs or partial 3-point turns, when aligning to a new heading.

Vector Pursuit is a geometric path-tracking method using screw theory. It calculates two instantaneous screws: one for translation, centred on a circle through the vehicle's origin and the lookahead point, and one for rotation, centred at the vehicle's origin. These screws combine into a single motion screw, determining the turning radius for accurate path tracking. It is more advanced than RPP as it also takes the path heading into account.

Ultimately, Nav2 maintainers [77] recommend the MPPI trajectory planner for both circular and non-circular differential drive robots. This controller represents the state of the art in terms of performance, particularly in scenarios with dynamic agents. This controller, however, comes at a high computational cost and is not available for the Humble distribution of ROS2. The controller's performance is dependent on factors such as the robot platform and its computational capabilities, as well as the application scenario. With this in mind, the DWB, RPP, and VP controllers were all tested, with the objective of determining experimentally which one would be the better fit for the UGV and scenario in question.

Chapter 4

Experimental Results

This chapter focuses on the methodology and results of the experimental procedures conducted to validate the multiple UGV features. Section 4.1 evaluates the accuracy and responsiveness of the ROS2 Control framework by comparing commanded and measured velocities. Section 4.2 assesses the developed collision avoidance system. Section 4.3 analyzes the performance of SLAM Toolbox for mapping and localization, using outdoor and indoor trials with quantitative metrics like ADNN and Root Mean Square Error (RMSE) to compare against ground truth data. Finally, section 4.4 tests the Navigation 2 framework's path planning and trajectory control, employing metrics such as path length, lateral error, and execution time across various controller configurations in a challenging obstacle course.

4.1 ROS2 Control Validation

At the basis of every feature implemented is the assumption that the velocity commands are being swiftly and accurately executed by the UGV. This assumption must be validated experimentally. This validation can be achieved by comparing the velocity command topic (`/cmd_vel`) received by `ros2_control`'s `diff_drive_controller` with the `/diff_cont/odom` topic that it publishes. This latter topic consists of the UGV's odometry, containing the linear and angular velocity of the robot's body, based on the wheel velocities estimated by the wheel encoders. Figure 4.1 displays the typical results obtained from this comparison, for the SLAM trials described in section 4.3.2 (trial 18 in this case).

The figure showcases that the `ros2_control`'s implementation is working as intended, with a few take-aways:

- The motors respond rapidly to new velocity commands, as the visible delay from the instance a command is taken to the instance actuation starts is minimal.
- There is a mismatch in the desired and measured velocities when the velocity commands shift drastically. This is expected, as the variations in the desired velocity can be almost instantaneous, while the measured velocity is limited by the acceleration of the motors.
- Sudden drops in the linear velocity were detected around seconds 16 and 21 of the trial. These

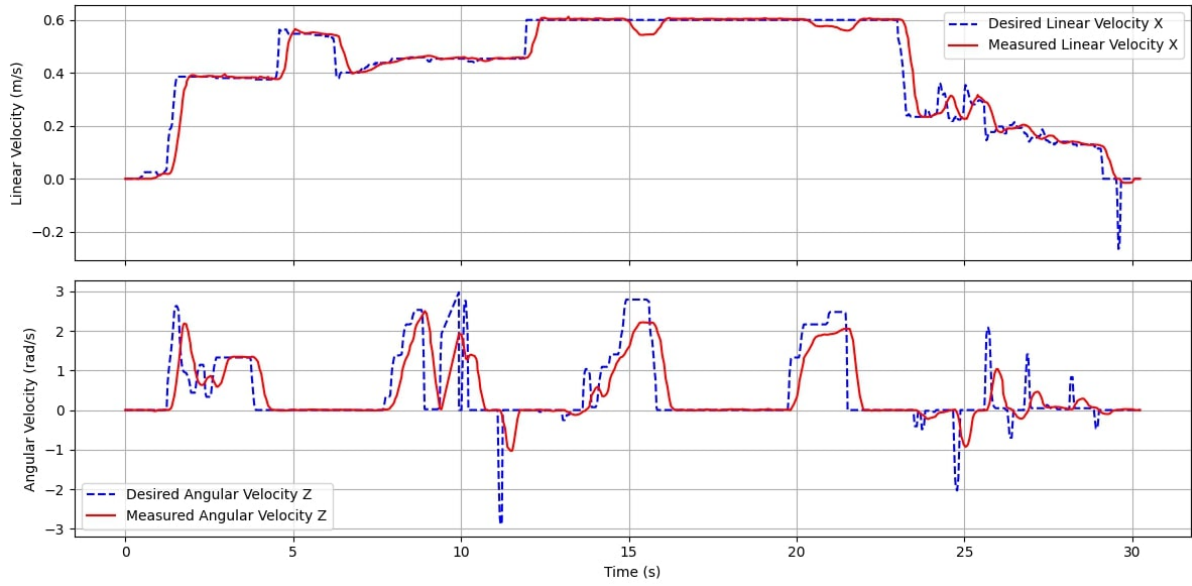


Figure 4.1: Velocity commands vs measured velocity on SLAM trial 18.

coincide with peaks in the desired angular velocity, which were also not met. This can be explained by the limitation in the speed of the motors. While a linear velocity of 0.6 m/s and an angular velocity of 3 rad/s are achievable independently, when combined in a single command, they demand a motor speed that is simply not attainable.

Figure 4.2 compares the desired and measured angular velocities for the robot's left and right wheels on the same SLAM trial.

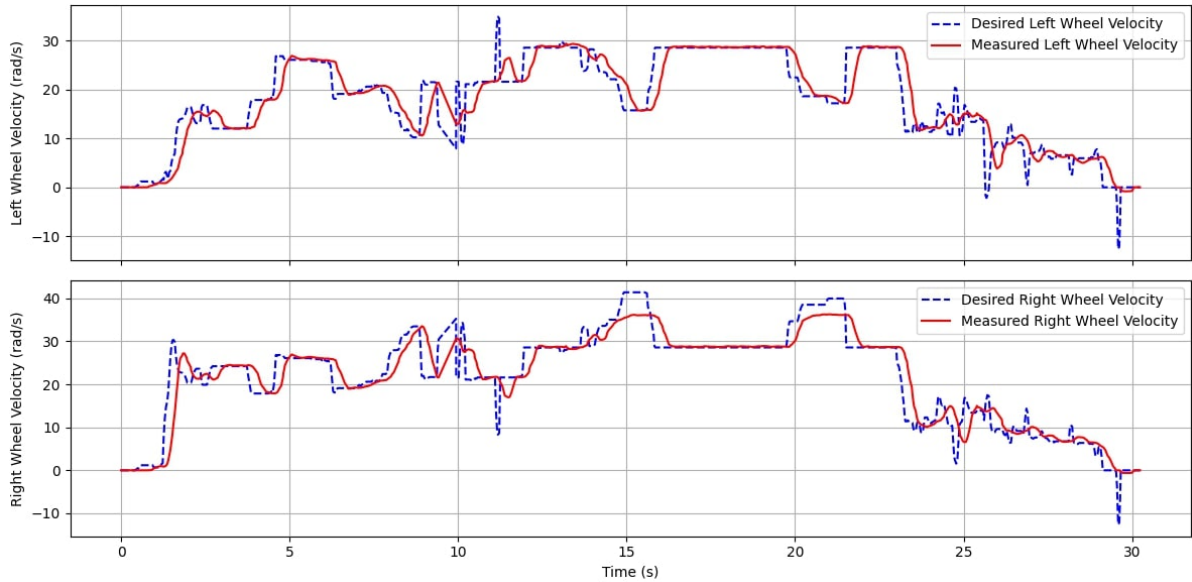


Figure 4.2: Commanded vs measured wheel angular velocities on SLAM trial 18.

The data reveals that in this trial, the maximum commanded velocity reached 41.397 rad/s (for the right wheel), which exceeds the physical limits of the motor and was therefore not attainable. The maximum measured velocity was 36.282 rad/s (for the right wheel also), slightly below the no-load

speed of 350 RPM (equivalent to 36.652 rad/s) specified by the motor manufacturer [10].

4.2 Obstacle Avoidance Algorithm Testing

Two scenarios were conceived in order to test the collision avoidance algorithm developed. In both scenarios, the UGV was teleoperated forward at the maximum linear speed of 0.6 m/s, with a null angular velocity command. This was established with the intention of simulating the use case defined in section 3.3.2, where the focus is placed on assisting the user in traversing a large area quickly, through teleoperation. Then, by analysing the user's velocity commands and how the distance to obstacles influences the corrective commands, it is possible to ascertain if the implementation is functional. Additional metrics can be used to assess the algorithm's performance, such as the UGV's trajectory, the smoothness and velocity of the robot's motion, and most importantly, the occurrence of collisions.

The first scenario involved teleoperating the UGV in a straight line from a stationary point, to a wall perpendicular to the trajectory (placed at a distance of 2 m from the origin point), therefore forcing the algorithm to steer the UGV away from a collision. This scenario can be seen in figure 4.3a.

The second scenario sought to examine the algorithm's performance in a narrow corridor. The robot was placed at one end of the corridor and commanded forward at the maximum linear speed of 0.6 m/s as stated earlier. Due to the angular velocity imposed by the collision avoidance algorithm, the UGV follows the corridor without collisions, exiting at the other end. A second variation of this scenario introduced a wall in a section of the corridor, blocking it entirely. This variation was incorporated to validate the collision avoidance implementation response when an obstacle is detected in zone 1, effectively stopping the UGV. Figure 4.3b showcases this scenario. Several trials were conducted for both scenarios, without recording any collision.

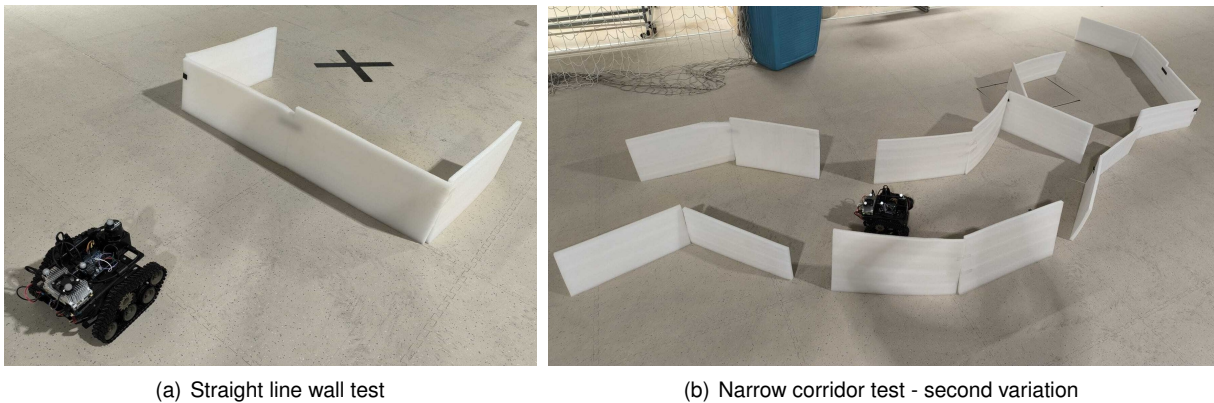


Figure 4.3: Collision avoidance algorithm test scenarios.

Figure 4.4 displays the different velocity commands published during a trial of the straight line wall test. The velocity commands provided by the user through teleoperation showcase the stationary state of the UGV initially, and the acceleration to a 0.6 m/s command, which remained constant for the duration of the trial, with an also constant 0 m/s angular velocity command. The following remarks can be inferred regarding the collision avoidance algorithm and the twist multiplexer:

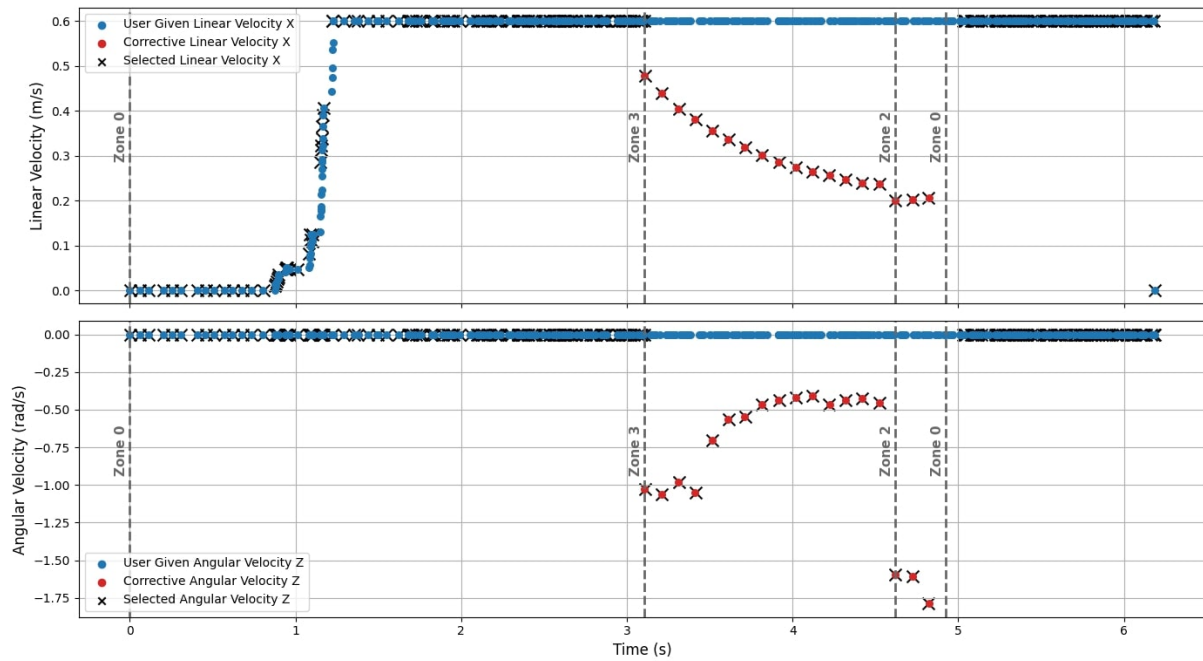


Figure 4.4: Teleoperation and corrective velocity commands on a straight line wall test trial (Fig. 4.3a).

- The corrective commands are issued almost immediately upon entering a new zone;
- The corrective linear command decreased throughout zone 3, as the distance to the wall diminished. The absolute corrective angular velocity also decreased as the UGV steered away from the wall. This behaviour goes in accordance with equations (3.11) and (3.12);
- The cross markings display the commands executed by `ros2_control`. When corrective commands are published, the twist multiplexer prioritizes them over the user-given command, which becomes active again when the UGV ends the evasive manoeuvre.

Figure 4.5 illustrates the trajectory of the UGV throughout this trial. Although the zones in the collision avoidance algorithm are defined relative to the LiDAR sensor's distance readings, the simple geometry of the wall allows the zone boundaries to be visualized in relation to the wall itself. This helps illustrate how entering each zone influences the robot's trajectory.

When considering the LiDAR sensor's trajectory, it can be concluded that:

- The angular corrections applied upon entering zone 3 are not reflected on the robot's trajectory immediately, given its high initial linear velocity and the modest correction applied in this zone;
- When the UGV enters zone 2, its linear velocity is already diminished, and the angular correction applied is more drastic, effectively manoeuvring the robot out of zone 2 as soon as it enters.

The objective of the narrow corridor test was accomplished in all attempts, as the UGV successfully traversed the corridor, collision free. Figure 4.6 presents the velocity commands published during one trial of the second variation of this scenario. Again, it is visible the constant linear velocity command of 0.6 m/s after the initial acceleration, and the constant 0 m/s angular velocity command. Emphasis is

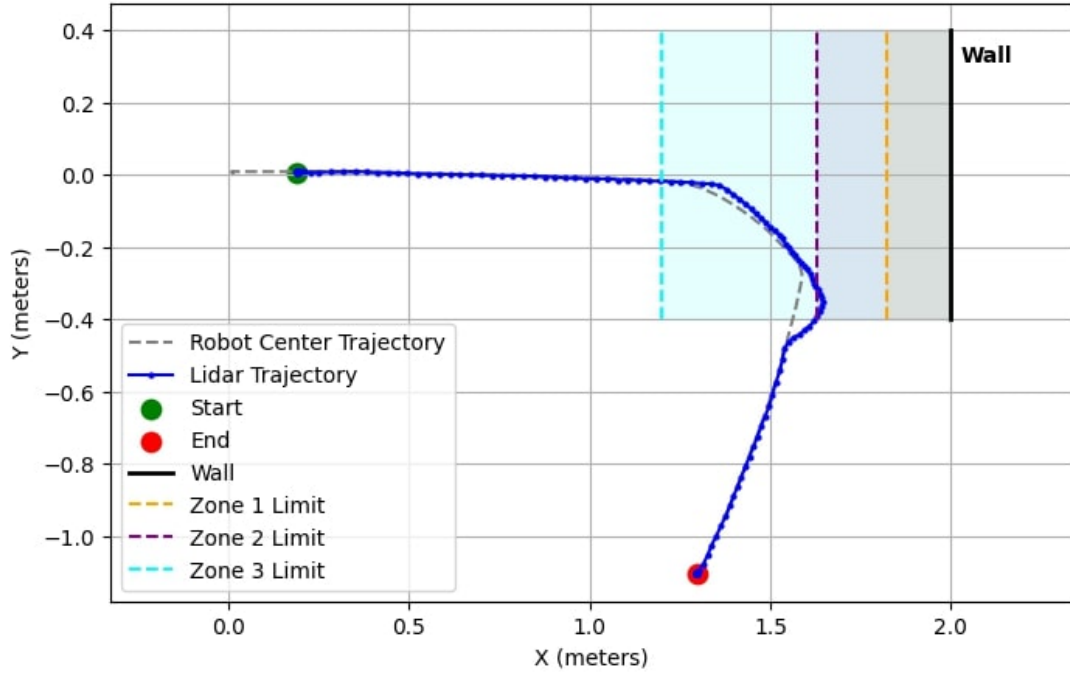


Figure 4.5: UGV's trajectory during a trial of the straight line wall test (Fig. 4.3a).

given on the behaviour conducted upon entering zone 1, where the robot is effectively stopped by the 0 m/s linear velocity command, validating equation (3.7).

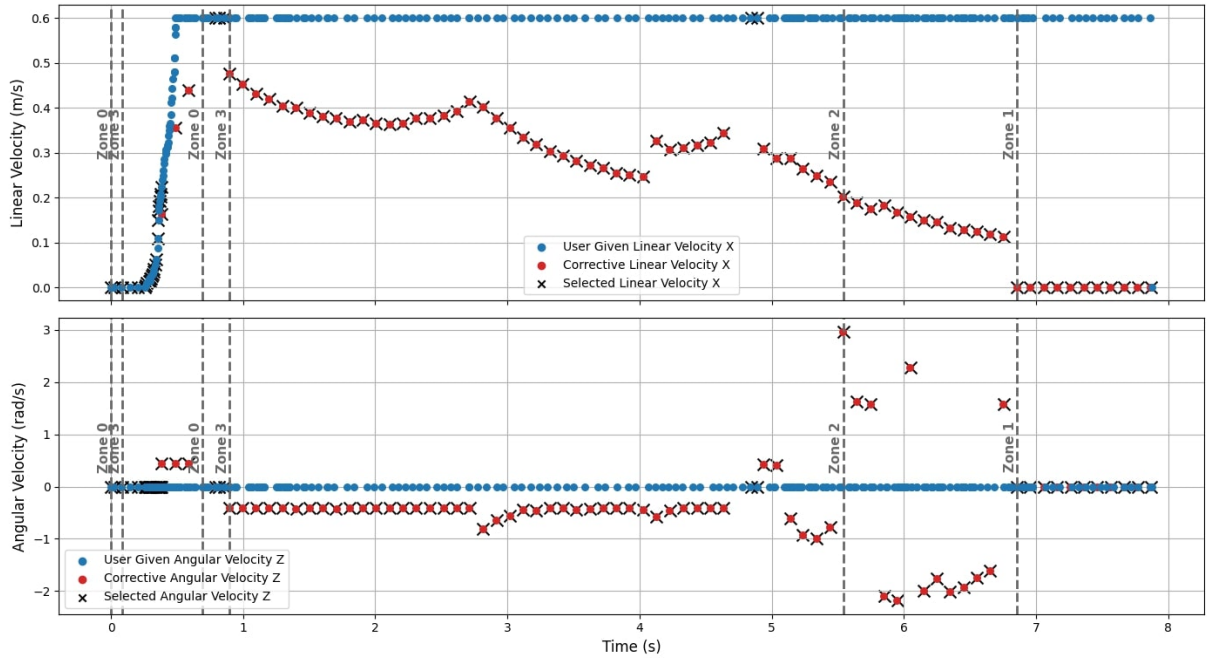


Figure 4.6: Teleoperation and corrective velocity commands on a trial of the narrow corridor test, second variation (Fig. 4.3b).

4.3 SLAM Validation

Visual comparison between the ground truth and the resulting map is not sufficient to properly evaluate SLAM performance, therefore a set of metrics must be established. Santos et al. [80] (followed by [58] and [59]) measured map accuracy using a k-nearest neighbour method, by calculating the euclidean distance from each of the ground truth points to the nearest map point generated by the SLAM algorithm. In [57], map alignment was performed using the ICP (Iterative closest point) algorithm, with the average distance to the nearest neighbour (ADNN) being the map evaluation metric, a metric similar to the k-nearest neighbour method.

As for evaluating the localization component of SLAM, the common procedure is to compare the robot's pose estimated by SLAM to the ground truth pose throughout the robot's activity, and average out the location and orientation errors. This error was calculated using: the root mean square error (RMSE) and the average normalised estimation error squared (NEES) in [81]; the Absolute Trajectory Error (ATE) in [56] and [59]; normalized translation relative error, normalized rotation relative error and absolute trajectory error in [61]; the RMSE in [62].

Beyond the error analysis, papers where SLAM analysis was performed often evaluate the computational resources spent, using CPU load (%) and/or memory load (%) as metrics, as in [80], [82], [58], [59] and [61] (*maximum memory* in this case). Success rate is another possible metric (as used in [61]).

4.3.1 SLAM Map Evaluation

In order to test the UGV's outdoor capabilities, the SLAM functionality was tested in an outdoor scenario, using satellite imagery as the ground truth. The chosen scenario can be seen in figure 4.7. It was selected due to its well-defined borders, in the form of walls, which help define the map, facilitating its comparison to the ground truth. Additionally, its terrain is irregular, composed of "calçada portuguesa", with slight elevation variations. This contrasts with the smooth flooring of the Laboratory Arena, providing a more challenging environment to assess the robustness of both the SLAM implementation and the overall performance of the UGV.

The major challenge when operating the UGV outside was the requirement for a low-latency SSH connection between the host computer and the Nvidia Jetson TX2. While the laboratory possesses a dedicated router, the transition to an external scenario excludes this possibility, unless accompanied by a power source, which can be deemed impractical. The solution found was to use a mobile phone hotspot with a 5G connection, which was shared with both the host and the Nvidia Jetson TX2, creating a common network that enables ROS2 communication. This solution excels in terms of practicality, as it only requires a smartphone to function. However, this approach introduces a higher network latency compared to using a dedicated router, which caused the connection to be inconsistent. Nonetheless, the mapping of the desired area was accomplished, with a successful trial seen in figure 4.9a.

A multi-step process was required to obtain a quantifiable evaluation of the map obtained. Firstly, satellite information from [83] was gathered to create the ground truth. This source's map has the particularity of containing a base layer which defines buildings' edges, which includes the edges of the



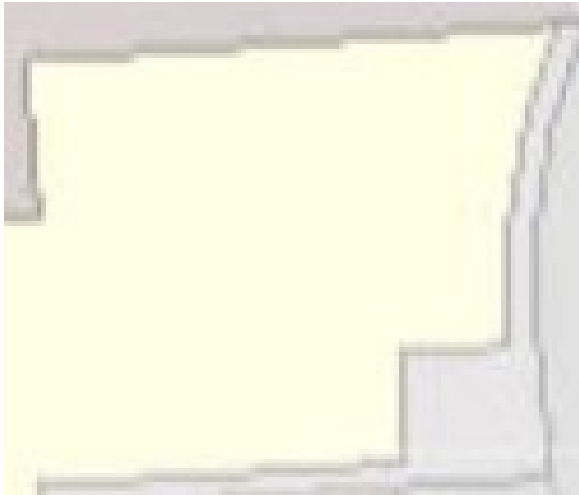
(a) Satellite image of the outdoor scenario.



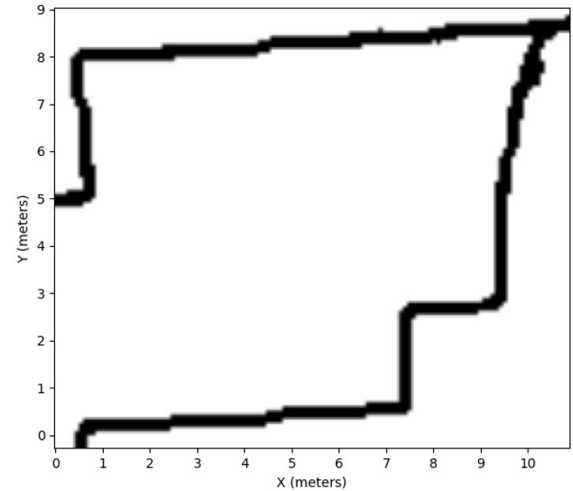
(b) The outdoor scenario.

Figure 4.7: Outdoor scenario used to test SLAM mapping performance.

area of study, as seen in figure 4.8a. This image was cropped to include only the mapped area, and a set of image processing tools from the Python OpenCV library were applied to transform it. A Canny edge detection algorithm was used to select the inner edges, which were then subject to Gaussian blur and dilation as to be of a comparable thickness as the boundaries in the SLAM map. Finally, the image was binarized, resulting in figure 4.8b, which was used as the ground truth in calculations.



(a) Buildings' edges based on satellite imagery from [83].



(b) Ground truth obtained after image processing.

Figure 4.8: Different satellite imagery of the outdoor scenario.

The map obtained from SLAM Toolbox was cropped and binarized. To align both maps, a global RANSAC-based feature matching approach was applied, resorting to Python's Open3D library [84]. As the maps were in the form of binary images, the occupied regions (black pixels) were extracted and represented as 3D point clouds with the assumption that all points lie in the same plane ($z = 0$). Voxel downsampling was then applied, reducing the number of points while preserving the maps' overall structure, as to decrease the computational cost. The Fast Point Feature Histograms (FPFH) were computed for each point cloud, followed by the RANSAC-based registration approach. This method

iteratively sampled correspondences between the two point clouds, estimating a transformation matrix, while rejecting outliers through the use of an edge length constraint and a distance-based filter. The transformation matrix was then applied to the SLAM-derived point cloud to align it with the ground truth.

RANSAC's performance is highly dependent on two important parameters, more specifically, the voxel size, which determines the level of downsampling applied to the point cloud before computing FPFH, and the distance threshold multiplier, which defines the maximum allowable correspondence distance during feature matching, scaled relative to the voxel size. A smaller threshold ensures stricter matching but may reject correct correspondences, while a larger threshold increases tolerance at the risk of introducing outliers. Grid search was conducted to determine the optimal values for these parameters, systematically testing different combinations of voxel sizes and distance threshold multipliers. Each combination was evaluated over multiple runs to mitigate variability in the registration results, as RANSAC is non-deterministic. The best combination of parameters was selected based on the lowest average distance to the nearest neighbor (ADNN), as it was an error metric commonly used in the literature review, as described earlier, facilitating a fair assessment of the mapping's performance. The ADNN can be calculated using (4.1):

$$\text{ADNN} = \frac{1}{N} \sum_{i=1}^N d_i \quad (4.1)$$

where d_i is the distance between cell i of the ground-truth map and its nearest occupied cell in the generated map, and N is the total number of occupied cells. Multiple trials were then performed using the best parameter combination. An early stopping criterion was introduced, which terminates the process if a trial yields an ADNN error that falls within a stipulated upper percentile. This method resulted in the map alignment showcased in figure 4.9b.

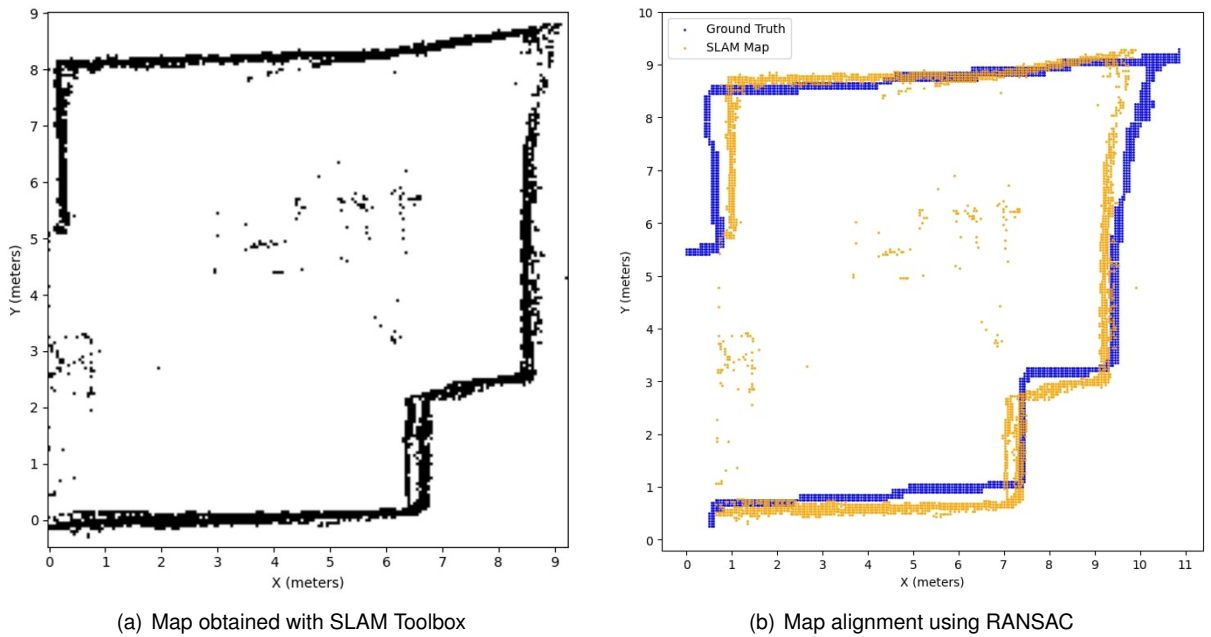


Figure 4.9: Map obtained with SLAM and alignment with the ground truth.

The resultant map from SLAM Toolbox (figure 4.9a) obtained an ADNN value of 3.278 cells, with a

resolution of 0.05 m per cell, which translates to an estimated error of 0.164 m .

This error is in line with the errors obtained in the reviewed literature, although slightly higher. This can be explained by the difference in the application scenario, as results presented in the experiments mentioned were either accomplished in simulation or, when in a real-life robot, in a controlled environment with a smooth surface. The increment in slippage caused by the irregular terrain and its elevation changes led SLAM to somewhat miscalculate the robot's position and orientation. Visual analysis of the SLAM/ground truth map comparison (4.9b) demonstrates the impact of the orientation error, particularly in the top-right section of the map, which caused the straight wall to be mapped as if it was bent slightly upwards. This error is further exacerbated due to the scenario's lack of distinguished features, which can cause false matches for pose estimation and loop closure optimization. Additionally, the execution of this experiment in an outdoor setting may influence the LiDAR's performance, as the used model is designed for indoor applications only, as stated in its specifications [19].

4.3.2 SLAM Localization Evaluation

Evaluating the localization aspect of the SLAM Toolbox implementation requires a ground truth that is able to report the UGV's poses throughout its operation. For this effect, the Robotics Arena of the Laboratório de Controlo, Automação e Robótica of the Department of Mechanical Engineering of Instituto Superior Técnico was chosen, as it is equipped with a Qualisys 3D infrared tracking system, equipped with 10 cameras, which serves as an accurate ground truth. SLAM Toolbox has already been validated in this arena by the work of Gonçalves [85], both in terms of map quality and pose estimation error. This evaluation, however, was conducted solely through visual inspection, without the calculation of any quantitative metrics. Additionally, the computational cost of the procedure was not considered.

An obstacle course was assembled in the arena, as seen in figure 4.10.

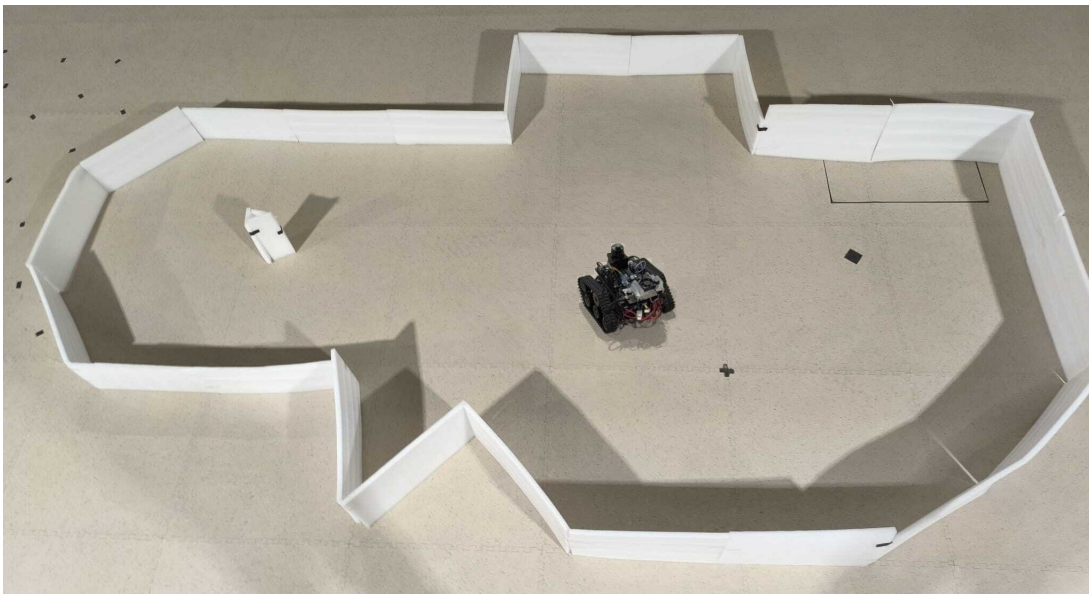


Figure 4.10: Obstacle course assembled in the Robotics Arena.

The UGV was then teleoperated within the enclosed area defined by the mounted walls. While this

occurred, the UGV's position was recorded in real time, as estimated by the arena's tracking system and by SLAM Toolbox, using its mapping mode, and starting with an empty map. The first trajectory had the UGV complete a lap around the course, starting at the origin, circumventing an obstacle and returning to the origin. For convenience, the UGV was set to always start the trials at the origin of the Qualisys tracking system reference frame. This ensures the Qualisys reference frame and the reference frame of the SLAM map match, as SLAM Toolbox defines the map origin based on the robot's initial pose by default. In total, 15 trials were conducted with this trajectory. Figure 4.11 showcases the map obtained from a successful trial, and the UGV trajectory as defined by the two different sources.

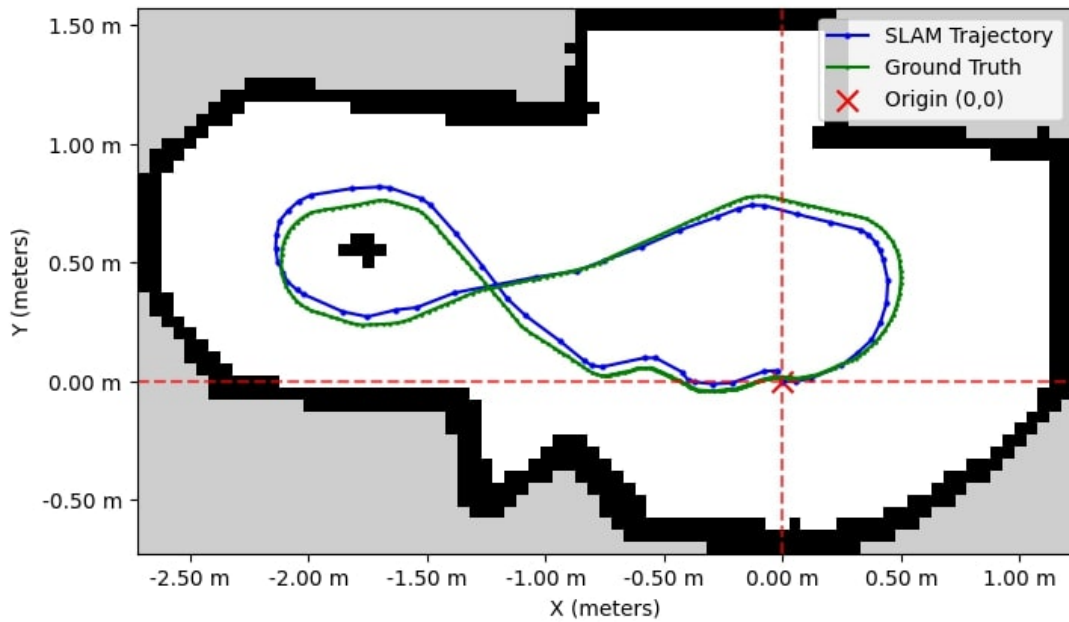


Figure 4.11: Trajectory comparison: Qualisys vs SLAM Toolbox.

The second trajectory tested had the UGV complete two laps around the track instead of one. 10 trials were conducted with this setup, with the objective of assessing if further teleoperation of the UGV would improve map quality and consequently reduce the pose estimation error. The 25 trials were then analysed based on map accuracy, as perceived visually, and the position and orientation errors of the UGV in each trial. SLAM Toolbox publishes estimated poses at a varying frequency, depending on the robot's motion. At first, to compute the pose estimation error, each estimated pose was matched with the corresponding ground-truth pose based on their timestamps. The clocks on the arena computer and the Nvidia Jetson were synchronized to ensure accurate time alignment. The Qualisys motion capture system publishes the UGV's pose at a fixed frequency of 20 Hz. This results in a maximum possible timestamp mismatch of 0.025 seconds. At the maximum linear speed of 0.5 m/s, this can introduce an additional position error of up to 1.25 cm, while at the maximum angular speed of 3.0 rad/s, the timestamp mismatch can result in an orientation error increase of up to 4.3°. To mitigate this issue, the closest earlier and later measurements from the Qualisys system were interpolated to estimate the ground truth pose at the exact timestamp of each recorded SLAM pose. The position error at each time instance was calculated using the Euclidean distance between the interpolated ground-truth position

(x_{gt}, y_{gt}) and the position estimated by SLAM Toolbox (x_{slam}, y_{slam}) as in equation (4.2).

$$e_{pos} = \sqrt{(x_{slam} - x_{gt})^2 + (y_{slam} - y_{gt})^2} \quad (4.2)$$

The orientation error was computed as the difference between the estimated SLAM Toolbox (θ_{slam}) and interpolated ground-truth (θ_{gt}) yaw angles, adjusted to remain within $[-180^\circ, 180^\circ]$ to prevent discontinuities caused by angle wrapping, as shown in equation (4.3).

$$e_\theta = ((\theta_{slam} - \theta_{gt}) + 180^\circ) \bmod 360^\circ - 180^\circ \quad (4.3)$$

To analyze the error throughout an entire trial, the RMSE of both the position and orientation errors was computed, using:

$$\text{RMSE}(e) = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2} \quad (4.4)$$

where e_i represents the position or orientation error at each time instance i , and N is the total number of time instances in the trial. By weighing these factors, a trial was identified as the best, from which the corresponding map was selected.

For the third and final set of trials (10 trials), this map was loaded, and SLAM Toolbox was set to run in its localization mode. The UGV was tasked with completing three laps around the course with this setup in order to validate the accuracy of the pose estimation over an extended operating period.

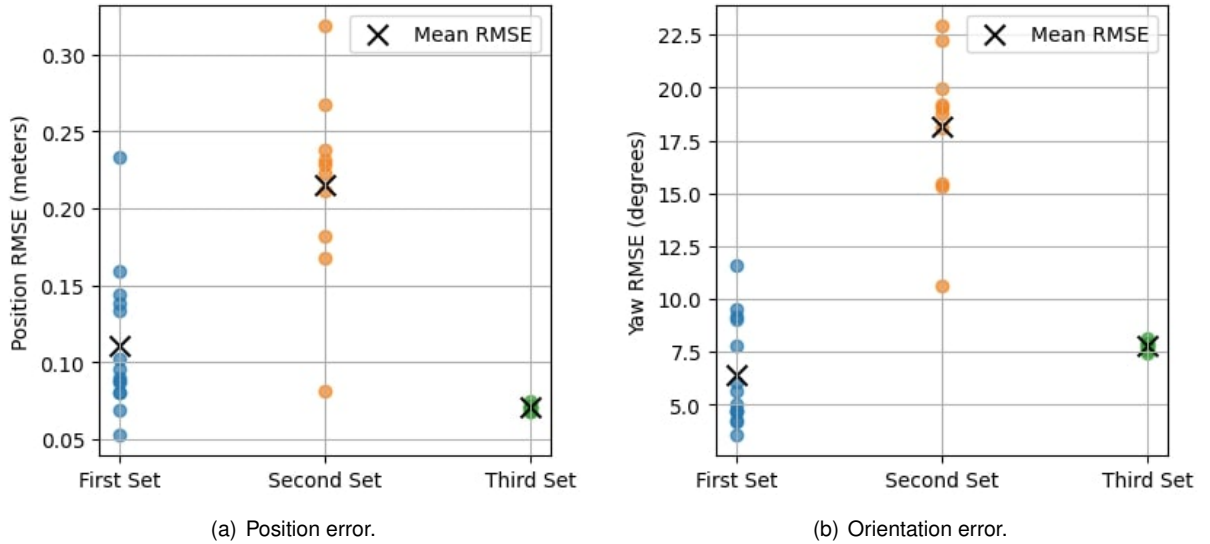


Figure 4.12: Position and orientation RMSE for the three sets of trials.

The RMSE of the position and orientation errors for the 35 trials can be seen in figure 4.12, from where certain conclusions can be drawn:

- Teleoperating the robot further while mapping increased both the position and orientation RMSE. This is a reflection of the poor quality of the maps generated in the second set of trials. The problem was observed when conducting the sharp rotations at the end of the map, most prominently

on the left side, as it is more accentuated. While completing the corner, SLAM Toolbox underestimated the rotation due to the track slippage effect on the odometry accuracy. This resulted in a misalignment of newly added scans with previously mapped areas, leading to distortions in the global map representation. When conducting two laps instead of one, this error is exacerbated as sharper rotations are performed, causing the map to progressively tilt. In one trial (trial 18 - lowest RMSE in figure 4.12), loop closure was achieved despite this error, and results were on par with the ones obtained from the first set. The inability of SLAM Toolbox to perform loop closure consistently can be partially explained by the small size of the mapped area and its lack of distinctive characteristics, as previous studies have shown that these factors can induce false matches for pose estimation and loop closure optimization [58];

- The third set of trials showcased a fairly low mean RMSE for both position and orientation, congruent with the literature referenced in section 4.3. Emphasis is given on the consistency of this set of trials, demonstrated by the low variance of the errors from trial to trial, in comparison with the first and second sets. The position and orientation of the UGV, as estimated by SLAM Toolbox and by the ground truth, throughout a trial of this set (trial 26) can be seen in Figure 4.13;
- Based on these results, the most effective approach when using SLAM with this setup, particularly if the area of operation remains consistent, is to begin with mapping mode, ensuring that each location is visited once, while actively monitoring the map creation. Once a satisfactory map is formed, the process should be stopped and the map saved. Thereafter, running SLAM Toolbox in localization mode using the saved map is recommended to ensure no permanent negative changes are made to the map, thus guaranteeing consistently accurate pose estimation.

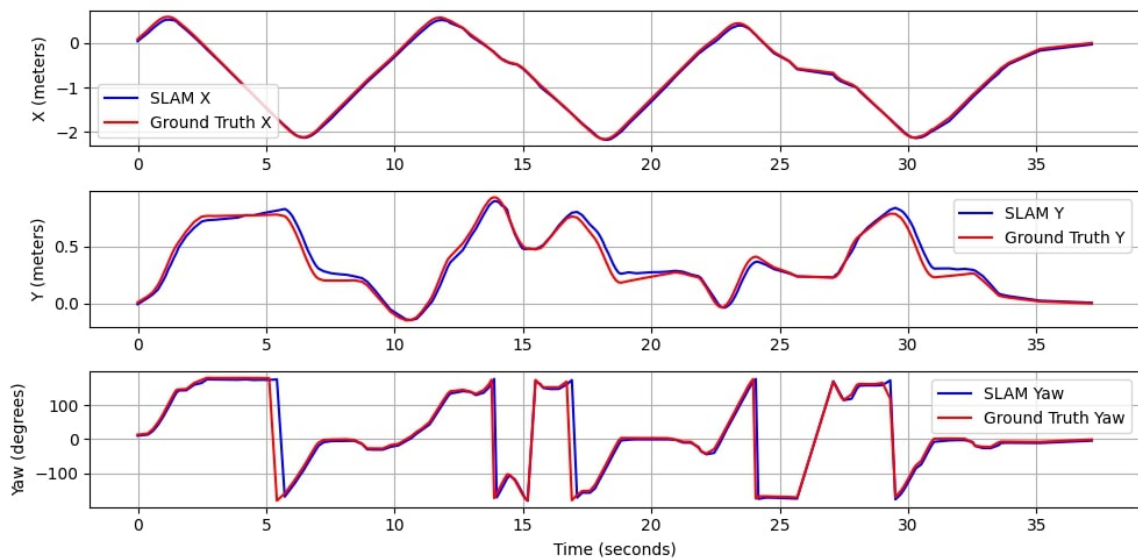


Figure 4.13: Position and orientation throughout trial 26.

It is important to note that when comparing two trials in the mapping mode, the pose accuracy at the end of the trial is often more important than the average accuracy throughout the trial, as it is

based on the map quality at each instance, which can improve rapidly when loop closure occurs, or deteriorate rapidly in case of extreme slippage. For example, in trial 18, mentioned earlier, the RMSE for the position and orientation was 0.082 m and 10.64° , respectively, while after loop closure occurred, the absolute position and orientation error stabilized at approximately 0.008 m and 3° , as seen in figure 4.14.

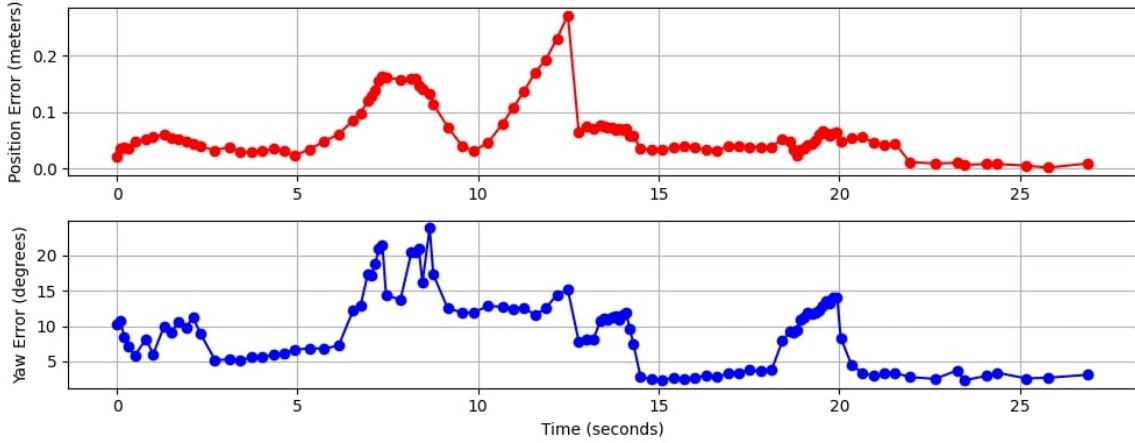


Figure 4.14: Position and orientation error throughout trial 18.

4.3.3 SLAM Computational Cost

The computational cost of the SLAM Toolbox was a limiting factor in this study. To keep SLAM running smoothly in real time, certain parameters had to be adjusted to lighten the computational load, most noticeably by reducing the map resolution and increasing the minimum distance of travel before processing a new scan, which may have negatively impacted the results in terms of mapping and pose estimation accuracy. In order to quantify the computational resources utilized by SLAM Toolbox, the CPU and memory usage were recorded during the 35 trials, using Python's *psutil* library. These measurements were performed every second and averaged over each trial. CPU usage was measured as a percentage of a single core, with values exceeding 100% indicating multiple cores being utilized. Memory usage was recorded as the percentage of total available RAM consumed by SLAM Toolbox. The Nvidia Jetson TX2 used possesses a CPU with 6 cores and 7.7 GB of available RAM.

Table 4.1: Mean ($\bar{x}$), and standard deviation (σ) of CPU and Memory Usage across trial groups.

Trial Group	$\bar{x}_{\text{CPU}}$ (%)	σ_{CPU} (%)	$\bar{x}_{\text{Memory}}$ (%)	σ_{Memory} (%)
First Set	24.10	6.73	0.51	0.02
Second Set	31.18	3.85	0.52	0.03
Third Set	79.71	6.81	0.62	0.02

Table 4.1 presents the summary of these measurements, from which several inferences can be drawn:

- The memory used by SLAM Toolbox is negligible;

- There was a slight increase in CPU usage from the first to the second set of trials. This is expected based on the principles of graph-based SLAM, as a longer trajectory results in a larger graph with more robot poses and environmental features, which in turn increases the complexity of the optimization problem and the consequent computational cost;
- There was a substantial increase in CPU usage in the third set. This can be explained by the shift to localization mode, which requires each new measurement to be matched against the entire pre-existing map, resulting in frequent and expensive scan-matching operations. Additionally, the increased number of laps in this trial set results in a much denser set of constraints in the pose-graph, further exacerbating the computational load. Figure 4.15 showcases the CPU usage throughout a trial of this set (trial 34), revealing an upward trend over time, with usage peaking above 200% by the trial's conclusion. This pattern was consistent across all trials in this set.

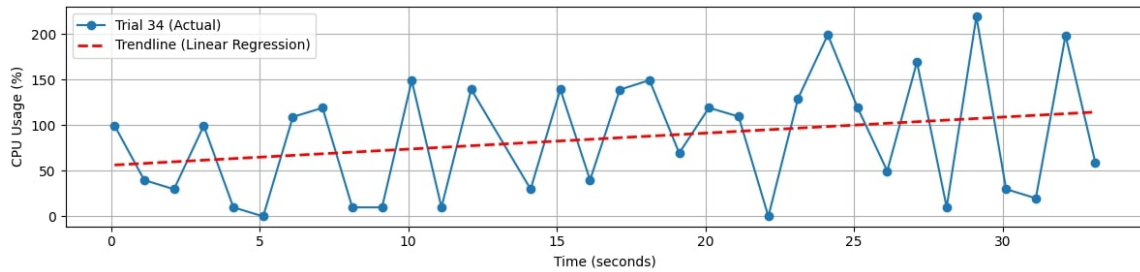


Figure 4.15: CPU usage throughout trial 34.

4.4 Waypoint Navigation Validation

In order to analyse the waypoint navigation solution implemented, evaluation metrics must be selected, for both the path planning and the trajectory control aspects of the implementation. For path planning, Liu et al. [63] and Macenski et al. [77] used the metrics of: path planning time and path length. For path tracking, lateral error is the most prevalent metric, used in [70], [71], [72], with the heading error also incorporated in [73] and [75]. Other metrics evaluate overall navigation performance, incorporating both aspects of planning and tracking. Wahab et al. [66] introduces metrics of this type, such as: success rate (also in [63]), number of collisions, execution time, battery consumption, and travel distance.

The experimental procedure to validate the implemented solution was also conducted in the laboratory's arena, described in section 4.3.2. Several obstacles were placed in it, creating a challenging environment for path planning and execution, which included several narrow passages, visible in figure 4.16.

Firstly, the area of application was mapped gradually, starting from an empty map. Contrary to the procedure in section 4.3.2, the UGV's motion in this phase was not induced through teleoperation, but through the waypoint navigation implementation itself. The robot was given waypoints manually in Rviz until a map deemed adequate was constructed. This was done to confirm the validity of the solution in unknown environments, which was proved, as the waypoints given in previously unaccessed locations



Figure 4.16: Obstacle course assembled to test waypoint navigation.

were successfully reached. The built map was then loaded with SLAM Toolbox, concurrently with Nav2, creating the global costmap of the obstacle course. Given the costmap, a set of 10 waypoints was defined randomly to avoid potential bias from manual selection. This was accomplished using a Python script in which several conditions were specified: the area was restricted by limiting the possible X and Y coordinates of the waypoints (final area of 6×2.5 m); a maximum cost threshold was set to ensure that waypoints were only placed in navigable areas, preventing their placement on obstacles; consecutive waypoints were set to be at a minimum Euclidean distance of 1 m.

With the aim of comparing the performance of the Nav2 implementation with different controller plugins, each set of 10 waypoints was executed with different configurations: NavFn planner with DWB controller; NavFn planner with RPP controller; NavFn planner with VP controller. Two sets of 10 waypoints were tested in total, with each configuration. All waypoints were reached, which indicates a success rate of 100%. Additionally, no collisions occurred throughout the entire process. A configuration using the Theta* planner with the DWB controller was also tested. This planner, however, created paths that routed near obstacles. While this did not cause collisions, it often forced recovery behaviours, which increased the execution time considerably. For this reason, configurations using the NavFn planner were preferred. In total, all 60 waypoints were executed, resulting in a success rate of 100%. Additionally, no collisions occurred throughout the entire experiment. The global costmap based on the saved SLAM map, marked with the waypoints and paths generated in the first trial can be seen in figure 4.17.

It is important to note that figure 4.17 is only illustrative of the state of the costmap at the beginning of navigation, as the costmap is not static, being updated constantly. Since each path is generated only when its corresponding waypoint becomes active, it is computed based on the costmap at that specific

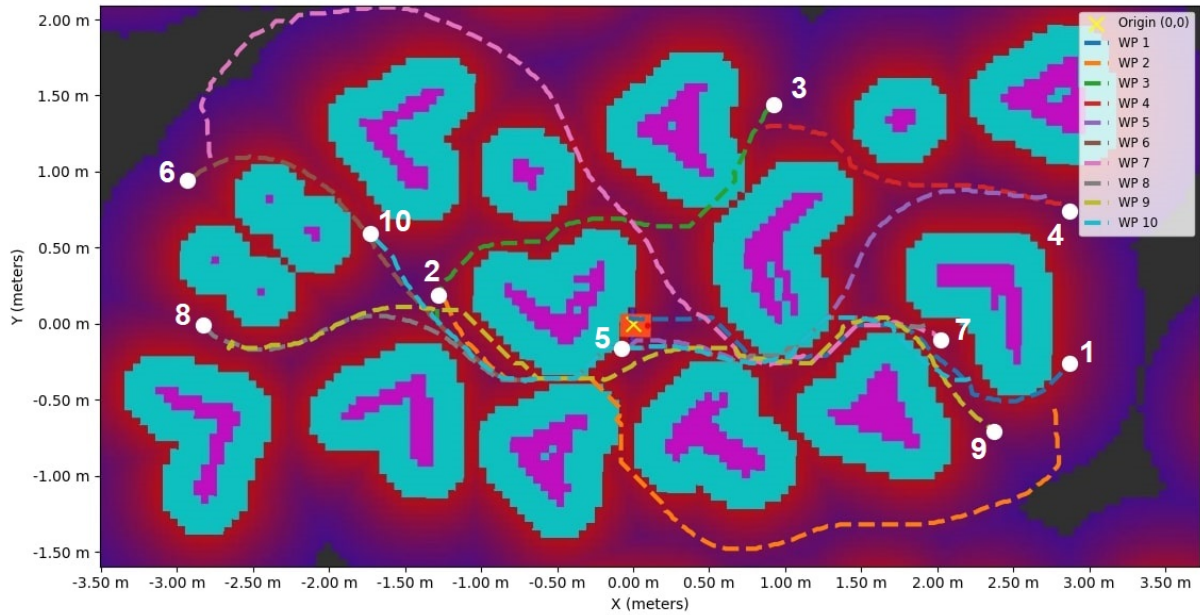


Figure 4.17: Global costmap of the obstacle course assembled in the Robotics Arena with waypoints and paths generated in the first trial.

moment, which may differ, however slightly, from its previous state. A noticeable mismatch can also be observed between each waypoint and the start of the next path. This is the result of two factors: there is a tolerance for the distance that the robot needs to be from a waypoint for it to be considered reached; each path consists of a sequence of poses, with the first one being already offset from the robot's actual position at the time of planning. The changing costmap, coupled with the variation of the robot's final pose due to the goal tolerance, reduces the reproducibility of the experiment, as the planner will generate different paths (while only slightly), even when using repeated waypoint combinations. This is inevitable, as the costmap updates are what enable the Nav2 framework to function in changing environments with dynamic obstacles, while the goal tolerance prevents the robot from dancing and twirling around the goal, trying to meet the exact waypoint pose. The consistency of path creation can be examined by comparing the length of the planned paths in different trials, as Figure 4.18 presents. Here, it is visible that the planner generated paths of similar length when presented with the same waypoints, although with slight variations. Waypoint 3 of the second set is the major exception. While in trial 5 the planner generated a path through the obstacle course, for trials 4 and 6 it created paths that circumvented the course entirely, resulting in a much higher path length.

4.4.1 Path Planner Evaluation

The reduced number of trials affected by the Theta* planner prevents a fair statistical comparison of the performance of both planners. Nonetheless, the metrics mentioned in 4.4 can be used to quantify the performance of the NavFn planner. Firstly, planning time was measured as the time elapsed from when a goal pose was received to when an executable path was generated. The results for each trial can be seen in Figure 4.19.

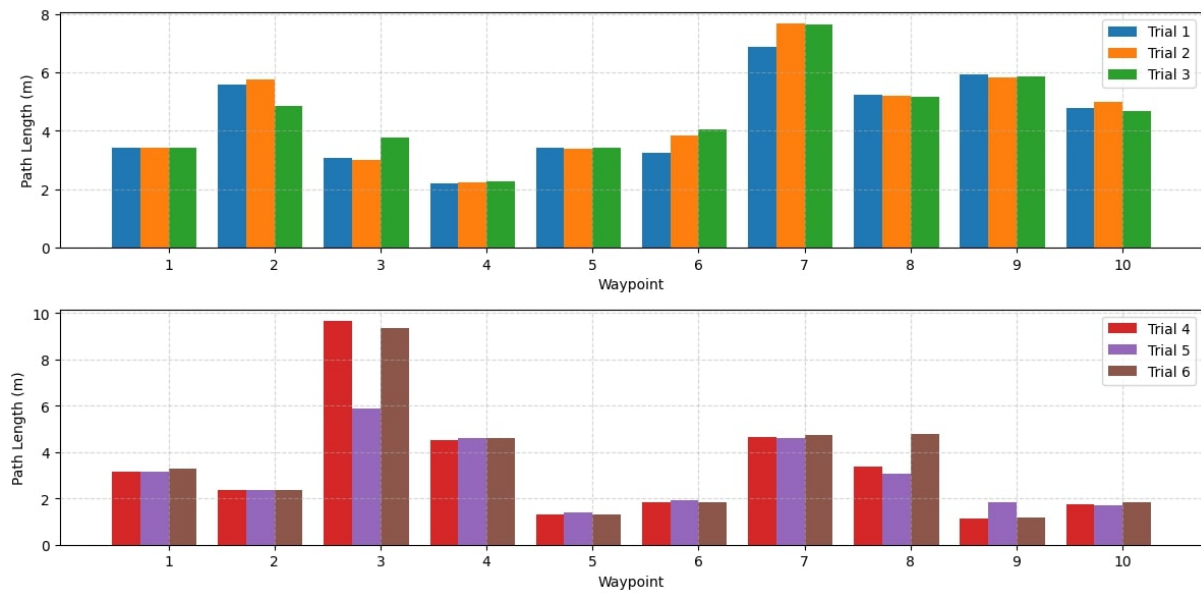


Figure 4.18: Planned path length across different waypoints and trials.

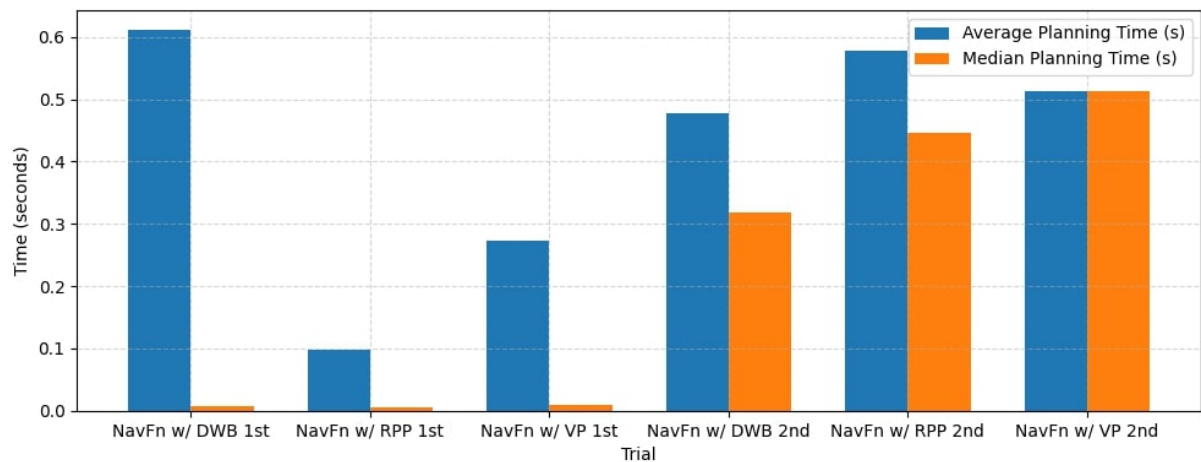


Figure 4.19: Average and median planning time in each trial.

A common problem that arose was the frequent abortion of goals instantly upon receipt. This forced planning to be repeated, increasing the total planning time. The instances where this occurred highly skewed the average, explaining the discrepancy between the average and the median times in the first waypoint set. In the second set this discrepancy diminished since the median rose, as more instances of this problem occurred. Globally, the average planning time was 0.425 seconds, while the median planning time was 0.008 seconds, demonstrating nearly instantaneous path creation, when disregarding the outlier cases.

4.4.2 Local Planners Evaluation

In order to evaluate the local planner's ability to adhere to the global path, the position of the UGV was recorded throughout the trials. This position estimate is given by SLAM Toolbox and will therefore be in the map coordinates. The SLAM map is also the reference frame of the planner-generated paths, facilitating the calculation of metrics that measure the deviation of the UGV from its given path. Since the path changes constantly, this deviation needs to be measured in relation to the active path at the instance that the pose is recorded. Given the discrete nature of the paths, each path was resampled using linear interpolation at a fixed spatial resolution. Then, for each logged robot pose (x_i, y_i) , the lateral error metric was calculated, defined as the shortest Euclidean distance from the robot's position to the interpolated reference path P' . This was computed using a k-dimensional tree (KD-tree) to enable nearest-neighbour search over the interpolated path points. Formally, the lateral error e is given by:

$$e = \min_{(x_p, y_p) \in P'} \|(x_i, y_i) - (x_p, y_p)\|_2 \quad (4.5)$$

where (x_p, y_p) is a point on the interpolated path P' , and $\|\cdot\|_2$ denotes the Euclidean norm. Figure 4.20 illustrates how this error is calculated, using data from trial 3.

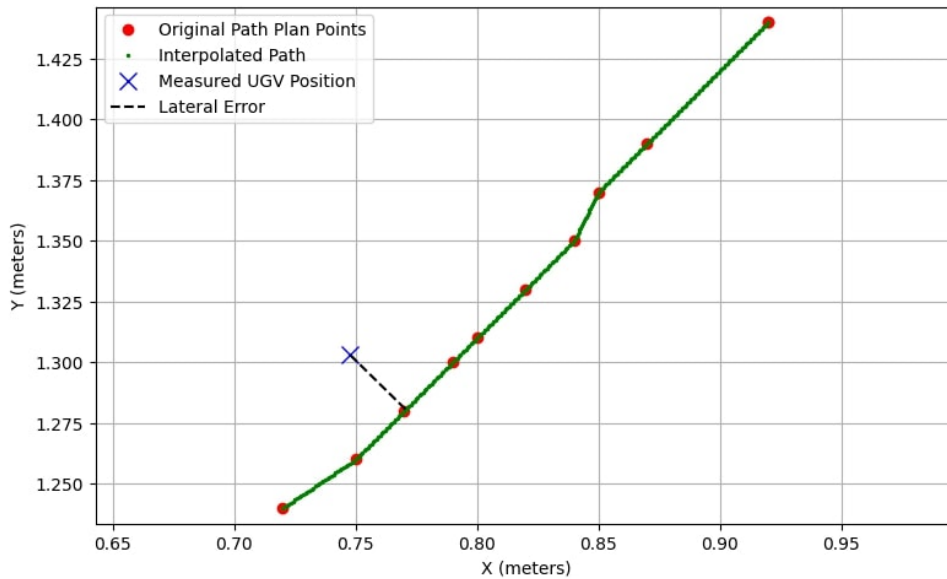


Figure 4.20: Planned path length across different waypoints and trials.

For each trial, the mean and standard deviation of the lateral error of all recorded poses were calculated. The results can be seen in Table 4.2.

As shown in table 4.2, trials using the DWB controller demonstrated slightly lower lateral error values, indicating marginally better performance, when compared to the trials involving the geometric controllers. This contradicts the expected results, as the path-tracking nature of the geometric methods (RPP and VP), which actively attempt to minimize the lateral error by design, would anticipate better performance in this metric when compared to the DWB controller, which does not have these characteristics. Overall, all trials achieved a favourable performance in this metric, with a global mean lateral error of 0.0368 m

Table 4.2: Mean ($\bar{x}$), standard deviation (σ), median of lateral error, and total path length for each trial.

Trial	$\bar{x}_e$ (m)	σ_e (m)	Med $_e$ (m)	Distance Traveled (m)
NavFn w/ DWB 1st	0.034	0.030	0.027	45.110
NavFn w/ RPP 1st	0.039	0.032	0.033	45.587
NavFn w/ VP 1st	0.038	0.034	0.029	46.376
NavFn w/ DWB 2nd	0.033	0.030	0.027	32.035
NavFn w/ RPP 2nd	0.034	0.030	0.026	32.850
NavFn w/ VP 2nd	0.039	0.042	0.029	44.719
Global	0.037	0.034	0.029	—

indicating accurate path tracking.

The total distance travelled is another relevant metric when assessing navigation performance. A smaller value in this metric is not necessarily better, as it can be the consequence of the UGV executing trajectories closer to obstacles, which represent more risk. Table 4.2 showcases the results obtained for this metric. For the first set of waypoints, the difference between the trials, while subtle, goes in accordance with the nature of the controllers, as the DWB controller tends to cut around corners, reducing the distance travelled. The VP controller, as the most accurate path tracker, is expected to cut corners the least, resulting in a longer distance travelled, which was observed. The same order was observed in the second waypoint set. The VP configuration struggled in this waypoint set, as it frequently resorted to recovery behaviours, which increased the distance travelled drastically.

The time taken to complete the course is another crucial metric. Figure 4.21 showcases the duration of each trial, segmented into planning, execution, and idle phases. As defined in 4.4.1, planning time refers to the interval between receiving a goal and generating a path. Execution time was measured as the time elapsed from the moment a path is generated and the moment the corresponding waypoint is reached. Idle time refers to the interval between reaching a waypoint and initiating planning for the next. This delay was intentionally introduced to ensure the UGV came to a complete stop after reaching each goal, making the transitions between waypoints easier to observe.

As shown in Figure 4.21, the RPP consistently produced the shortest total navigation times across both trials, while the DWB and VP controllers both had at least one trial with considerably longer duration.

4.4.3 Navigation Computational Cost

As had occurred with SLAM Toolbox, computational cost was a limiting factor when implementing Nav2. Several compromises had to be made during parameter tuning to minimize this cost, most notably by reducing the frequency of all Nav2 servers. The resolution of the local and global costmaps was also reduced, as well as the frequency of their updates. The CPU and memory usage were recorded using Python's psutil library, and their interpretation was detailed in 4.3.3. Table 4.3 shows the mean, standard deviation, and maximum values for these metrics across all trials.

Memory usage was relatively modest, with low variability and without significant peaks. On the other hand, CPU usage was quite high, averaging at around 1.5 cores, and with peaks of about 2 cores. Table

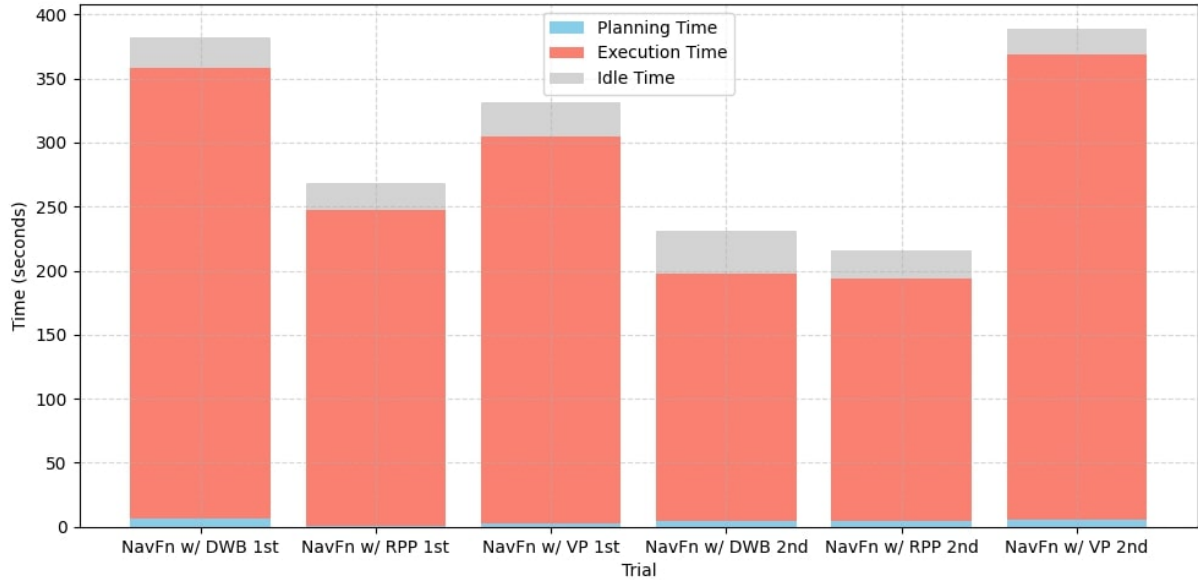


Figure 4.21: Breakdown of planning, execution, and idle time for each navigation trial.

Table 4.3: Mean ($\bar{x}$), Standard Deviation (σ) and Maximum CPU and memory usage across all trials.

Trial	$\bar{x}_{\text{CPU}}$ (%)	σ_{CPU} (%)	Max_{CPU} (%)	$\bar{x}_{\text{Memory}}$ (%)	σ_{Memory} (%)	Max_{Memory} (%)
All Trials	145.38	12.00	204.80	3.78	0.11	3.87

4.4 displays the resources used by the Nav2 controllers, separated by trial group.

Table 4.4: Mean ($\bar{x}$) and standard deviation (σ) of CPU and memory usage by the Nav2 controllers.

Trial	$\bar{x}_{\text{CPU}}$ (Ctrl %)	σ_{CPU} (Ctrl %)	$\bar{x}_{\text{Memory}}$ (Ctrl %)	σ_{Memory} (Ctrl %)
Trials w/ DWB	25.31	2.83	0.53	0.01
Trials w/ RPP	21.45	1.80	0.51	0.03
Trials w/ VP	22.37	1.84	0.50	0.02

The DWB controller has been reported in the literature [77] as being more computationally demanding than alternatives like the RPP controller. While this expectation was confirmed in the experiments, the difference measured was relatively small.

Chapter 5

Conclusions

This chapter presents the final reflections and conclusions drawn from the work developed throughout this thesis and proposes directions for future development and research.

5.1 Achievements

All objectives set in section 1.2.3 were achieved. A functional robot was developed with the capabilities and the features desired, which were successfully validated experimentally.

The robot was built from the ground up, involving a challenging construction process in terms of hardware architecture design, given the compact structure of the Devastator platform and the integration of multiple components necessary to enable the UGV's capabilities. Additionally, incremental upgrades were applied in the electrical system, most notably in the Nvidia Jetson TX2 power supply. This resulted in a robust platform, capable of operating at a variety of scenarios, as proven by its resilience on the outside setting described in section 4.3.1. Emphasis is also given on the modular nature of the hardware architecture design, which separates subsystems such as perception, computation, power distribution, and actuation. This modularity allows for easier maintenance, debugging, and future upgrades or component replacements without requiring a complete redesign of the system. For example, sensor modules are connected via the standard USB interface to the USB Hub, and can be easily added or removed to the platform depending on the mission profile. This is facilitated by the 3D printed camera and radar mount, which can be effortlessly removed if not in use.

The ROS2-based software was containerized using Docker, enabling the reproducibility of the code behind the applications on different robot platforms and computing hardware, aiding further developments. The ROS2 workspace itself was organized intuitively to facilitate development and enhance code readability and maintainability, particularly for future developers. Furthermore, the execution of the UGV functionalities is managed through a centralized launch file, which initiates the various nodes and packages of the UGV system. This approach allows specific features to be easily added, removed, or modified depending on the mission requirements or platform constraints. Such modularity within the ROS2 environment contributes significantly to the scalability and adaptability of the developed solution.

Access to all the developed software is made available in the robot's Github repository [7].

A low-level control solution was successfully implemented, to receive ROS2 velocity commands in relation to the robot's frame and convert them into precise motor actuation. This serves as the foundation for all higher-level functionalities of the UGV (waypoint navigation, teleoperation, and collision avoidance), whose different velocity command sources are structured hierarchically using the Twist Multiplexer, allowing for seamless prioritization and switching between them.

A functional and ergonomic teleoperation solution was successfully implemented using a Dualshock 4 gamepad.

A custom collision avoidance algorithm was successfully implemented and validated. The algorithm preserves user intent while enhancing safety, issuing proportional angular corrections to steer the robot away from obstacles and halt its motion when necessary. Real-world testing on two different scenarios demonstrated the system's effectiveness.

The robot is SLAM capable, through an implementation based on the SLAM Toolbox package. This implementation was successfully validated in an outdoor and an indoor scenario. Quantitative evaluation using the ADNN metric confirmed the map's accuracy, while trials aimed at assessing localization accuracy showed consistent pose estimation with low RMSE values.

A functional waypoint navigation solution was implemented, utilizing the Navigation 2 framework. Three different local planners were tested, in a challenging scenario, where waypoints were assigned randomly. A total of 60 waypoints were reached with a 100% success rate and zero collisions, demonstrating consistent autonomous navigation.

Beyond the implemented functionalities, this work leaves behind a robust and versatile robotic platform, designed with future development and integration in mind. Its modular hardware and software architecture provides a solid foundation for extending capabilities, testing new algorithms, or integrating additional sensors and components, making it a valuable asset for continued research and experimentation.

5.2 Future Work

Improving the main computing unit would significantly enhance the UGV's capabilities. Several compromises had to be made when configuring SLAM Toolbox and Nav2 parameters due to the limited computational resources available, that more capable system would render unnecessary. In addition, upgrading the host computer setup would be highly beneficial. The limitations of the host computer in terms of processing power and graphical capabilities were exacerbated by the use of a virtual machine, which is inherently suboptimal for performance. Upgrading this setup would support the use of the realistic robot model described in section 2.9.1. Additionally, a more powerful host setup could offload certain computational tasks from the main computing unit by running some of the required programs itself. This capability would be indispensable for future work involving image processing, which is typically resource intensive and would otherwise overwhelm the onboard system.

The high latency and instability of the mobile hotspot made consistent outdoor trials difficult, often

disrupting ROS2 communication and limiting the number of successful mapping attempts. Future work should address this limitation by using a dedicated portable router, which would significantly reduce latency and improve network reliability. This enhancement would enable more mapping trials (described in section 4.3.1) to be conducted, providing a more robust and statistically meaningful validation of the mapping performance of the SLAM implementation. Additionally, a stable connection would allow for the evaluation of waypoint navigation and other future functionalities across a wider range of outdoor scenarios.

While the robot demonstrated functional behaviour in dynamic environments with moving obstacles, this capability was not evaluated systematically using defined performance metrics. Future work should include structured testing under controlled dynamic conditions to properly assess and quantify the system's robustness in such scenarios.

Another promising direction for future work is the incorporation of a haptic element to the teleoperation solution. This improvement would allow the remote operator to receive tactile feedback from the controller, improving situational awareness and control.

Bibliography

- [1] Q. Liu, Z. Li, S. Yuan, Y. Zhu, and X. Li. Review on vehicle detection technology for unmanned ground vehicles. *Sensors*, 21(4), 2021. ISSN 1424-8220. doi: 10.3390/s21041354. URL <https://www.mdpi.com/1424-8220/21/4/1354>.
- [2] A. Machado, A. Moutinho, M. Pinho, and T. Silva. Unmanned ground vehicles development in nato. In *International Scientific Conference" Strategies XXI"*, volume 20, pages 192–201. "Carol I" National Defence University, 2024.
- [3] S. International. Sae j3016 taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles. <https://www.sae.org/blog/sae-j3016-update>, May 2021. Accessed: 2024-08-21.
- [4] A. D. Force. Concept for robotic and autonomous systems. Technical report, Australian Defence Force, November 2020. URL <https://tasdcrc.com.au/wp-content/uploads/2020/12/ADF-Concept-Robotics.pdf>. Pages 15-18, 29-30. Version 1.0, Reference: DPN BN9939583. Accessed: 2024-08-21.
- [5] A. Army. Robotic & autonomous systems strategy v2.0. Technical report, Australian Army, August 2022. URL <https://researchcentre.army.gov.au/sites/default/files/Robotic%20and%20Autonomous%20Systems%20Strategy%20V2.0.pdf>. Page 6. Accessed: 2024-08-21.
- [6] E. Commission. Annex to the commission implementing decision on the financing of the european defence fund and adoption of the work programme for 2024 - part ii, with amendments to previous work programmes (2021, 2022, 2023). https://defence-industry-space.ec.europa.eu/document/download/77afc5d2-30b2-4f3f-917c-2c4368eb18e3_en?filename=EDF%202024%20Call%20Topic%20Descriptions.pdf, March 2024. Pages 126-137. Accessed: 2024-08-21.
- [7] M. Pinho. Devastator software. <https://github.com/mppinhoo/Devastator>, 2025. Accessed: July 3, 2025.
- [8] DFRobot. Fit0493 dc motor with encoder. <https://www.dfrobot.com/product-1477.html>, 2025. Accessed: February 15, 2025.
- [9] DFRobot. Devastator tank mobile platform (metal dc gear motor) sku: Rob0128. https://wiki.dfrobot.com/Devastator_Tank_Mobile_Platform__Metal_DC_Gear_Motor__SKU_R0B0128, 2025. Accessed: February 15, 2025.

- [10] DFRobot. Micro dc motor with encoder. <https://www.dfrobot.com/product-1462.html>, 2025. Accessed: February 15, 2025.
- [11] DFRobot. 7a dual dc motor driver sku: Dri0041. https://wiki.dfrobot.com/7A_Dual_DC_Motor_Driver_SKU__DRI0041#target_4, 2025. Accessed: February 15, 2025.
- [12] Q. Zheng, X. Tian, Z. Yu, Y. Ding, A. Elhanashi, S. Saponara, and K. Kpalma. Mobilerat: A lightweight radio transformer method for automatic modulation classification in drone communication systems. *Drones*, 7(10), 2023. ISSN 2504-446X. doi: 10.3390/drones7100596. URL <https://www.mdpi.com/2504-446X/7/10/596>.
- [13] Auvideo. J121 carrier board for jetson tx2. <https://auvideo.eu/product/j121-70718/>, 2025. Accessed: February 15, 2025.
- [14] N. Corporation. *Jetson TX2 Developer Kit User Guide*, 2019. URL https://developer.download.nvidia.com/embedded/L4T/r32-3-1_Release_v1.0/jetson_tx2_developer_kit_user_guide.pdf. Accessed: Feb. 15, 2025.
- [15] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza. *Introduction to Autonomous Mobile Robots*. The MIT Press, 2nd edition, 2011. ISBN 0262015358.
- [16] J. Vargas, S. Alsweiss, O. Toker, R. Razdan, and J. Santos. An overview of autonomous vehicles sensors and their vulnerability to weather conditions. *Sensors*, 21(16), 2021. ISSN 1424-8220. doi: 10.3390/s21165397. URL <https://www.mdpi.com/1424-8220/21/16/5397>.
- [17] C. MacLeod, R. Summan, G. Dobie, and S. Pierce. Quantifying and improving laser range data when scanning industrial materials. *IEEE Sensors Journal*, 16:7999–8009, 11 2016. doi: 10.1109/JSEN.2016.2601822.
- [18] Hokuyo Automatic Co., Ltd. Hokuyo automatic co., ltd. - urg-04lx-ug01 specifications, 2021. URL <https://www.hokuyo-aut.jp/search/single.php?serial=166>. Accessed: 2024-08-28.
- [19] Hokuyo Automatic Co., Ltd. *SICK - Hokuyo URG-04LX-UG01 Specifications*, January 2009. URL https://tribotix.com/wp-content/uploads/2018/04/SPECS-URG-04LX-UG01_C-42-3635_.doc.pdf. Accessed: 2024-08-28.
- [20] Arduino. *Arduino Duemilanove*, 2010. Available online: <https://docs.arduino.cc/retired/boards/arduino-duemilanove> (accessed on 15 September 2024).
- [21] Texas Instruments. *AWR1642: 77-GHz Single-Chip Radar Sensor with Integrated DSP and MCU*, 2024. URL <https://www.ti.com/product/AWR1642>. Accessed: 2024-08-28.
- [22] Texas Instruments. *AWR1642BOOST: AWR1642 Evaluation Module (AWR1642BOOST)*, 2024. URL <https://www.ti.com/tool/AWR1642B00ST#description>. Accessed: 2024-08-28.
- [23] Logitech. *Brio 4K HDR Webcam*, 2024. URL <https://www.logitech.com/en-eu/products/webcams/brio-4k-hdr-webcam.960-001106.html>. Accessed: 2024-09-02.

- [24] Dassault Systèmes. *SolidWorks*, 2024. Version 2024. Available: <https://www.solidworks.com/> (accessed on 27 May 2025).
- [25] M. Quigley, B. Gerkey, K. Conley, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Ng. Ros: an open-source robot operating system. In *Proc. of the IEEE Intl. Conf. on Robotics and Automation (ICRA) Workshop on Open Source Robotics*, Kobe, Japan, may 2009.
- [26] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. doi: 10.1126/scirobotics.abm6074. URL <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [27] D. Merkel et al. Docker: lightweight linux containers for consistent development and deployment. *Linux j*, 239(2):2, 2014.
- [28] H. Kam, S.-H. Lee, T. Park, and C.-H. Kim. Rviz: a toolkit for real domain data visualization. *Telecommunication Systems*, 60:1–9, 10 2015. doi: 10.1007/s11235-015-0034-5.
- [29] F. Inc. Foxglove: Build better robotics. <https://foxglove.dev/product>. Accessed: 2024-08-23.
- [30] Open Source Robotics Foundation. Solidworks urdf exporter. https://github.com/ros/solidworks_urdf_exporter. Accessed: 2024-09-15.
- [31] M. S. P. De Melo, J. G. da Silva Neto, P. J. L. Da Silva, J. M. X. N. Teixeira, and V. Teichrieb. Analysis and comparison of robotics 3d simulators. In *2019 21st Symposium on Virtual and Augmented Reality (SVR)*, pages 242–251. IEEE, 2019.
- [32] E. Rohmer, S. P. N. Singh, and M. Freese. V-rep: A versatile and scalable robot simulation framework. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1321–1326, 2013. doi: 10.1109/IROS.2013.6696520.
- [33] N. Koenig and A. Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, volume 3, pages 2149–2154, 2004.
- [34] A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar, et al. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627*, 2018.
- [35] J. Newans. ros_arduino_bridge - ros package for arduino communication. https://github.com/joshnewans/ros_arduino_bridge. Accessed: 2024-09-15.
- [36] N. Strawa, D. Ignatyev, A. Zolotas, and A. Tsourdos. On-line learning and updating unmanned tracked vehicle dynamics. *Electronics*, 10:187, 01 2021. doi: 10.3390/electronics10020187.
- [37] S. Chitta, E. Marder-Eppstein, W. Meeussen, V. Pradeep, A. Rodríguez Tsouroukdissian, J. Bohren, D. Coleman, B. Magyar, G. Raiola, M. Lüdtke, and E. Fernández Perdomo. ros_control: A generic and simple control framework for ros. *The Journal of Open Source Software*, 2017. doi: 10.21105/joss.00456. URL <http://www.theoj.org/joss-papers/joss.00456/10.21105.joss.00456.pdf>.

- [38] ROS Contributors. *Joint State Broadcaster Documentation*, 2024. Available: https://control.ros.org/rolling/doc/ros2_controllers/joint_state_broadcaster/doc/userdoc.html (accessed on 27 May 2025).
- [39] ROS Contributors. *robot_state_publisher*, 2024. Available: https://wiki.ros.org/robot_state_publisher (accessed on 27 May 2025).
- [40] Buzzology. *diffdrive_arduino - differential drive robot controller with arduino*. https://github.com/Buzzology/diffdrive_arduino/tree/main. Accessed: 2024-09-15.
- [41] Y. Wang, X. Li, J. Zhang, S. Li, Z. Xu, and X. Zhou. Review of wheeled mobile robot collision avoidance under unknown environment. *Science Progress (1933-)*, 104(3):pp. 1–26, 2021. ISSN 00368504, 20477163. URL <https://www.jstor.org/stable/27155270>.
- [42] R. . N. S. Contributors. *Navigation2: Collision monitor*. https://github.com/ros-navigation/navigation2/tree/main/nav2_collision_monitor, 2025. Accessed: February 15, 2025.
- [43] V. Kumar. *Turtlebot3 obstacle avoidance ros2*. <https://github.com/vinay06vinay/Turtlebot3-Obstacle-Avoidance-ROS2/tree/main>, 2025. Accessed: February 15, 2025.
- [44] O. S. R. Foundation. *twist_mux package overview - ros 2 documentation*. https://index.ros.org/p/twist_mux/#humble-overview, 2024. Accessed: 2025-04-28.
- [45] T. Bailey, J. Nieto, J. Guivant, M. Stevens, and E. Nebot. Consistency of the ekf-slam algorithm. In *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3562–3568, 2006. doi: 10.1109/IROS.2006.281644.
- [46] M. Montemerlo, S. Thrun, D. Koller, B. Wegbreit, et al. Fastslam: A factored solution to the simultaneous localization and mapping problem. *Aaai/iaai*, 593598:593–598, 2002.
- [47] S. Thrun and M. Montemerlo. The graph slam algorithm with applications to large-scale mapping of urban structures. *The International Journal of Robotics Research*, 25(5-6):403–429, 2006.
- [48] M. Mielle, M. Magnusson, and A. J. Lilienthal. A comparative analysis of radar and lidar sensing for localization and mapping. In *2019 European Conference on Mobile Robots (ECMR)*, pages 1–6, 2019. doi: 10.1109/ECMR.2019.8870345.
- [49] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós. Orb-slam: A versatile and accurate monocular slam system. *IEEE Transactions on Robotics*, 31(5):1147–1163, 2015. doi: 10.1109/TRO.2015.2463671.
- [50] J. Engel, T. Schöps, and D. Cremers. Lsd-slam: Large-scale direct monocular slam. In D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, editors, *Computer Vision – ECCV 2014*, pages 834–849, Cham, 2014. Springer International Publishing. ISBN 978-3-319-10605-2.

- [51] G. Grisetti, C. Stachniss, and W. Burgard. Improved techniques for grid mapping with rao-blackwellized particle filters. *IEEE Transactions on Robotics*, 23(1):34–46, 2007. doi: 10.1109/TRO.2006.889486.
- [52] S. Kohlbrecher, O. von Stryk, J. Meyer, and U. Klingauf. A flexible and scalable slam system with full 3d motion estimation. In *2011 IEEE International Symposium on Safety, Security, and Rescue Robotics*, pages 155–160, 2011. doi: 10.1109/SSRR.2011.6106777.
- [53] K. Konolige, G. Grisetti, R. Kümmerle, W. Burgard, B. Limketkai, and R. Vincent. Efficient sparse pose adjustment for 2d mapping. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 22–29, 2010. doi: 10.1109/IROS.2010.5649043.
- [54] W. Hess, D. Kohler, H. Rapp, and D. Andor. Real-time loop closure in 2d lidar slam. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1271–1278, 2016. doi: 10.1109/ICRA.2016.7487258.
- [55] M. Labbé and F. Michaud. Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation. *Journal of Field Robotics*, 36: 416 – 446, 2018. URL <https://api.semanticscholar.org/CorpusID:83459364>.
- [56] M. Filipenko and I. Afanasyev. Comparison of various slam systems for mobile robot in an indoor environment. In *2018 International Conference on Intelligent Systems (IS)*, pages 400–407. IEEE, 2018.
- [57] R. Yagfarov, M. Ivanou, and I. Afanasyev. Map comparison of lidar-based 2d slam algorithms using precise ground truth. In *2018 15th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, pages 1979–1983. IEEE, 2018.
- [58] X. Fan, Y. Wang, and Z. Zhang. An evaluation of lidar-based 2d slam techniques with an exploration mode. *Journal of Physics: Conference Series*, 1905:012021, 05 2021. doi: 10.1088/1742-6596/1905/1/012021.
- [59] K. Trejos, L. Rincón, M. Bolaños, J. Fallas, and L. Marín. 2d slam algorithms characterization, calibration, and comparison considering pose error, map accuracy as well as cpu and memory usage. *Sensors*, 22:6903, 09 2022. doi: 10.3390/s22186903.
- [60] S. Macenski and I. Jambrecic. Slam toolbox: Slam for the dynamic world. *Journal of Open Source Software*, 6(61):2783, 2021. doi: 10.21105/joss.02783. URL <https://doi.org/10.21105/joss.02783>.
- [61] E. Piazza, P. U. Lima, and M. Matteucci. Performance models in robotics with a use case on slam. *IEEE Robotics and Automation Letters*, 7(2):4646–4653, 2022. doi: 10.1109/LRA.2022.3151162.
- [62] M. A. V. Torres, A. Braun, and A. Borrmann. Occupancy grid map to pose graph-based map: Robust bim-based 2d-lidar localization for lifelong indoor navigation in changing and dynamic environments. *arXiv preprint arXiv:2308.05443*, 2023.

- [63] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, and P. Wang. Path planning techniques for mobile robots: Review and prospect. *Expert Systems with Applications*, 227:120254, 2023. ISSN 0957-4174. doi: <https://doi.org/10.1016/j.eswa.2023.120254>. URL <https://www.sciencedirect.com/science/article/pii/S095741742300756X>.
- [64] J. R. Sánchez-Ibáñez, C. J. Pérez-del Pulgar, and A. García-Cerezo. Path planning for autonomous mobile robots: A review. *Sensors*, 21(23), 2021. ISSN 1424-8220. doi: 10.3390/s21237898. URL <https://www.mdpi.com/1424-8220/21/23/7898>.
- [65] A. Loganathan and N. S. Ahmad. A systematic review on recent advances in autonomous mobile robot navigation. *Engineering Science and Technology, an International Journal*, 40: 101343, 2023. ISSN 2215-0986. doi: <https://doi.org/10.1016/j.jestch.2023.101343>. URL <https://www.sciencedirect.com/science/article/pii/S2215098623000204>.
- [66] M. N. A. Wahab, S. Nefti-Meziani, and A. Atyabi. A comparative review on mobile robot path planning: Classical or meta-heuristic methods? *Annual Reviews in Control*, 50:233–252, 2020. ISSN 1367-5788. doi: <https://doi.org/10.1016/j.arcontrol.2020.10.001>. URL <https://www.sciencedirect.com/science/article/pii/S1367578820300675>.
- [67] N. A. I. Ruslan, N. H. Amer, K. Hudha, Z. A. Kadir, S. A. F. M. Ishak, and S. M. F. S. Dardin. Modelling and control strategies in path tracking control for autonomous tracked vehicles: A review of state of the art and challenges. *Journal of Terramechanics*, 105:67–79, 2023. ISSN 0022-4898. doi: <https://doi.org/10.1016/j.jterra.2022.10.003>. URL <https://www.sciencedirect.com/science/article/pii/S0022489822000714>.
- [68] A. Al-Mayyahi, W. Wang, and P. Birch. Path tracking of autonomous ground vehicle based on fractional order pid controller optimized by pso. In *2015 IEEE 13th International Symposium on Applied Machine Intelligence and Informatics (SAMII)*, pages 109–114. IEEE, 2015.
- [69] A. D. Sabiha, M. A. Kamel, E. Said, and W. M. Hussein. Ros-based trajectory tracking control for autonomous tracked vehicle using optimized backstepping and sliding mode control. *Robotics and Autonomous Systems*, 152:104058, 2022. ISSN 0921-8890. doi: <https://doi.org/10.1016/j.robot.2022.104058>. URL <https://www.sciencedirect.com/science/article/pii/S0921889022000276>.
- [70] T. Hiramatsu, S. Morita, M. Pencelli, M. Niccolini, M. Ragaglia, and A. Argiolas. Path-tracking controller for tracked mobile robot on rough terrain. *International Journal of Electrical and Computer Engineering*, 13(2):59–64, 2019.
- [71] Y. Li, J. Yu, X. Guo, and J. Sun. Path tracking method of unmanned agricultural vehicle based on compound fuzzy control. In *2020 IEEE 9th Joint International Information Technology and Artificial Intelligence Conference (ITAIC)*, volume 9, pages 1301–1305. IEEE, 2020.
- [72] J. Tao, H. Liu, Y. Li, H. Guan, J. Liu, and H. Chen. Design of trajectory tracking controller of unmanned tracked vehicles based on torque control. In *2021 IEEE International Conference on Unmanned Systems (ICUS)*, pages 85–92. IEEE, 2021.

- [73] D. Li, S. Wu, Y. Zhao, Z. Li, and J. Gong. A hierarchical path tracking method for high-speed unmanned tracked vehicle. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 38–43. IEEE, 2021.
- [74] Q. Yao, Y. Tian, Q. Wang, and S. Wang. Control strategies on path tracking for autonomous vehicle: State of the art and future challenges. *IEEE Access*, PP:1–1, 08 2020. doi: 10.1109/ACCESS.2020.3020075.
- [75] M. Rokonzaman, N. Mohajer, S. Nahavandi, and S. Mohamed. Review and performance evaluation of path tracking controllers of autonomous vehicles. *IET Intelligent Transport Systems*, 15(5): 646–670, 2021.
- [76] S. Macenski, F. Martín, R. White, and J. Ginés Clavero. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. URL <https://github.com/ros-planning/navigation2>.
- [77] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson. From the desks of ros maintainers: A survey of modern capable mobile robotics algorithms in the robot operating system 2. *Robotics and Autonomous Systems*, 168:104493, Oct. 2023. ISSN 0921-8890. doi: 10.1016/j.robot.2023.104493. URL <http://dx.doi.org/10.1016/j.robot.2023.104493>.
- [78] ROS Navigation2 Contributors. Tuning guide. <https://docs.nav2.org/tuning/index.html>, October 2024. Accessed: October 20, 2024.
- [79] K. Zheng. *ROS Navigation Tuning Guide*, pages 197–226. Springer Cham, 07 2021. ISBN 978-3-030-75471-6. doi: 10.1007/978-3-030-75472-3_6.
- [80] J. M. Santos, D. Portugal, and R. P. Rocha. An evaluation of 2d slam techniques available in robot operating system. In *2013 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, pages 1–6, 2013. doi: 10.1109/SSRR.2013.6719348.
- [81] Y. Zhang, T. Zhang, and S. Huang. Comparison of ekf based slam and optimization based slam algorithms. In *2018 13th IEEE Conference on Industrial Electronics and Applications (ICIEA)*, pages 1308–1313, 2018. doi: 10.1109/ICIEA.2018.8397911.
- [82] Y. K. Tee and Y. C. Han. Lidar-based 2d slam for mobile robot in an indoor environment: A review. In *2021 International Conference on Green Energy, Computing and Sustainable Technology (GECOST)*, pages 1–7, 2021. doi: 10.1109/GECOST52368.2021.9538731.
- [83] C. M. de Lisboa. Lisboa munisig - interactive map viewer. Online. URL <https://websig.cm-lisboa.pt/MuniSIG/visualizador/index.html?viewer=LxInterativa.LXi>. Accessed: March 2, 2025.
- [84] Q.-Y. Zhou, J. Park, and V. Koltun. Open3d: A modern library for 3d data processing, 2018. URL <https://arxiv.org/abs/1801.09847>.

- [85] F. V. P. Gonçalves. Simultaneous Localisation and Mapping with a Tracked Mobile Robot. Master's thesis, Instituto Superior Técnico, Nov. 2022.
- [86] T. M. Dar and R. G. Longoria. Slip estimation for small-scale robotic tracked vehicles. In *Proceedings of the 2010 American Control Conference*, pages 6816–6821, 2010. doi: 10.1109/ACC.2010.5531638.
- [87] J. Han, F. Wang, and Y. Wang. A control method for the differential steering of tracked vehicles driven independently by a dual hydraulic motor. *Applied Sciences*, 12:6355, 06 2022. doi: 10.3390/app12136355.

Appendix A

Tracked Differential Drive Kinematics

Tracked differential drive kinematic models are called slip kinematic models and mainly differentiate from the simplified kinematic model by incorporating a track slip rate component i to provide an adjustment to the wheel velocities, such as in (A.1) and (A.2) formulated by Dar and Longoria [86]:

$$\dot{x} = \frac{r}{2} [\omega_L(1 - i_L) + \omega_R(1 - i_R)] \quad (\text{A.1})$$

$$\omega = \frac{r}{b} [\omega_R(1 - i_L) - \omega_L(1 - i_R)] \quad (\text{A.2})$$

where i_L and i_R are the left and right side track slip, respectively. This track slip rate component i can then be calculated differently, depending on the slip model. Since the track slips when the sprocket angular velocity exceeds the longitudinal velocity, Dar and Longoria [86] calculate the track slip rate using:

$$i = \frac{r\omega - V}{\max(r\omega, V)} \quad (\text{A.3})$$

where V is the actual forward speed of the track, r is the pitch circle of the sprocket, and ω is the angular speed of the sprocket. This formulation relies on the assumption that the actual forward speed of the track can be measured.

Other models take into account terrain properties to estimate slippage. Han et al. [87] utilizes the Bekker theory of land locomotion which states that the track slip rate when the vehicle is in motion can be calculated using the following equation:

$$i = -\frac{K}{L} \ln \left(\frac{F}{F_{\max}} \right) \quad (\text{A.4})$$

where K is the ground shear deformation coefficient, L is the track grounding length and F is the traction. The obvious downside of such models is that in most realistic applications the terrain is not uniform, causing the model to take into account parameters that will not be correct throughout all of the robot's activity.

